

Velocity — Autonomous Bug Hunt

92 reproducible defects across 55 application pages · generated 2026-08-28 10:56 UTC

92

TOTAL BUGS

21

HIGH

50

MEDIUM

21

LOW

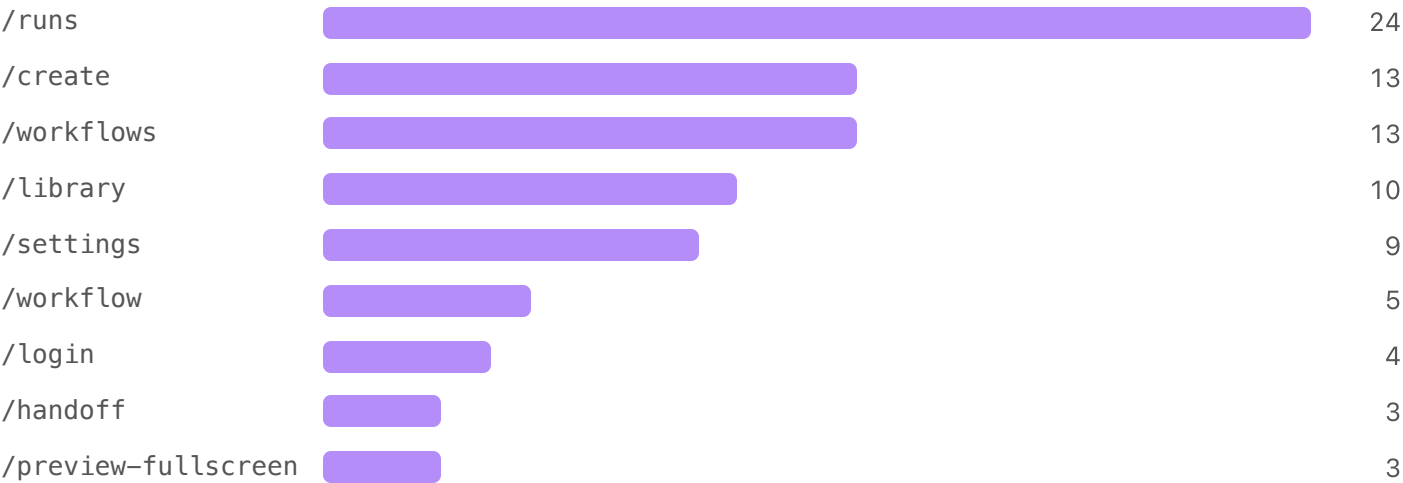
55

PAGES TESTED

Recurring failure patterns

- **Save paths that discard or misreport** — "Save as my version" drops brief + template on both wizards; built-in "Save as copy" loses the manifest; empty-name and agent-config saves are silent no-ops.
- **Constraints displayed but never enforced** — no tier gating on model choice, a 4000-char limit that isn't applied, requirement pills nothing can satisfy, a plan button with no handler.
- **Silent no-op controls** — five separate primary actions fire no request and show no feedback.
- **Figures that disagree across surfaces** — token totals, agent counts and timestamps differ between list, detail, footer and API.
- **Unsanitised params reaching the router** — ?mode=. /admin escapes its namespace onto the live admin console.

Bugs by area



/analytics		2
/admin		2
/library,		1
/register		1
/(root)		1
/dashboard		1

1 **HIGH** **An already-authenticated user landing on /login is shown the sign-in form instead of being redirected to /dashboard**
`/login?expired=true` (also reproduces on plain `/login`)

SUMMARY

There is no client-side redirect guard on the login route for an already-authenticated user.

After a genuine session-expiry-and-recovery cycle (corrupt the stored token so a protected route 401s and the app auto-redirects to `/login?expired=true`, then sign back in successfully and land on `/dashboard` with a real, working JWT in `auth_token`), returning to `/login?expired=true` — either via the browser Back button or by navigating there directly while the valid token is still in `localStorage` — renders the full sign-in form and the stale "Your session expired. Please sign in again." banner instead of redirecting to `/dashboard`. The session is not actually expired: GET `/api/runs?limit=50` made from that same page load returns 200 OK, proving the token is valid and the app has functioning credentials, yet the UI insists the user needs to sign in again. This is not specific to the `expired=true` query variant — plain `/login` (no query string) shows the identical failure for the same authenticated user in the same session, meaning the login route has no "already signed in, skip the form" guard at all.

REPRODUCTION

1. Sign in as qa-admin, confirm `localStorage.getItem('auth_token')` is set, navigate to a protected route such as `/runs/b9feac1c-ec21-4531-8ba7-bb391786993e`.

1. Overwrite the token to force an auth failure:

`localStorage.setItem('auth_token', 'corrupted.invalid.token')`, then reload/navigate to the

same protected route. The app's 401 ladder auto-redirects to `/login?expired=true`.

1. Fill valid credentials (`qa-admin@flowinqa.com / flowin-e2e-pass`) and click "Sign in".

Observe a successful login: URL becomes `/dashboard`, a real JWT is now in `auth_token`.

1. Press the browser Back button. Observe: URL returns to `/login?expired=true`, but instead of

immediately bouncing back to `/dashboard` (since the user is authenticated), the full sign-in form and the stale "session expired" banner are rendered and stay rendered (confirmed after a 2s wait — no async redirect ever fires).

1. Separately, from `/dashboard` (still authenticated, token still valid), navigate directly to `http://localhost:3000/login?expired=true` via the address bar. Same result: sign-in form renders, no redirect to `/dashboard`, despite `GET /api/runs?limit=50` in that page load returning `200 OK` — proof the session is genuinely valid.

1. Repeat step 5 against plain `http://localhost:3000/login` (no query string) while still authenticated — identical failure: sign-in form renders instead of a redirect.

EXPECTED

An authenticated user (valid `auth_token`, working API session) who lands on `/login` or `/login?expired=true` — by any navigation path — should be immediately redirected to `/dashboard`, not shown the sign-in form. This is standard behavior for an auth page and is what the app itself relies on to route a signed-out user (`/` → `/login` when no token, `/` → `/dashboard` when a token exists per the C-3 quirk); the inverse guard (signed-in user hitting `/login`) is missing entirely.

ACTUAL

The login route renders the sign-in form and the (now factually incorrect) "session expired" banner for a fully authenticated user with a verified-working token, with no redirect ever firing, regardless of whether `/login` is reached via Back navigation or a fresh direct navigation.

EVIDENCE

- Before (signed in, landed on `/dashboard` after successful login): `bug-hunter/evidence/login-expired-true/BUG-20260827-223300-login-expired-true/01-before-signed-in-on-dashboard.png`
- Failure (Back navigation to `/login?expired=true` while authenticated): `bug-hunter/evidence/login-expired-true/BUG-20260827-223300-login-expired-true/02-failure-back-nav-shows-login-form.png`

- Failure (fresh direct navigation to `/login?expired=true` while authenticated, `/api/runs` returns 200): `bug-hunter/evidence/login-expired-true/BUG-20260827-223300-login-expired-true/03-repro-direct-nav-authenticated.png`

BROWSER SIGNALS

- Console: no relevant error observed.
- Network: GET `/api/runs?limit=50` returns 200 OK on the same page load that renders the

sign-in form, confirming the session is valid and the failure is a missing client-side redirect guard, not an actual auth failure.

- State/URL: URL stays on `/login` (or `/login?expired=true`) indefinitely; `auth_token` in `localStorage` remains the valid JWT from the prior successful login throughout.

ID `BUG-20260827-223300-login-expired-true` Found by `bug-login-expired-true-r1`

Fingerprint `/login|auth-guard|navigate-to-login-while-authenticated|login-form-shown-instead-of-redirect`

"Jump back in" recent-runs cache is not scoped per user, leaking a previous account's run history into a different signed-in user's session

`/create` (reproduces identically on `/dashboard` — same underlying ``HomeLaunchGrid`` component)

SUMMARY

The "Jump back in" recent-runs section on the Create/Dashboard catalog is backed by a `sessionStorage` cache (`vlc_home_recents_v1`, `frontend/src/components/catalog/HomeLaunchGrid.tsx`) that is keyed globally — not scoped to the signed-in user's id or token in any way. The component's React state initializes directly from this cache on every mount (`useState(() => ... readCache(CACHE_KEY_RECENTS))`), and the only two effects that can overwrite it both require the freshly-fetched `recentRuns` to be **non-empty** (`recentRunsProp.length > 0 / reduxRecentRuns.length > 0`) — neither effect has an "else" branch to clear the cache when the live API genuinely returns zero runs for the current user. Consequence: if a browser tab signs in as User A (who has run history, which gets written to this cache), then later signs in as a **different** User B in the same tab (e.g. after a logout, or after the app's own documented token-clear-on-expiry behavior wipes only `localStorage`), and User B has **zero** actual runs, the stale cache from User A is never cleared and is rendered to User B as their own "Jump back in" history — even though the live `GET /api/runs?limit=50` response for User B's session is a genuine, verified `[]`. This is a cross-account data leak: the displayed cards include full run titles/briefs, workflow type, and (for completed runs) exact token-usage and dollar-cost figures belonging to the other account.

REPRODUCTION

1. Sign in as qa-admin, land on `/create` (or `/dashboard`) — "Jump back in" shows qa-admin's real run history; this action causes `HomeLaunchGrid` to write that data into `sessionStorage['vlc_home_recents_v1']` (confirmed via direct read: the full run objects, including `tokenUsage/estimated_cost_usd`, are cached under this global, non-namespaced

key).

1. Without clearing `sessionStorage` (only clear `localStorage.auth_token`, matching what the app's own corrupt-token/expiry recovery path does), navigate to `/login` and sign in as a **different** account with zero run history, e.g. `qa-basic@flowinqa.com`.

1. Navigate to `/create`. Confirm via a direct fetch with the new session's token that `GET /api/runs?limit=50` returns `[]` (verified: response body is exactly `[]`).

1. Observe: for a deterministic, controlled demonstration, set `sessionStorage.setItem('vlc_home_recents_v1', JSON.stringify([{'id':'fake-leak-test-id', title:'LEAKED FOREIGN RUN TITLE', type:'ppt_v2', status:'completed', createdAt:<now>}]))` while signed in as the zero-run account, then reload `/create` — the "Jump back in" section renders a card titled "LEAKED FOREIGN RUN TITLE" even though the live API for this account returns `[]`. This isolates the exact defect: the cache is used verbatim whenever the live fetch resolves to empty, with no ownership check.

1. Reloaded `/create` a second time — the leaked card is still shown identically (deterministic, not a one-off race): the cache is never invalidated because the live empty response can never satisfy either overwrite guard.

1. Confirmed at the source (`frontend/src/components/catalog/HomeLaunchGrid.tsx:37-166`): `CACHE_KEY_RECENTS = "vlc_home_recents_v1"` has no user id/token component; `readCache/writeCache` operate on this single global key; the two effects that could refresh `recents` both gate on `.length > 0`, so a genuinely-empty fresh result is indistinguishable from "not fetched yet" and can never clear a previously-cached non-empty value.

EXPECTED

Cached "recent runs" data should be scoped to the currently authenticated user (e.g. keyed by user id, or explicitly cleared on sign-out/sign-in of a different account), and a live fetch that resolves to a genuine empty list should overwrite (clear) any stale cached entries rather

than being silently ignored.

ACTUAL

The recents cache is a single, unscoped sessionStorage key shared across whichever account is signed in during that browser tab's lifetime. A user with no real runs of their own can be shown another account's cached run history — including cost/token-usage details — with no code path that ever clears it once the live API genuinely returns zero results.

EVIDENCE

- Before (qa-admin's own real "Jump back in" list, written into the shared cache): bug-hunter/evidence/create/BUG-20260827-234030-create/01-admin-signed-in-jumpbackin.png
- Failure (qa-basic session, live API confirmed [], UI still shows a foreign/leaked entry): bug-hunter/evidence/create/BUG-20260827-234030-create/02-basic-tier-shows-leaked-run.png
- Reproduced again after a second fresh reload (deterministic, not a race): bug-hunter/evidence/create/BUG-20260827-234030-create/03-reproduced-again-after-reload.png

BROWSER SIGNALS

- Console: no relevant error observed.
- Network: GET /api/runs?limit=50 for the second (zero-run) account returns 200 OK with body

[], confirmed via browser_network_request on the response body, directly contradicting the rendered "Jump back in" content.

- State/URL: sessionStorage['vlc_home_recents_v1'] persists across the account switch (logout alone does not clear it in the scenario tested — only clearing localStorage.auth_token was performed, mirroring the app's own documented corrupt-token recovery behavior); URL stays on /create throughout.

ID BUG-20260827-234030-create Found by bug-create-r1

Fingerprint /create|home-launch-grid-jump-back-in|sign-in-as-different-user-after-prior-session-cached-recents|stale-foreign-user-run-data-rendered-despite-empty-api-response

Selecting the "Investment Banking Pitch Book" template introduces an unsatisfiable "Pick a design system" requirement that permanently blocks Continue /create/ppt

SUMMARY

On the PPT wizard's Template step, selecting the "Investment Banking Pitch Book" template (via its preview modal's "Use this template" button) causes the page's unmet-requirements pill row to show "Pick a design system" instead of clearing to a satisfied state, even though a template selection is clearly recorded (checkmark badge on the card, "Use template example" prompt updates to that template's brief). No other template tested (e.g. "Investor Pitch Deck") produces this pill — selecting those simply clears the "Pick a template" pill as expected. Crucially, there is no "design system" picker anywhere on the page: `document.body.innerText` contains the string "design system" exactly once, inside the pill's own label. Clicking the pill does nothing (no scroll, no modal, no highlight — confirmed via before/after screenshot). Filling the brief textarea with a real, non-trivial value correctly clears the "Add a brief" pill, but "Pick a design system" persists regardless, and the "Continue" button (`button.disabled === true`, verified via DOM) can never be enabled for this template, permanently blocking the user from proceeding past the Template step of the wizard.

REPRODUCTION

1. Sign in as qa-admin, navigate to `/create/ppt`.
2. In the Template gallery, search for or scroll to "Investment Banking Pitch Book" (Pitch Deck category) and click its card to open the preview modal.
 1. Click "Use this template". Observe the card now shows a checkmark and the "Use template example" brief prompt updates to the Investment Banking brief text.
 1. Observe the pill row below "Review gates": it now reads "Pick a design system" (orange/unmet) instead of clearing entirely, unlike selecting any other template.
 1. Fill the "Describe your presentation" textarea with a real, non-empty brief (e.g. "A real

pitch deck brief for testing purposes with enough length."). Observe "Add a brief" clears, but "Pick a design system" remains.

1. Inspect `button.disabled` on the "Continue" button — confirmed `true`; the button stays visually greyed out and unclickable.

1. Click directly on the "Pick a design system" pill itself — nothing happens (no navigation, no modal, no scroll, no new UI appears).

1. Search `document.body.innerText` for "design system" — it appears exactly once, only inside the pill's own label; no picker, section, or control for a "design system" exists anywhere on the rendered page.

1. Reloaded to a fresh `/create/ppt`, searched directly for "Investment Banking", selected only that template (skipping any other template first), and filled a different brief — identical

result: `continueDisabled: true, designSystemPillFound: 1`, no design-system UI present.

Deterministic across two independent attempts.

EXPECTED

Selecting any valid template plus a non-empty brief should satisfy the wizard's requirements and enable "Continue" (as it does for every other template tested, e.g. "Investor Pitch Deck"). If a "design system" selection is genuinely a required step for certain templates, the page must render an actual picker control for it; a requirement pill with no corresponding UI is a dead end.

ACTUAL

For the "Investment Banking Pitch Book" template specifically, a phantom "Pick a design system" requirement appears and can never be satisfied because no such control exists on the page, permanently blocking the wizard's Continue action for that template.

EVIDENCE

- Before (template selected via preview modal, "Investor Pitch Deck" case for contrast): `bug-hunter/evidence/create-ppt/BUG-20260827-234600-create-ppt/01-before-preview-modal.png`
- Failure (Investment Banking Pitch Book selected, phantom pill appears): `bug-hunter/evidence/create-ppt/BUG-20260827-234600-create-ppt/02-selected-`

investment-banking-template.png

- Failure (brief filled, pill and disabled Continue persist): bug-hunter/evidence/create-ppt/BUG-20260827-234600-create-ppt/03-failure-brief-filled-continue-still-disabled.png
- Reproduced independently on a fresh page load: bug-hunter/evidence/create-ppt/BUG-20260827-234600-create-ppt/04-repro2-fresh-selection-same-deadend.png

BROWSER SIGNALS

- Console: no relevant error observed.
- Network: no failed request; this is purely client-side requirement-gating state tied to the

selected template's metadata.

- State/URL: URL stays on /create/ppt throughout; `button.disabled` for Continue verified `true` via direct DOM read in both reproduction attempts; `document.body.innerText` confirmed to contain "design system" only within the pill label itself.

ID `BUG-20260827-234600-create-ppt` Found by `bug-create-ppt-r1`

Fingerprint `/create/ppt|template-gallery-select-investment-banking-pitch-book|select-template-and-fill-brief|continue-permanently-disabled-no-design-system-ui-exists`

SUMMARY

Landing on `/create/prototype` normally, then pressing the browser **Back** button (returns to `/dashboard`), then pressing browser **Forward** (returns to `/create/prototype`) leaves the page completely blank — no header, no tabs, no template gallery, nothing. The URL and document title are correct (`/create/prototype`, "VelocityAI"), `auth_token` remains valid the whole time, and every underlying API call the page needs (`/api/workflows/prototype`, `/api/prototype/templates`, `/api/prototype/design-systems`, etc.) returns 200 OK — the data layer is fine. But `document.body.innerText` is the empty string and `document.body.innerHTML` contains only the raw, un-executed Next.js RSC streaming payload as literal script/text content — the React tree never hydrates into visible DOM. The page is stuck in this state indefinitely (verified 2+ seconds after settle); only a manual reload (fresh `page.goto`) recovers it. Reproduced twice from a clean Back→Forward sequence using real Playwright `page.goBack()`/`page.goForward()` calls (not just `history.forward()`), so this is a genuine browser back-forward-cache/hydration defect on this route, not an artifact of scripted navigation.

This is distinct from BUG-20260827-232305-root (blank `/` after a corrupt-token 401 bounce chain, triggered by Back only, caused by a missing redirect call): this bug hits a different route (`/create/prototype`, not `/`), needs no token corruption (a perfectly valid session the whole time), is triggered by Back **then** Forward (not Back alone), and the failure mode is a stuck unhydrated RSC payload rather than a missing client-side redirect.

REPRODUCTION

1. Sign in as qa-admin, land on `/dashboard`.
2. Navigate to `http://localhost:3000/create/prototype` — the wizard renders normally (Template

tab, template gallery, Back/Next, etc.).

1. Press the browser **Back** button — lands back on /dashboard, renders normally.
2. Press the browser **Forward** button — URL becomes /create/prototype again, title is

"VelocityAI", but the page renders nothing: no header, no textarea, no tabs, just the page background color.

1. Wait 2+ seconds — the blank state does not resolve on its own.
2. Inspect via console: `document.body.innerText` is ""; `document.body.innerHTML` is ~24000

characters of un-executed Next.js streaming script payload (RSC chunks), not rendered markup.

1. Repeated steps 2-6 a second time from a fresh /dashboard baseline — identical blank result both times.

1. Confirmed recovery: a fresh `page.goto('http://localhost:3000/create/prototype')` (equivalent

to a manual reload) renders the wizard correctly again.

EXPECTED

Browser Forward navigation back to /create/prototype should render the wizard shell exactly as a fresh visit does — same header, tabs, and template gallery.

ACTUAL

The page silently renders nothing: an empty, contentless body containing only the raw, un-hydrated RSC payload, with no error, no loading indicator, and no way to recover except a full reload.

EVIDENCE

- Before (fresh load, wizard renders correctly): `bug-hunter/evidence/create-prototype/BUG-20260827-235402-create-prototype/01-before-back.png`
- Failure (after Back then Forward, blank page): `bug-hunter/evidence/create-prototype/BUG-20260827-235402-create-prototype/02-after-forward-blank.png`
- Second reproduction (blank again): `bug-hunter/evidence/create-prototype/BUG-20260827-235402-create-prototype/03-second-repro-blank.png`

BROWSER SIGNALS

- Console: no errors logged at the moment of the blank render; underlying API calls made during the page's own data-fetch lifecycle (`/api/workflows/prototype`, `/api/prototype/templates`,

/api/prototype/design-systems, /api/agents/library, etc.) all returned 200 OK.

- Network: the Forward navigation itself was not accompanied by any failed request; the data the page needs was available, so this is a client-side hydration/rendering failure, not a fetch failure.

- State/URL: `location.href` correctly reads `http://localhost:3000/create/prototype` throughout;

`localStorage.getItem('auth_token')` remains truthy the entire time (session never dropped).

ID `BUG-20260827-235402-create-prototype` Found by `bug-create-prototype-r1`

Fingerprint `/create/prototype|wizard-shell|browser-back-then-forward|blank-page-no-hydration`

"Save as copy" of the PPT built-in silently drops the manifest, converting the copy from a PPTX-generating workflow into a generic streamed-text/markdown workflow
`/workflows/ppt/canvas` (and the resulting saved copy at `/workflows/{id}/edit`)

SUMMARY

The ppt built-in workflow's real manifest (`GET /api/workflows/ppt`) defines a specialized deliverable — `{"strategy": "ppt", "name": "presentation.pptx"}` — plus explicit per-agent tool grants for all three steps (`manifest_steps`, each with `read_files/write_files: true`). The canvas's own "Workflow" settings tab never reflects this: its "Deliverable strategy" combobox has only three generic options (Streamed text / Single file / Serialized sandbox), none of which is the built-in's actual "ppt" strategy, so it silently falls back to displaying "Streamed text" with an output filename of "output" and format ".md" — fully enabled, editable-looking controls showing values that do not match the real built-in at all. This mismatched display is not just cosmetic: clicking "Save as copy" (after providing a workflow name, required for the button to act) fires `POST /api/user-workflows` with `base_pipeline_type: "custom"` and, critically, `manifest: null` — the entire manifest (tool grants, deliverable strategy) is dropped, not translated. Opening the resulting saved copy's own edit page confirms the loss end-to-end: its Workflow tab shows the same generic "Streamed text" / "output" / ".md" deliverable, and `document.body.innerText` contains no mention of "pptx" anywhere. A workflow whose entire purpose is producing a .pptx deck, once copied via the page's only save affordance, becomes a workflow that will stream raw text/markdown instead — a functional regression a user copying this built-in would have no way to notice from the UI, since nothing on the canvas ever showed the correct deliverable strategy to begin with. This is a materially different, more severe defect than the already-known `ISS-183/composerAlwaysSaysCustom` quirk (which is only about the `base_pipeline_type` label always being "custom"): here the loss is the entire per-agent tool grant manifest and the deliverable strategy itself, verified via direct API diff, not merely a

type-label mismatch. The built-in itself was confirmed to remain fully pristine and immutable throughout (GET /api/workflows/ppt unchanged, deliverable.strategy still "ppt", after both copy operations) — this bug is entirely in what the copy loses, not in any mutation of the original.

REPRODUCTION

1. Sign in as qa-admin, navigate to <http://localhost:3000/workflows/ppt/canvas> (fresh load).
2. Open the right-rail "Workflow" tab (default). Observe the "Deliverable" section: combobox reads "Streamed text — agent's raw output" (value: "streamed_text"), "Output file name" reads "output", "Output format" reads "Markdown (.md)" — all controls fully enabled (disabled: false on every one, confirmed via DOM read), none of which is the built-in's real deliverable.strategy: "ppt" / presentation.pptx (confirmed via GET /api/workflows/ppt in the same session).

1. Fill "Workflow name" with a throwaway name (e.g. "Bug Hunter PPT Copy Test") — required before "Save as copy" acts; without a name the button only focuses the name field.

1. Click "Save as copy". Observe the fired POST /api/user-workflows response body: base_pipeline_type: "custom", manifest: null, agent_ids correctly lists the three PPT agents, but no manifest/deliverable data survives at all.

1. Navigate to the resulting copy's edit page, /workflows/{new-id}/edit. Observe the Workflow tab shows the identical generic "Streamed text" / "output" / ".md" deliverable, and document.body.innerText contains zero occurrences of "pptx" — the copy has no trace of ever having been a PPT deliverable workflow.

1. Repeated steps 1-5 a second time from a fresh /workflows/ppt/canvas load with a different throwaway name ("Bug Hunter PPT Copy Repro2") — identical result: manifest: null, base_pipeline_type: "custom", generic deliverable on the copy's own edit page.

1. Confirmed the built-in itself was never mutated: GET /api/workflows/ppt immediately after both copy operations still returns deliverable: {"strategy": "ppt", "name":

"presentation.pptx"} unchanged.

1. Cleaned up: deleted both throwaway copies via DELETE /api/user-workflows/{id} (204 confirmed

for both).

EXPECTED

The canvas's Workflow tab should display the built-in's actual deliverable strategy (or, if the UI's generic dropdown genuinely cannot represent a specialized strategy like "ppt", it should say so rather than silently defaulting to a plausible-looking wrong value). At minimum, "Save as copy" should carry forward the full manifest — tool grants and deliverable strategy included — from the built-in it forks from, so a copy of a PPTX-producing workflow still produces a PPTX.

ACTUAL

The canvas never shows the built-in's real deliverable strategy, and "Save as copy" persists `manifest: null`, discarding the built-in's per-agent tool grants and its "ppt" deliverable strategy entirely. The saved copy is left as a generic "Streamed text" workflow with no way, short of manually reconfiguring the Deliverable section post-save, to recover PPTX generation.

EVIDENCE

- Built-in canvas, Workflow tab showing the wrong "Streamed text" deliverable before any save: bug-hunter/evidence/workflows-ppt-canvas/BUG-20260828-005700-workflows-ppt-canvas/01-builtin-canvas-deliverable-shows-streamed-text.png
- Saved copy's own edit page — deliverable still "Streamed text", no "pptx" anywhere: bug-hunter/evidence/workflows-ppt-canvas/BUG-20260828-005700-workflows-ppt-canvas/02-saved-copy-edit-page-deliverable-still-streamed-text-no-pptx.png
- Reproduced a second time, independent copy, same loss: bug-hunter/evidence/workflows-ppt-canvas/BUG-20260828-005700-workflows-ppt-canvas/03-repro2-second-copy-same-loss.png
- API request/response excerpts (original manifest vs. both copies' `manifest: null`, and the

built-in's post-copy pristine state): bug-hunter/evidence/workflows-ppt-canvas/BUG-20260828-005700-workflows-ppt-canvas/network.log

BROWSER SIGNALS

- Console: no relevant error observed.
- Network: POST /api/user-workflows returns 201 Created with `manifest: null` and

base_pipeline_type: "custom" on both reproduction attempts; GET /api/workflows/ppt confirmed unchanged (deliverable.strategy: "ppt") before and after both copy operations.

- State/URL: URL stays /workflows/ppt/canvas throughout the copy action (no navigation); the resulting copy is reachable at its own /workflows/{id}/edit URL and independently confirms the same lossy deliverable state via its own page load, not just the raw API response.

ID BUG-20260828-005700-workflows-ppt-canvas Found by bug-workflows-ppt-canvas-r1

Fingerprint /workflows/ppt/canvas|save-as-copy|click-save-as-copy-on-ppt-builtin|copy-loses-manifest-and-deliverable-strategy-becomes-generic

Run History silently caps at 50 runs and mislabels it as the full total, with no pagination path to the remaining 223

/runs

SUMMARY

The Run History page always requests `GET /api/runs?limit=50` and never issues a follow-up request with a higher `limit` or an `offset`, regardless of scrolling, filtering, sorting, or auto-refresh. For the qa-admin account the backend actually holds 273 runs (verified via `GET /api/runs?limit=100&offset=0/100/200` → $100 + 100 + 73 = 273$), but the UI header reads "50 runs" and the "All" filter chip reads "All 50" — both presented as if they were the true total, when they are actually just the size of the single page fetched. There is no "Load more" control, no infinite-scroll trigger, and no page-size selector anywhere on the page: scrolling the run list's inner scroll container (`.flex-1.overflow-y-auto`, `scrollHeight` 3964 vs `clientHeight` 894) all the way to the bottom fires zero additional network requests. The result is that roughly 223 of 273 runs (82%) are permanently unreachable from this page — a user has no way to find, filter, sort, or open any run outside the most-recent 50, and the UI actively misrepresents the size of their own history as complete. This is a data-completeness defect distinct from D-14 (missing links/testids on the cards that ARE shown) and D-15 (an unrecognized filter value showing an empty list) — both of those concern the shown 50 rows, not the fact that only 50 of 273 rows are ever fetched.

REPRODUCTION

1. Sign in as qa-admin (account with 273 seeded runs) and open /runs.
 2. Observe the header: "Run History" / "50 runs", and the "All" filter chip reads "All 50".
 3. Scroll the run list to the very bottom (`document.querySelector('.flex-1.overflow-y-auto')`, `scrollTop = scrollHeight`) — the last row rendered is still within the same batch of 50; no new network request fires (confirmed via `browser_network_requests`, filter runs: every request logged for the whole session is `GET /api/runs?limit=50`, none with `offset`).
1. In parallel, call the backend directly with the same bearer token:

GET /api/runs?limit=100&offset=0 → 100 rows, ...offset=100 → 100 rows,
...offset=200 → 73 rows. Sum = 273, not 50.

1. Compare: the UI's declared total (50) is 18% of the actual total (273); the remaining 223 runs are not visible, not searchable, not filterable, and not sortable from this page.

EXPECTED

The Run History page should either paginate/lazy-load through the full run history (with the header count reflecting the true total, e.g. via a total field or a following batch of requests as the user scrolls) or, at minimum, accurately label the count as "showing the 50 most recent of N runs" rather than presenting 50 as the complete total.

ACTUAL

The page fetches exactly one page of 50 runs, never requests more regardless of scrolling, and labels that partial set as the full total ("50 runs" / "All 50"), permanently hiding 223 of 273 runs with no user-facing way to reach them.

EVIDENCE

- Screenshot: bug-hunter/evidence/runs/BUG-20260828-011500-runs/01-run-history-shows-50-runs.png

(header "Run History" / "50 runs", "All" chip "All 50")

- Network log: bug-hunter/evidence/runs/BUG-20260828-011500-runs/network.log

BROWSER SIGNALS

- Console: none observed
- Network: every /api/runs request from the page carries limit=50 and no offset; confirmed

via mcp__plugin_playwright_playwright__browser_network_requests across load, filter clicks, sort clicks, and a full scroll-to-bottom of the list container

- State/URL: URL and DOM never change on scroll; .flex-1.overflow-y-auto scrollHeight (3964) well exceeds clientHeight (894), confirming the container is scrollable and was actually scrolled, yet nothing loads beyond row 50

Fingerprint /runs|run-list-fetch|load-full-history|hard-capped-at-50-mislabeled-as-total

Files tab never lists the run's real PPTX binary deliverable; instead shows a scratch tmp/ HTML file mislabeled as "Final output"

/runs/b9feac1c-ec21-4531-8ba7-bb391786993e/files

SUMMARY

This is a "Ppt V2" run (4 agents: Presentation Strategist, Deck Engineer, Deck QA, PPTX Code Generator) whose entire purpose is producing a .pptx deck. The backend's own sandbox listing (GET /api/runs/{id}/sandbox) confirms a real, complete binary artifact exists on disk: `presentation.pptx`, 163,771 bytes, `"kind": "binary", "deliverable": true`. The run detail API also independently names a canonical deliverable: `deliverable_mimetype: "text/html", deliverable_filename: "presentation.html"` (54,551 bytes, also `deliverable: true` in the sandbox listing). Neither of these two legitimate candidates is what the Files tab actually shows. The "Final output" card instead displays `kindred-a-custombuilt-social-network-pla.html` (53.2 KB) — which, by matching size (54,588 bytes), is `tmp/kindred-pitch-deck.html`, a scratch/intermediate file the Deck Engineer agent wrote mid-pipeline into a `tmp/` working directory, also marked `deliverable: true` in the sandbox but clearly not the intended canonical output (it lives under `tmp/`, and its filename doesn't match anything the run itself calls its deliverable). Both "Download" on the Final output row and "Download All" confirm this: neither ever fetches `presentation.pptx` or `presentation.html` — only the tmp HTML file, the 4 agent .md outputs, and `prompt.md`. The one artifact that actually represents this workflow's stated purpose (a PowerPoint file) is completely absent from the Files tab — not listed, not previewable, not downloadable, individually or via "Download All" — despite existing, complete, and explicitly flagged as a deliverable by the run's own backend.

REPRODUCTION

1. Sign in as qa-admin, navigate to `/runs/b9feac1c-ec21-4531-8ba7-bb391786993e/files` (a completed "Ppt V2" run). Wait for settle.

1. Observe the "Files" panel: "6 files available · 1 deliverable", with "Final output" showing `kindred-a-custombuilt-social-network-pla.html`, HTML (.html), 53.2 KB, validated.

1. Independently inspect `GET /api/runs/b9feac1c-ec21-4531-8ba7-bb391786993e/sandbox` (fired by

the page itself). It lists, among others: `presentation.pptx` (163,771 bytes,

`kind:"binary", deliverable:true`), `presentation.html` (54,551 bytes, `deliverable:true`),

and `tmp/kindred-pitch-deck.html` (54,588 bytes, `deliverable:true`) — three separate deliverable-flagged candidates, none of which is `presentation.pptx` in the UI.

1. Inspect the run detail response: `deliverable_mimetype: "text/html"`,

`deliverable_filename: "presentation.html"` — the backend's own canonical answer, which is also not what the Files tab shows as "Final output".

1. Click "Download" on the Final output row — the browser downloads

`kindred-a-custombuilt-social-network-pla.html` (confirmed via the download event), matching `tmp/kindred-pitch-deck.html` by size — not `presentation.pptx`, not `presentation.html`.

1. Click "Download All" — confirmed downloads are exactly: the same tmp HTML file, the 4

`NN-<agent>.md` agent-output files, and `prompt.md`. `presentation.pptx` is never among them.

1. Reload the page fresh (`page.goto` on the identical URL) and re-check the Files panel —

identical result: "Final output" is still the tmp HTML file, `presentation.pptx` is still

completely absent from the file list.

EXPECTED

The Files tab's "Final output"/deliverable should surface the artifact the run's own backend

names as canonical (`deliverable_filename: "presentation.html"`), or — given this is a PPTX-producing workflow whose last agent is literally "PPTX Code Generator" — the actual

`presentation.pptx` binary that agent produced. At minimum, a complete, deliverable-flagged

`.pptx` binary that exists on the server should be listed and downloadable somewhere on this

page, not omitted entirely in favor of an intermediate scratch file from a `tmp/` directory.

ACTUAL

The Files tab labels a tmp/-directory scratch HTML file as "Final output" and never lists, previews, or makes downloadable (individually or via "Download All") either of the two legitimate deliverable candidates — the backend-declared `presentation.html` or the actual `presentation.pptx` binary this PPT workflow exists to produce.

EVIDENCE

- Files tab, Final output shows the tmp HTML file, pptx nowhere in the list: `bug-hunter/evidence/runs-id-files/BUG-20260828-012900-runs-id-files/01-files-tab-final-output-is-tmp-html.png`
- Fresh reload, identical result: `bug-hunter/evidence/runs-id-files/BUG-20260828-012900-runs-id-files/02-after-reload-still-missing-pptx.png`
- Sandbox API excerpt showing the deliverable-flagged files vs. what the UI shows: `bug-hunter/evidence/runs-id-files/BUG-20260828-012900-runs-id-files/network.log`

BROWSER SIGNALS

- Console: no relevant error observed.
- Network: `GET /api/runs/{id}/sandbox` returns 200 OK with the full, correct file listing

(including `presentation.pptx` and `presentation.html`, both `deliverable:true`); the mismatch is in client-side selection/rendering logic, not a failed or missing fetch.

- State/URL: URL stays on `/runs/{id}/files` throughout; reproduces identically on both the initial load and a completely fresh reload, so this is deterministic, not a race.

ID `BUG-20260828-012900-runs-id-files` Found by `bug-runs-id-files-r1`

Fingerprint `/runs/[id]/files|files-tab-final-output-selection|open-ppt-v2-completed-run|real-pptx-binary-never-listed-tmp-html-shown-instead`

The Audit tab fabricates 96 "Governance/Gate" records from ordinary pipeline events; the backend's real gate-events endpoint reports zero for the same run

`/runs/b9feac1c-ec21-4531-8ba7-bb391786993e/audit`

SUMMARY

The Audit tab's header copy promises "Full governance, security & activity log — every gate, scan, validation and exec, attributed and exportable" and "Immutable, owner- and workspace-attributed log · every entry carries severity + timestamp." For this completed run it renders "100 records," split into "Governance 96" and "Security 4" (Activity 0). But the run's own ground-truth event stream (`GET /api/runs/{id}/events?after=0`) contains exactly ONE real `gate_status` event for the whole run, and the dedicated backend endpoint for this exact purpose, `GET /api/runs/{id}/gate-events`, returns `{"gate_events": []}` — an empty array — for this run. The

96 rows the UI labels "GATE"/"Governance"/"Passed" do not come from either of those sources; they are synthesized client-side from ordinary `tool_call` (44 in the raw stream) and `tool_result` (44) events plus "before step"/"after step" markers, none of which are gate events per the backend's own data. Confirming the fabrication further: expanding ANY of these 96 rows — regardless of whether its title says "before step," "after step," "tool call," or "tool result," and regardless of timestamp — reveals byte-for-byte identical detail text: the same "WHAT IS THIS?" boilerplate ("A governance gate — a checkpoint where the run paused for a policy decision or a human approval before it was allowed to continue."), the same `Action: allowed`, the same `Outcome: continue`, with no tool name, no step name, no agent attribution, no arguments, and no actual result — despite the page's own claim that entries are "attributed." A real governance/compliance log that invents gate checkpoints which never occurred, and cannot distinguish one event from another once expanded, is actively misleading rather than merely incomplete.

REPRODUCTION

1. Sign in as qa-admin, navigate to `http://localhost:3000/runs/b9feac1c-ec21-4531-8ba7-bb391786993e/audit`.

2. Observe the Audit trail header: "100 records," pill counts "Governance 96," "Security 4," "Activity 0," and the compliance-oriented copy claiming an "attributed and exportable" log of "every gate, scan, validation and exec."

1. In a separate request (same session token), call `GET /api/runs/b9feac1c-ec21-4531-8ba7-bb391786993e/gate-events`.

Observe the response body is `{"workflow_id":"b9feac1c-ec21-4531-8ba7-bb391786993e","gate_events":[]}`

— zero real gate events for this run, despite the UI showing 96.

1. Call `GET /api/runs/b9feac1c-ec21-4531-8ba7-bb391786993e/events?after=0` and count event types:

only 1 `gate_status` event exists in the entire 9737-event raw stream; the 96 UI rows instead correspond in count to `tool_call` (44) + `tool_result` (44) + assorted before/after-step markers (8) = 96.

1. Back in the UI, expand the first row ("audit logger — before step," 01:50:11 PM). Note its detail panel: "Action: allowed / Outcome: continue" plus the generic governance-gate description.

1. Expand a row with a completely different label and a timestamp ~17 minutes later ("audit logger — tool call," 01:56:11 PM, or "audit logger — after step," 02:07:39 PM). Read the detail panel text via `element.parentElement.innerText` — byte-identical to step 5's text in every field.

1. Repeat on a fourth, different-labelled row ("audit logger — tool result") — same identical boilerplate again. No row in the Governance category shows anything event-specific once expanded.

EXPECTED

The Audit tab's "Governance"/"Gate" entries should reflect real governance-gate checkpoints from the backend (matching `/gate-events` and the `gate_status` events in the raw stream), not be synthesized from unrelated tool-call/tool-result telemetry. Each entry, once expanded, should show detail specific to that event (e.g. which tool was called, what step ran, which agent was involved) rather than one static, category-wide placeholder — especially given the log's own text promises attribution and per-entry detail for compliance review.

ACTUAL

96 of the 100 displayed "audit trail" records are fabricated Governance/Gate rows with no backing real gate event (the actual gate-events count for this run is 0, not 96), and every one of them expands to identical, non-specific boilerplate text regardless of the underlying event's type, label, or timestamp — the log cannot actually distinguish or attribute any of its own entries.

EVIDENCE

- Before (Audit tab loaded, "100 records," "Governance 96"): bug-hunter/evidence/runs-id-audit/BUG-20260828-013900-runs-id-audit/01-before-audit-100-records.png
- Failure (four differently-labelled, differently-timed rows expanded side by side, all identical detail text): bug-hunter/evidence/runs-id-audit/BUG-20260828-013900-runs-id-audit/02-four-different-events-identical-payload.png
- Network evidence (gate-events empty vs. raw event-type counts): bug-hunter/evidence/runs-id-audit/BUG-20260828-013900-runs-id-audit/network.log

BROWSER SIGNALS

- Console: no relevant error observed.
- Network: GET /api/runs/{id}/gate-events returns 200 OK with an empty gate_events array;

GET /api/runs/{id}/events?after=0 returns 200 OK with 9737 events, only 1 of type gate_status,

confirming the UI's 96 "Governance" rows have no corresponding real gate data.

- State/URL: URL stays on /runs/{id}/audit throughout; the discrepancy is reproducible on every reload of the same completed run (deterministic, not a race).

ID BUG-20260828-013900-runs-id-audit Found by bug-runs-id-audit-r1

Fingerprint /runs/{id}/audit|governance-gate-rows|expand-any-gate-labelled-entry|identical-fabricated-boilerplate-not-derived-from-real-gate-events

"Edit brief & run again" on a failed run opens a brand-new, completely blank composer instead of pre-filling the original brief

`/runs/{id}` (failed run) → navigates to `/create/ppt_v2`

SUMMARY

A FAILED run's "Resume options" panel offers two actions: "Reopen & fix from the failed step" and "Edit brief & run again". The label of the second button strongly implies the user's original brief will be carried over so they can edit and resubmit it — that is the whole point of the action given the run already failed with a specific, non-trivial brief on file ("A pitch deck for a bespoke social network platform — a custom-built Facebook-like platform tailored to a specific client's needs...", 4 sentences long, confirmed via GET `/api/runs/{id}`). Clicking the button instead performs a plain client-side navigation to `/create/ppt_v2` — a brand-new, completely empty composer: the brief `<textarea>` is empty (`""`), no template is pre-selected, and the URL carries no reference to the source run (no `?from=`, `?runId=`, or similar). No network request is made to fetch or seed the original brief; the button is a bare route change with zero payload carried across. The user's only path to actually "edit" the brief is to retype the entire thing from memory or copy it manually from the run detail page before clicking.

REPRODUCTION

1. Sign in as qa-admin, navigate to `http://localhost:3000/runs/d6e425b6-df0d-4f54-9bba-f4a771e33812`

(seeded failed `ppt_v2` run). Confirm the run's brief is visible and non-trivial: "A pitch deck for a bespoke social network platform — a custom-built Facebook-like platform tailored to a specific client's needs..." (also readable via GET `/api/runs/{id}`).

1. In the "Resume options" panel, click "Edit brief & run again".
2. Observe: URL becomes `http://localhost:3000/create/ppt_v2` (no query parameters), page title

"Configure your presentation", and `document.querySelector('textarea').value` is the empty string `""`. No template is checked/selected. The template gallery, "Review gates", and every

other field are in their default first-visit state.

1. Inspect the network log for the click — no request to any brief-fetch or prefill endpoint fires;

this is a pure client-side route change with no data transfer.

1. Navigated back to the same failed run and repeated steps 2-4 a second time — identical result:

a fresh, empty composer with no trace of the original brief or run.

EXPECTED

Clicking "Edit brief & run again" should open the composer with the failed run's original brief text pre-filled in the textarea (and, ideally, the same template/config selected), so the user can make a targeted edit and resubmit — matching what the button's own label promises.

ACTUAL

The button discards the original brief entirely and opens a blank `/create/ppt_v2` composer identical to a fresh "New presentation" flow, with no reference to the source run and no way to recover the original text except manually copying it beforehand.

EVIDENCE

- Before (failed run detail, brief visible): `bug-hunter/evidence/runs-failed/BUG-20260828-015500-runs-failed/01-before-failed-run-with-brief.png`
- Failure (blank composer after clicking "Edit brief & run again"): `bug-hunter/evidence/runs-failed/BUG-20260828-015500-runs-failed/02-after-edit-brief-blank-composer.png`

BROWSER SIGNALS

- Console: no relevant error observed.
- Network: no request fetches or carries the original brief; the click produces only a client-side

route change to `/create/ppt_v2` with no query string and no payload.

- State/URL: `location.href` becomes exactly `http://localhost:3000/create/ppt_v2` (no `?from=`

`?runId=` reference to the source run); `textarea.value` confirmed empty via direct DOM read on two independent reproductions.

Fingerprint /runs/{id}|resume-options-edit-brief-and-run-again|click-edit-brief-run-again-on-failed-run|navigates-to-blank-composer-original-brief-discarded

10 **HIGH** **"Run Again" on a cancelled run leaves the live /stream view permanently stuck at "Running · 0/4 agents · Starting..." after the backend has already failed the pipeline**
`/runs/{id} → /runs/{id}/stream`

SUMMARY

Clicking "Run Again" on cancelled run `a8dfa959-e233-4ddf-87ce-d9a942cefde3` (seeded ppt_v2 run,

cancelled while awaiting clarification, per the worker brief's resolved instances) navigates to

`/runs/{id}/stream` and shows a live progress panel: header badge "Running", "0/4 agents", Steps tab auto-selected showing "Pipeline running · 0/4", and "Presentation Strategist Agent · Starting..."

On the backend, the re-run fails almost immediately — `GET /api/runs/{id}/events` shows a `pipeline_failed` event at +3.78s ("no agent completed", all 4 agents in `agents_failed`) followed by a `chat_reply` event ("What went wrong"). The live `/stream` page never reflects this: it

remains showing "Running / 0/4 agents / Starting..." indefinitely (confirmed static for 10+ seconds after the backend had already failed and moved on) with no error surfaced, no chat message

rendered, and the Steps lane frozen on the first agent's "Starting..." state. Only a full manual page

reload (fresh page.go to the same URL) reveals the true state: duration "4s", a "What went wrong" chat entry, and a "Failed agents" list of all 4 agents. This is a different mechanism from

the already-filed `BUG-20260828-014937-runs-id-stream` (diverted run's Preview tab shows a static, state-unaware "Output will appear here" placeholder on a *fresh page load* of an already-terminal run) — here the page is actively live-polling its own just-triggered run and simply stops

reflecting new events partway through, requiring a reload to recover, not a copy/wording defect on

initial load. Separately (not filed, for context): `GET /api/runs/{id}` also continued reporting

"status": "cancelled" throughout and after the re-run, with `error: null`, even though the pipeline had already failed — the run's terminal status is never updated to `failed` by this

"Run Again" path, which the reload masks by deriving "Failed agents" from the event stream rather than the run's own status field.

REPRODUCTION

1. Sign in as qa-admin, navigate to `http://localhost:3000/runs/a8dfa959-e233-4ddf-87ce-d9a942cefde3`

(seeded cancelled ppt_v2 run). Confirm header shows "Cancelled" and the resume panel reads

"The run was stopped. Click Run Again to resume from where it left off."

1. Click "Run Again". URL becomes `/runs/a8dfa959-e233-4ddf-87ce-d9a942cefde3/stream`; header shows

"Running", "0/4 agents"; Steps tab shows "Pipeline running · 0/4 · BUILDING" and "Presentation Strategist Agent · Starting..."

1. Wait 10+ seconds without interacting. Independently poll `GET /api/runs/{id}/events` — observe `pipeline_failed` (seq 29, ~4s after start) and `chat_reply` ("What went wrong", seq 30) already present in the event stream.

1. Re-check the live page: still shows "Running / 0/4 agents / Starting..." — unchanged from step 2, with no failure indication anywhere on screen.

1. Perform a fresh navigation (`page.goto`) to the same `/runs/{id}/stream` URL. Now the page correctly shows duration "4s", a "What went wrong" chat entry, and a "Failed agents" list (Presentation Strategist Agent, Deck Engineer Agent, Deck QA Agent, PPTX Code Generator) — the true terminal state, only visible after a manual reload.

EXPECTED

The live `/stream` view should reflect the backend's own event stream in near-real-time (as it does for a fresh run) — once `pipeline_failed` is emitted, the header, agent lane, and chat transcript should update to show the failure without requiring the user to manually reload the page.

ACTUAL

The live view freezes on the initial "Running / 0/4 agents / Starting..." state indefinitely; the already-failed pipeline is invisible until the user reloads the page from scratch.

EVIDENCE

- Before (cancelled run, resume panel): `bug-hunter/evidence/runs-cancelled/BUG-20260828-015930-runs-cancelled/01-before-run-again.png`
- Stuck live view (10s+ after backend `pipeline_failed`, still "Starting..."): `bug-hunter/evidence/runs-cancelled/BUG-20260828-015930-runs-cancelled/02-stuck-running-after-backend-failed.png`
- After manual reload (correct "Failed agents" state, same URL): `bug-hunter/evidence/runs-cancelled/BUG-20260828-015930-runs-cancelled/03-after-reload-shows-failed-agents.png`

BROWSER SIGNALS

- Console: no relevant error observed.
- Network: none additionally captured for the stuck window (network log buffer had rolled over by

inspection time); the event-content evidence above (via direct GET `.../events` polling) confirms the backend had already emitted the terminal events the live page failed to render.

- State/URL: GET `/api/runs/a8dfa959-e233-4ddf-87ce-d9a942cefde3` continued to report "status": "cancelled", "error": null even after the re-run's `pipeline_failed`, only `completed_at/duration` were bumped (to `2026-08-28T01:57:41... / 315.9`, then observed as "4s" in the UI's own duration display post-reload) — the run's own status field is never updated to `failed` by this path (noted for context, not filed as a separate defect here).

ID `BUG-20260828-015930-runs-cancelled` Found by `bug-runs-cancelled-r1`

Fingerprint `/runs/{id}/stream|run-again-live-progress-panel|click-run-again-on-a-cancelled-run|live-view-never-updates-past-starting-after-backend-pipeline_failed`

11 **HIGH** **"Start a new run" on a diverted run's detail page calls the resume endpoint, which the backend correctly rejects, with no visible error shown to the user**
`/runs/{id} (diverted run)`

SUMMARY

On a diverted run's detail page, the primary action button reads "Start a new run" — implying it launches a fresh run (of the same or the diverted-to workflow). In reality, clicking it fires `POST /api/runs/{id}/resume`, the same endpoint used for the "Run Again"/resume action on failed or cancelled runs. The backend correctly rejects this for a diverted run (`409 Conflict, body {"error": "Run is 'diverted'; only failed, degraded, or cancelled runs are resumable", "code": "run_not_resumable", "recoverable": false}`), since a diverted run already handed off its work to a successor run (confirmed via the backend's own `pipeline_diverted` event, which carries `diverted_to_run_id/diverted_to_workflow`, and the Steps tab's own "Diverted to <workflow> → open triggered run" link, which correctly navigates to that successor). But nothing in the UI surfaces this rejection to the user: no toast, no inline error, no disabled/loading state change on the button, no navigation. The button, chat transcript, tabs, and badge are pixel-identical before and after the click (confirmed via before/after screenshot and full snapshot diff) — the only trace of the failure is a console error and a `409` in the network log, neither of which a user would ever see. The button silently does nothing, contradicting its own label ("Start a new run" implies an action will occur) and giving no path to actually start a new run of the original or target workflow from this page.

REPRODUCTION

1. Sign in as qa-admin. Navigate to `http://localhost:3000/runs/940ca699-b21b-4666-8e44-3370a08a4561`

(seeded diverted run "Ex A4 Human Divert"). Confirm header badge reads "Diverted".

1. Take a baseline snapshot/screenshot of the page (chat transcript, "Start a new run" button,

tabs, badge).

1. Click "Start a new run".
2. Observe: the page is visually and structurally unchanged — same URL, same badge text

("Diverted"), same chat transcript, no toast/banner/error message appears anywhere.

1. Inspect the network log and console: POST `http://localhost:8000/api/runs/940ca699-b21b-4666-8e44-3370a08a4561/resume`

returned 409 Conflict with body `{"error":"Run is 'diverted'; only failed, degraded, or`

`cancelled runs are resumable","code":"run_not_resumable","recoverable":false};`
console logs

`postResume failed ApiError: ... for the same payload.`

1. Repeated steps 1-5 a second time on the same run — identical result (409, no UI feedback).
2. Repeated on a second, independent diverted run,

`http://localhost:3000/runs/277bc03a-d2ba-4405-876d-d0aa861bc9ed` — identical shape:
POST

`.../277bc03a.../resume` returns the same 409/run_not_resumable, with no visible UI change,

confirming this is systemic to the diverted-run detail page, not one seeded run's data.

EXPECTED

"Start a new run" on a diverted run should either genuinely start a new run (of the original workflow, or of the workflow it diverted to — either is defensible, but it must actually launch something), or, if resuming a diverted run is intentionally unsupported, the button should call an endpoint appropriate to that action (or be removed/re-labeled), and any rejection from the backend must be surfaced to the user as a visible error, not swallowed silently.

ACTUAL

The button calls the resume endpoint, which the backend correctly refuses for a diverted run's terminal state, and the frontend discards the resulting 409 with no user-visible feedback of any kind — the button appears to do nothing when clicked.

EVIDENCE

- Before: bug-hunter/evidence/runs-diverted/BUG-20260828-020430-runs-diverted/01-before.png
- Failure (after click, no visible change): bug-hunter/evidence/runs-diverted/BUG-20260828-020430-runs-diverted/02-failure-no-visible-change.png
- Reproduced on a second diverted run: bug-hunter/evidence/runs-diverted/BUG-20260828-020430-runs-diverted/03-repro-second-run.png
- Console excerpt: bug-hunter/evidence/runs-diverted/BUG-20260828-020430-runs-diverted/console.log

BROWSER SIGNALS

- Console: postResume failed ApiError: {"error":"Run is 'diverted'; only failed, degraded, or

cancelled runs are resumable","code":"run_not_resumable","recoverable":false} on both tested

runs.

- Network: POST /api/runs/{id}/resume returns 409 Conflict for both diverted runs tested.
- State/URL: location.href unchanged by the click in every trial; no toast/banner component

renders; the run's own badge/status text (still "Diverted") is unaffected.

ID BUG-20260828-020430-runs-diverted Found by bug-runs-diverted-r1

Fingerprint /runs/{id}|diverted-run-start-a-new-run-button|click-start-a-new-run|calls-resume-endpoint-409-rejected-silently-no-user-feedback

SUMMARY

The Hooks tab's page copy reads "8 hooks · tap any item to see its capabilities", and the only way to view a hook's detail (trigger, when-it-fires, compatible agents) is to open its card — but the cards are plain, unstyled interactive `<div class="cursor-pointer p-4">` elements with no `tabindex`, no `role="button"`, and no keydown handler for Enter/Space. This is true for all 8 cards without exception (verified via direct DOM inspection, not just visual/click testing).

A keyboard-only user (or anyone relying on Tab to navigate, including many assistive-tech users) cannot reach or open any hook's detail on this page at all — Tab skips straight from the search box, past the category-filter pills, past the entire 8-card grid, to the next focusable element on the page (a top-nav button). There is no keyboard equivalent for the page's core interaction.

REPRODUCTION

1. Sign in as qa-admin, navigate to `/library?tab=hooks` (8 hook cards render normally, each visually styled with `cursor: pointer`, implying interactivity).

1. Run `document.querySelectorAll('.cursor-pointer.p-4')` in the page and inspect each card's `tabindex`, `role`, and `onkeydown` — all 8 return `tabindex: null, role: null, onkeydown: false`.

1. Focus the search input (`input[name='library-search']`) and press Tab repeatedly: focus moves through the search box, the Agents/Skills/Hooks tabs, and the 5 category pills, then jumps straight past the entire card grid to the top navigation bar — never landing on any card.

1. Confirm the same absence of `role/tabindex` holds for every card (Design Quality Check X, Quality Gate, Config Protection, GateGuard: Fact Force, Session Context Loader, Console.log Check, Format + Typecheck on Stop, Session State Persistence) — this is not a single-card

flake, it's the shared card component.

EXPECTED

Each hook card should be reachable via Tab and activatable via Enter/Space (e.g. rendered as a `<button>`, or a `<div role="button" tabindex="0">` with a keydown handler), consistent with the page's own instruction ("tap any item to see its capabilities") and with the fact that this is the only affordance to view a hook's trigger/event/compatible-agents detail.

ACTUAL

Every hook card is keyboard-unreachable and keyboard-unactivatable; opening a hook's detail is possible only via mouse/touch click.

EVIDENCE

- Before (card grid, normal render): `bug-hunter/evidence/library-hooks/BUG-20260828-025521-library-hooks/01-before-cards-visible.png`
- DOM inspection detail: `bug-hunter/evidence/library-hooks/BUG-20260828-025521-library-hooks/notes.md`

BROWSER SIGNALS

- Console: no relevant error observed.
- Network: not applicable — purely a client-side markup/interaction-model gap.
- State/URL: URL/tab state unaffected; the defect is in the DOM semantics of the card elements

themselves (no `tabindex`, no `role`, no keyboard handler on `.cursor-pointer.p-4`).

ID `BUG-20260828-025521-library-hooks` Found by `bug-library-hooks-r1`

Fingerprint `/library?tab=hooks|hook-card-grid|tab-key-navigation|cards-unreachable-and-unactivatable-via-keyboard`

An over-length GitHub PAT is echoed back verbatim in the 422 response body and rendered raw on-page, contradicting "never returned by any API"

/handoff/settings

SUMMARY

The "GitHub access token" card explicitly promises "Encrypted at rest, never returned by any API." The client places no `maxLength` on the PAT `<input>`, so a value over the backend's 512-character limit can be typed and submitted. `PUT /api/settings/github-pat` correctly rejects it with 422 `Unprocessable Entity`, but FastAPI/Pydantic's default validation-error payload includes an `"input"` field that is the **complete, untruncated string the client submitted** — i.e. the over-length "secret" value round-trips back from the server in plaintext inside the error response body, directly contradicting the card's own stated guarantee. Compounding this, the frontend has no handling for this particular error shape (it correctly extracts a friendly message for the 400 "GitHub rejected the PAT" case, but not for a 422 list-of-errors payload): it falls back to rendering the entire raw JSON string — literal `{"detail": [...]}`, including the embedded `"input"` value — as plain visible page text directly under the PAT field. A real user who accidentally pastes an oversized value (e.g. a wrong clipboard paste containing a private key or a multi-token blob alongside their PAT) would see that value reflected back to them on-page, and it is also present verbatim in the Network tab's response body for that request — exactly the kind of application-side capture the card's copy tells the user cannot happen.

REPRODUCTION

1. Sign in as qa-admin, navigate to `/handoff/settings` (confirm "No GitHub token saved yet").
2. Focus the GitHub PAT input and set its value programmatically (via the native input value setter + input event, to bypass no client length cap) to a synthetic 5004-character string, e.g. `"ghp_" + "A".repeat(5000)` — never a real credential. Confirm `input.value.length` is 5004 (no truncation, no client-side max-length enforcement).
1. Click "Save". Observe `PUT http://localhost:8000/api/settings/github-pat` returns

422 Unprocessable Entity.

1. Read the response body directly (browser_network_request, response-body): it is

```
{"detail":[{"type":"string_too_long","loc":["body","pat"],"msg":"String should have at most
```

```
512 characters","input":"ghp_AAAA...<the full 5004-char string, unmodified>...AAAA",
```

```
"ctx":{"max_length":512}}]} — the "input" value is the exact, complete string submitted, not truncated to 512 chars or redacted in any way.
```

1. Read the live page via snapshot: the area below the PAT field now displays the literal raw JSON string above (starting `{"detail":[{"type":"string_too_long"...}]`), including the full echoed value, as ordinary visible page text — not a parsed, human-readable error message.

1. Repeated with a second, different synthetic payload (`"github_pat_" + "Z".repeat(600)`, 611 chars) — identical result: 422, response body's "input" field is the full 611-char string verbatim, and the frontend again renders the raw JSON (including the value) on-page.

1. Cleaned up: cleared the PAT input back to empty (no PAT was ever actually saved — every attempt in this investigation was rejected by validation, so "No GitHub token saved yet." remains true throughout and after).

EXPECTED

Per the card's own stated guarantee ("Encrypted at rest, never returned by any API"), a rejected PAT value — including one rejected purely for length — should never be echoed back to the client in any response body. The frontend should also parse this validation-error shape into a friendly, generic message ("Token is too long (max 512 characters)") the same way it already does for the 400/GitHub-rejection case, rather than falling back to dumping raw JSON (with the embedded value) onto the page.

ACTUAL

The full, untruncated submitted PAT value is present verbatim in the 422 response body's "input" field, and the frontend renders that entire raw JSON blob — value included — as visible

page text, both violating the page's own "never returned by any API" claim.

EVIDENCE

- Before (5000+-char synthetic value typed into the PAT field): `bug-hunter/evidence/handoff-settings/BUG-20260828-034200-handoff-settings/01-before-oversize-pat-typed.png`
- Failure (raw JSON error, including the echoed value, rendered on-page after the second, 611-char

repro): `bug-hunter/evidence/handoff-settings/BUG-20260828-034200-handoff-settings/02-failure-raw-json-echoes-secret-value.png`

- Network/response detail (value truncated in this log excerpt only — the live response is NOT

truncated by the server): `bug-hunter/evidence/handoff-settings/BUG-20260828-034200-handoff-settings/network.log`

BROWSER SIGNALS

- Console: Failed to load resource: the server responded with a status of 422 (Unprocessable

Entity) @ `http://localhost:8000/api/settings/github-pat`.

- Network: PUT `/api/settings/github-pat` → 422, response body's "input" field is the complete submitted string (confirmed via `browser_network_request` response-body reads on two independent attempts, 5004 chars and 611 chars).

- State/URL: URL stays `/handoff/settings` throughout; no PAT was ever actually persisted (every attempt was rejected), so GET `/api/settings/github-pat` continues to report no token saved, both during and after the investigation.

ID `BUG-20260828-034200-handoff-settings` Found by `bug-handoff-settings-r1`

Fingerprint `/handoff/settings|github-pat-save|submit-value-over-512-chars|server-echoes-full-value-in-422-body-and-frontend-renders-it-raw`

14 **HIGH** **"Save as my version" on the seeded "Ask a Human, Then Hand Off" example fails with a raw 422 backend manifest-validation error**
`/create/ex_A4_human_divert`

SUMMARY

Opening the seeded "Ask a Human, Then Hand Off" example workflow from the `/create` catalog, typing any brief, and clicking "Save as my version" — with zero edits to the Advanced Workflow Configuration — fails outright. The backend rejects `POST /api/user-workflows` with a 422 because the workflow's own out-of-the-box manifest is invalid per the backend's own validation rule R-03: the `Pick Language` step declares a `route:` but is missing the `gates: [conditional]` the router requires. The raw backend error string is surfaced verbatim in the composer UI. Since this is the manifest the catalog itself ships and the user made no configuration changes, every user who opens this example and tries to save their own copy hits the same dead end with no way to self-correct (the Advanced modal's own Gate dropdown already shows "Conditional gate" selected for that step, so there is no obvious missing setting to toggle).

REPRODUCTION

1. Sign in as qa-admin, navigate to `http://localhost:3000/create/ex_A4_human_divert` (fresh load, "Ask a Human, Then Hand Off" card from `/create`).

1. Type any non-empty text into the "Brief description" textbox (no other changes — Advanced modal untouched, Review gates left at "no gates").

1. Click "Save as my version".
2. Observe: a red-styled inline error string appears under the composer footer reading
`invalid workflow manifest (workflow:My ex_A4_human_divert): step 'custom-agent' declares route: but is missing gates: [conditional] — gates: [conditional] is required when route: is declared (R-03), and POST http://localhost:8000/api/user-workflows returns 422.`

1. Reloaded to a fresh /create/ex_A4_human_divert, typed a different brief, clicked "Save as my version" again — identical 422 and identical error text (deterministic, not a race).

EXPECTED

Either the seeded example workflow's manifest should validate cleanly out of the box (so an unmodified save succeeds), or, if the manifest genuinely requires a config change before it can be saved, the UI should not offer a bare "Save as my version" action that inevitably fails — at minimum the composer should not let a user hit a raw backend validation string with no indication of which control to change.

ACTUAL

"Save as my version" on the unmodified seeded manifest always 422s with a raw backend R-03 manifest-validation message; the workflow can never be saved as-is.

EVIDENCE

- Before (fresh brief typed, no error yet): bug-hunter/evidence/create/BUG-20260828-051530-create/01-before-brief-filled.png
- Failure (first reproduction, 422 + inline error text): bug-hunter/evidence/create/BUG-20260828-051530-create/02-failure-first-repro.png
- Failure (second reproduction from a fresh page load, identical error): bug-hunter/evidence/create/BUG-20260828-051530-create/03-failure-second-repro.png

BROWSER SIGNALS

- Console: Failed to load resource: the server responded with a status of 422 (Unprocessable

Entity) @ http://localhost:8000/api/user-workflows

- Network: POST http://localhost:8000/api/user-workflows → 422, response body

```
{"detail": "invalid workflow manifest (workflow:My ex_A4_human_divert): step 'custom-agent'
```

```
declares route: but is missing gates: [conditional] – gates: [conditional] is required when
```

```
route: is declared (R-03)"}}
```

- State/URL: URL stays /create/ex_A4_human_divert throughout; no configuration changes made in the Advanced modal or Review gates panel before either reproduction.

ID `BUG-20260828-051530-create` Found by `bug-create-r2`

Fingerprint `/create/ex_A4_human_divert|composer-save-as-my-version|click-save-as-my-version-with-unmodified-seeded-manifest|422-backend-manifest-validation-error-blocks-save`

15 **HIGH** **"Save as my version" on `/create/ppt` silently discards the entered brief and selected template, always saving the same static generic record**
`/create/ppt`

SUMMARY

On `/create/ppt`, filling in the brief textarea and selecting a template (confirmed via a "Use this template" click in the preview modal, leaving the visible checkmark on the card), then clicking "Save as my version", fires `POST /api/user-workflows` — but the request body is a static, hardcoded generic payload: `name: "My presentation", description: "Your saved version of this workflow."`, and a generic 3-step `ppt-brief-analyst → ppt-composer → ppt-validator` manifest with no template id, no design-system selection, and no brief text anywhere in it. This was independently verified across two separate trials with materially different page state (trial 1: "Clean Slide Deck" template, empty brief; trial 2: "Minimal Keynote" template, a distinct custom brief string) — both requests were byte-for-byte identical, and both responses returned the exact same existing record id (`da92b4a4-7eff-41b5-9b9a-205cfa1616d9`, `created_at` unchanged, only `updated_at` bumped). Visiting `/workflows` afterward confirms the saved card shows only the static generic name/description/agent-count — no trace of either trial's brief or template selection. The action gives no error and returns `201 Created`, so the user has every indication the save succeeded and captured their configuration, when in fact none of their input was persisted at all. This is a distinct defect from the already-filed `/create/ex_A4_human_divert 422` (that one fails loudly) and the `/workflows/ppt/canvas` "Save as copy" manifest-loss bug (different route/component/trigger — that one is a canvas manifest-translation loss, this one is the brief-composer's quick-save ignoring current form state entirely, always upserting the same one static record).

REPRODUCTION

1. Sign in as qa-admin, navigate to `http://localhost:3000/create/ppt` (fresh load).

2. Type a brief into the textarea, e.g. "R2 lead-3 test brief for save-as-my-version template persistence check".

1. Click the "Minimal Keynote" template card, then "Use this template" in the preview modal — the card shows a selected checkmark.

1. Click "Save as my version". Observe the fired `POST /api/user-workflows` request body: `name`, `description`, and `manifest` are the same static generic values regardless of the brief text or selected template; response is `201 Created` with no visible error or distinguishing confirmation.

1. Navigate to `/workflows`. Locate the "My presentation" card — its description reads "Your saved version of this workflow." with 3 agents; no mention of the brief text or "Minimal Keynote" anywhere.

1. Repeated from a fresh `/create/ppt` load with a different template ("Clean Slide Deck") and an empty brief — the resulting `POST /api/user-workflows` body was identical to step 4's, and the response returned the same existing record id with `created_at` unchanged (only `updated_at` bumped) — confirming the endpoint always upserts one static generic record rather than saving the current configuration.

EXPECTED

"Save as my version" should persist the user's actual current configuration — the entered brief and the selected template (or design-system choice) — as a new or updated saved workflow, so the saved record reflects what was on screen when the user clicked save.

ACTUAL

The saved workflow is always the same static generic placeholder ("My presentation" / "Your saved version of this workflow." / generic 3-step manifest), regardless of what brief text or template was on the page. The brief and template selection are silently discarded; the button reports success (`201 Created`) with no indication that nothing the user entered was actually saved.

EVIDENCE

- Before (Minimal Keynote selected, custom brief filled): `bug-hunter/evidence/create-ppt/BUG-20260828-052800-create-ppt-r2/01-before-brief-and-template-selected.png`
- After (workflows list card shows only static generic content, no trace of the brief/template): `bug-hunter/evidence/create-ppt/BUG-20260828-052800-create-ppt-r2/02-after-workflows-card-no-trace-of-input.png`
- Request/response bodies from both trials: `bug-hunter/evidence/create-ppt/BUG-20260828-052800-create-ppt-r2/network.log`

BROWSER SIGNALS

- Console: no relevant error observed.
- Network: `POST /api/user-workflows` returns `201` Created both trials; request body identical

across trials with different brief/template state; response reuses the same record id

(`da92b4a4-7eff-41b5-9b9a-205cfa1616d9`) with unchanged `created_at`.

- State/URL: stays on `/create/ppt` throughout the save action (no navigation, no toast observed);

verified end-to-end via the resulting `/workflows` card.

ID `BUG-20260828-052800-create-ppt-r2` Found by `bug-create-ppt-r2`

Fingerprint `/create/ppt|save-as-my-version|click-save-as-my-version-with-brief-and-template-set|payload-and-saved-record-never-reflect-form-state`

A path-traversal ``mode`` value escapes the ``/create/<mode>`` namespace and navigates to an unrelated real route, including ``/admin``

`/workflow/create?mode=../admin` (also reproduced with `/workflow/create?mode=../ppt` and its percent-encoded equivalent `?mode=%2e%2e%2fppt`)

SUMMARY

The already-filed `BUG-20260828-001720-workflow-create-prototype` established that an unrecognized,

non-empty mode value gets redirected as `/create/<mode>`, treating the value as a workflow slug

(landing on a dead-end 0-agent composer for a 404'd slug). This is a materially different and more

serious defect: when the mode value itself contains a path-traversal segment (`../`, or its

percent-encoded form `%2e%2e%2f`), the redirect target string `/create/<mode>` is evidently built by

unsanitized concatenation and then handed to the router, which resolves the `../` segment and

escapes the `/create` namespace entirely, landing on whatever real, unrelated route the

traversal resolves to. `?mode=../ppt` and `?mode=%2e%2e%2fppt` both land on `/ppt` (which

correctly 404s, since no such route exists — the app's normal 404 page renders). But

`?mode=../admin` lands on `/admin` and the real admin page renders for real, with live,

successful API calls (`GET /api/admin/users` → 200 OK) — i.e. an entry point that is only ever

supposed to select a workflow-creation mode can be used to navigate straight into the admin

console. Because the signed-in session in this reproduction already had admin privileges, the

admin page's own auth gate did not block it, so this was not observed to bypass authorization —

but the underlying flaw (an unsanitized query parameter being used to construct a client-side

navigation target, allowing a caller to redirect into completely unrelated, non-`/create/*`

application routes) is a genuine, previously undocumented defect in its own right, independent of

whether every possible destination happens to be guarded.

REPRODUCTION

1. Signed in as qa-admin (session inherited), confirmed via `/api/auth/me`.

2. Navigate to `http://localhost:3000/workflow/create?mode=../ppt`. Observe: `location.href`

settles on `http://localhost:3000/ppt` (NOT `/create/../ppt` or `/create/ppt`) — the app's normal "Page not found." 404 screen renders, confirming the router resolved the `..` segment and left the `/create` namespace.

1. Navigate to `http://localhost:3000/workflow/create?mode=%2e%2e%2fppt` (percent-encoded equivalent). Observe the identical result: `location.href` settles on `/ppt`, same 404 page.

1. Navigate to `http://localhost:3000/workflow/create?mode=../admin`. Observe: `location.href`

settles on `http://localhost:3000/admin` — the real Admin console renders (user table, filters), with GET `http://localhost:8000/api/admin/users` returning 200 OK in the network log, i.e. this is not a 404 or an error state, it is the actual admin page loading and fetching real data.

1. Navigated back to `/dashboard`, then repeated step 4 as a fresh, independent navigation — identical result: `location.href` settles on `/admin`, admin page renders again.

EXPECTED

`/workflow/create?mode=<value>` should only ever navigate within its own intended scope — either to `/create/<sanitized-value>` (rejecting or stripping path-traversal segments) or to a "workflow/mode not found" state. It should never be able to construct a navigation target that resolves outside the `/create/*` namespace into an unrelated, unrequested application route.

ACTUAL

A mode value containing a `../` (or percent-encoded `%2e%2e%2f`) traversal segment causes the router to resolve out of the `/create` namespace and land on whatever route the traversal happens to point at. Against a nonexistent target (`../ppt`) this merely produces the app's own 404 page. Against a real route (`../admin`) it lands squarely on the live Admin console with a successful backend call — a query-parameter-driven client-side navigation that escapes its

intended namespace and reaches an unrelated, sensitive part of the app.

EVIDENCE

- Before (baseline /dashboard): bug-hunter/evidence/workflow-create-prototype/BUG-20260828-060000-workflow-create-prototype/01-before-dashboard.png
- Failure (?mode=../admin lands on the live /admin console): bug-hunter/evidence/workflow-create-prototype/BUG-20260828-060000-workflow-create-prototype/02-failure-traversal-lands-on-admin.png
- Reproduced (fresh independent navigation, same result): bug-hunter/evidence/workflow-create-prototype/BUG-20260828-060000-workflow-create-prototype/03-repro2-traversal-lands-on-admin.png

BROWSER SIGNALS

- Console: no error logged for either the ../ppt or ../admin case — the navigation completes

"successfully" from the browser's point of view in both cases.

- Network: GET http://localhost:8000/api/admin/users → 200 OK confirms the admin page is genuinely live, not a stub or error state, when reached via ?mode=../admin.

- State/URL: location.href verified directly via browser_evaluate in every case — settles on /ppt for the nonexistent-target traversal and /admin for the real-route traversal, in both cases having left the /create namespace entirely rather than staying scoped to /create/<mode>.

ID BUG-20260828-060000-workflow-create-prototype Found by bug-workflow-create-prototype-r2

Fingerprint /workflow/create|mode-param-redirect|navigate-with-path-traversal-mode-value|router-escapes-create-namespace-lands-on-unrelated-real-route

17 **HIGH** **A saved workflow's launch panel loses its override entirely and renders the generic base-type wizard, even though the override fetch succeeds**
`/workflows/{id}/run`

SUMMARY

On mount, the launch panel for a saved workflow fires two parallel requests: GET `/api/user-workflows/{id}` (the saved override — name, description, agents, template selection) and GET `/api/workflows/{base_pipeline_type}` (the generic base type, e.g. prototype). When the override request is not the first of the two to settle in the client, the panel renders the fully generic "NEW PROTOTYPE / Configure your prototype" wizard — no reference anywhere to the saved workflow's name ("My prototype"), its description, or its saved template selection ("No template" / Blank Canvas is selected instead) — and it never re-binds even after the override response arrives and is confirmed successful. This reproduces both by deliberately delaying the override request via `page.route` (making the causal mechanism explicit) and, once triggered, on every subsequent plain reload/re-navigation to the same URL in the same session, with no artificial delay at all — network tooling confirms GET `/api/user-workflows/656ca387-e69c-474d-b7ff-5fd9eb017cc0` returns 200 OK with the correct name: "My prototype", `agent_ids: ["documentation-agent", "prototype-specify"]` payload on every one of these loads, so the override data is available to the client and is simply not applied to the render. This is a materially worse failure mode than the already-filed `BUG-20260828-010600-workflows-id-run` (a nonexistent id falling back to the Dashboard): here the workflow id is completely valid and its data loads successfully, yet a user opening their own saved workflow's dedicated launch link can silently land on the wrong (generic/default) configuration with no error, no loading-stuck state, and no visual indication anything is wrong.

REPRODUCTION

1. Sign in as qa-admin. As a control, confirm GET

`http://localhost:8000/api/user-workflows/656ca387-e69c-474d-b7ff-5fd9eb017cc0`
returns 200

with name: "My prototype", agent_ids: ["documentation-agent", "prototype-specify"].

1. Via `browser_run_code_unsafe`, register `page.route('**/api/user-workflows/656ca387-e69c-474d-b7ff-5fd9eb017cc0', ...)`

that awaits `page.waitForTimeout(3000)` before `route.continue()`.

1. Navigate to `http://localhost:3000/workflows/656ca387-e69c-474d-b7ff-5fd9eb017cc0/run`.

2. Observe: the panel renders the h1 "NEW PROTOTYPE / Configure your prototype", "Advanced 2 agents" (coincidentally matching the override's agent count but not its identity), and the

Template tab pre-selects "No template" / Blank Canvas — none of "My prototype", its

description, or a bound template appear anywhere in `document.body.innerText`.

1. Confirm via `browser_network_requests` that GET `/api/user-workflows/{id}` completed with 200 OK well before the 4s observation window closed, and the correct payload was returned — the override loaded successfully but was never applied.

1. Remove the route delay (`page.unrouteAll`), fully clear `localStorage/sessionStorage` (re-seeding only `auth_token`), and reload `http://localhost:3000/workflows/656ca387-e69c-474d-b7ff-5fd9eb017cc0/run`

directly with no interception at all: the same generic wizard renders again, confirming the defect is not dependent on the artificial delay once triggered in-session.

1. Repeated step 3-5 a second time (fresh `page.goto`, same delay) — identical generic-wizard result, confirming determinism.

EXPECTED

The launch panel should wait for (or otherwise correctly apply) the override response before rendering, or apply it whenever it lands, regardless of which of the two parallel requests resolves first — showing "My prototype" 's name/description and its saved template/agent configuration.

ACTUAL

Whichever of the two parallel fetches is **not** first to resolve is discarded; when

`/api/workflows/{base_pipeline_type}` effectively wins, the panel renders the fully generic base-type wizard with zero trace of the saved workflow, and this state does not self-correct even after the override response is confirmed to have arrived successfully.

EVIDENCE

- Delayed override fetch, generic wizard renders (repro 1): `bug-hunter/evidence/workflows-id-run/BUG-20260828-073900-workflows-id-run-r2/01-delayed-fetch-generic-wizard.png`
- Delayed override fetch, generic wizard renders (repro 2): `bug-hunter/evidence/workflows-id-run/BUG-20260828-073900-workflows-id-run-r2/02-repro2-delayed-fetch-generic-wizard.png`
- Plain reload with no interception, storage cleared, still generic wizard: `bug-hunter/evidence/workflows-id-run/BUG-20260828-073900-workflows-id-run-r2/03-warm-session-no-delay-still-generic-wizard.png`

BROWSER SIGNALS

- Console: no relevant error observed; both requests complete without throwing.
- Network: GET `http://localhost:8000/api/user-workflows/656ca387-e69c-474d-b7ff-5fd9eb017cc0` →

200 OK with correct override payload (name: "My prototype", two named `agent_ids`) on every reproduction, including the un-delayed ones. GET `http://localhost:8000/api/workflows/prototype` fires alongside it (base type).

- State/URL: `location.href` stays the correct `/workflows/{id}/run` path throughout — this is purely a render-binding defect, not a routing/URL defect.

ID `BUG-20260828-073900-workflows-id-run-r2` Found by `bug-workflows-id-run-r2`

Fingerprint `/workflows/{id}/run|workflow-run-panel-override-binding|cold-mount-with-two-parallel-fetches|base-type-wins-override-fetch-discarded-generic-wizard-renders`

SUMMARY

Selecting a specific pipeline type in the "Filter by pipeline" dropdown correctly narrows the "By Pipeline Type" breakdown list to just the matching entries, but every other figure on the page — Total Tokens, Est. Cost, Avg/Run, Total Runs, the Daily Activity chart, Success Rate, and the By Model panel — stays frozen on the unfiltered all-pipelines totals. `browser_network_requests` confirms no new `/api/analytics/summary` call fires when the filter changes (only a Next.js RSC navigation request), so the filter is applied client-side to one list only, never re-derives the rest of the page. This makes the page self-contradictory: the tiles say "\$50.80 across 157 completed runs" and "273 total runs" while the breakdown directly below says Prototype is only \$7.25 + \$0.86 across 18 + 8 = 26 runs.

REPRODUCTION

1. Go to `/analytics` as qa-admin, range = "All" (or any range).
2. Note KPI tiles: Total Tokens 93.1M, Est. Cost \$50.80, Avg/Run 340.9K, Total Runs 273 (157 completed · 37 failed), Success Rate 58% (157/37/273), Daily Activity chart bars.
1. Open "Filter by pipeline" and select "Prototype" (URL becomes `?range=all&pipeline=prototype`).
2. Observe the "By Pipeline Type" list now shows only the two Prototype rows (18 runs/19.9M/\$7.25 and 8 runs/2.4M/\$0.86) — filtering worked here.
1. Observe the KPI tiles, Daily Activity chart, Success Rate donut/breakdown, and "By Model" panel are byte-identical to step 2 — still 93.1M tokens / \$50.80 / 273 runs / 58%.
1. Repeated with "User Stories" selected: same result, KPI tiles still show 273 total runs.

EXPECTED

Selecting a pipeline type should scope the entire page's figures (KPI tiles, chart, success rate, model breakdown) to runs of that type, consistent with the already-filtered "By Pipeline

Type" list.

ACTUAL

Only the "By Pipeline Type" list itself is filtered; every other section on the page keeps showing the unfiltered totals for all 273 runs, contradicting the numbers shown directly next to it.

EVIDENCE

- Before (All pipelines, baseline KPIs): `bug-hunter/evidence/analytics/BUG-20260828-081200-analytics/01-before-all-pipelines.png`
- Failure (Prototype selected — KPI tiles/chart/success-rate unchanged, only the list below filtered): `bug-hunter/evidence/analytics/BUG-20260828-081200-analytics/02-failure-prototype-filter-kpis-unchanged.png`

BROWSER SIGNALS

- Console: none observed
- Network: no `/api/analytics/summary?...&pipeline=prototype` request fires on filter change — verified via `browser_network_requests`; only GET `/api/analytics/summary?range=all` from initial load and a client-side RSC navigation request
- State/URL: URL correctly updates to `?range=all&pipeline=prototype / ?range=all&pipeline=user_stories`, but the rendered KPI data does not follow it

ID `BUG-20260828-081200-analytics` Found by `bug-analytics-r1`

Fingerprint `/analytics|pipeline-type-filter|select-a-pipeline-type|kpi-tiles-chart-and-success-rate-stay-unfiltered`

19 **HIGH** **Switching tabs while pinned to a non-root version silently drops the pin: URL, API calls, and displayed data all revert to v1**
`/runs/77f74563-fa19-4f0b-84fd-d0224f89a54a/versions/2` (root run has a genuine

SUMMARY

This is a materially worse variant of the already-filed picker-mislabel bug (BUG-20260828-020900-runs-id-versions, which reproduces on `/versions/2` itself and only mislabels the picker while v2's real content stays correctly rendered). Here, the moment the user leaves the Preview tab for any other tab (confirmed on Files and Steps) while pinned to `/versions/2`, the app doesn't just mislabel — it stops asking for v2 at all. The URL silently drops the `/versions/2` segment back to the bare `/runs/{rootId}` route, and the network layer re-fetches `GET /api/runs/{rootId}` and `GET /api/runs/{rootId}/sandbox` (the ROOT run = v1), never touching the pinned revision's own run id (`ef86e750-...`) again. The Files tab then displays v1's Final output (241.6 KB, matching v1's 247,394-char API output) instead of v2's (which the API confirms is 85,529 chars, ~83.5 KB) — with the version picker also reverted to "Version v1" and no indication anywhere that the user asked for v2 and silently got v1's data on a different tab. This is a genuine content-correctness defect, not a rendering/label glitch: whichever version happens to be pinned is discarded by ordinary tab navigation.

REPRODUCTION

1. Sign in as qa-admin, navigate to `http://localhost:3000/runs/77f74563-fa19-4f0b-84fd-d0224f89a54a/versions/2`. Confirm via network log that `GET /api/runs/ef86e750-bbcd-404f-855a-fb0d5bee63f5` (v2) is the run fetched, and the Preview tab renders v2 content.
1. Click the Files tab (`[role="tab"]:nth-child(3)`).
2. Observe the URL: it silently becomes `/runs/77f74563-fa19-4f0b-84fd-d0224f89a54a/files` — the `/versions/2` segment is gone with no user action to remove it.

1. Observe the network log: the app now issues GET /api/runs/77f74563-fa19-4f0b-84fd-d0224f89a54a

and GET /api/runs/77f74563-fa19-4f0b-84fd-d0224f89a54a/sandbox — the ROOT run id (v1), not

the pinned ef86e750-... (v2) id.

1. Observe the Files tab's "Final output" row: 241.6 KB — matches v1's known size, not v2's.

The version picker in the toolbar also now reads "Version v1".

1. Repeat from /versions/2 clicking the Steps tab instead of Files: same shape — URL reverts to /runs/{rootId}/steps, picker reverts to "Version v1".

EXPECTED

Navigating between tabs while a specific artifact version is pinned should keep that pin: the URL should stay scoped to /versions/2/<tab> (or equivalent), the app should keep fetching the pinned revision's own run id, and every tab (Files, Steps, Workspace, Audit) should show that version's real data.

ACTUAL

Any tab switch away from Preview silently discards the version pin: the URL collapses back to the unversioned root route, the app re-fetches the ROOT run (v1) instead of the pinned revision, and the Files tab (and by the same fetch, every other tab) displays v1's file, not v2's — with no warning, error, or visual cue that the pin was lost.

EVIDENCE

- Before: bug-hunter/evidence/runs-id-versions/BUG-20260828-090732-runs-id-versions/01-before-versions-2-preview.png
- Failure (Files tab, URL and size both reverted to v1): bug-hunter/evidence/runs-id-versions/BUG-20260828-090732-runs-id-versions/02-failure-files-tab-reverts-to-v1.png
- Second reproduction (Steps tab shows the identical pattern): bug-hunter/evidence/runs-id-versions/BUG-20260828-090732-runs-id-versions/03-second-repro-steps-tab-also-drops-version.png
- Network excerpt confirming which run id is actually fetched: bug-hunter/evidence/runs-id-versions/BUG-20260828-090732-runs-id-versions/network.log

BROWSER SIGNALS

- Console: no errors, one pre-existing unrelated warning (unchanged across states).
- Network: GET /api/runs/77f74563-fa19-4f0b-84fd-d0224f89a54a and

.../77f74563-fa19-4f0b-84fd-d0224f89a54a/sandbox fire after the tab click — the ROOT run id, never the pinned revision id ef86e750-bbcd-404f-855a-fb0d5bee63f5.

- State/URL: /runs/{id}/versions/2/... collapses to /runs/{id}/... on any non-Preview tab click; a reload at that point stays on v1 (confirms it is genuine state loss, not a transient render race).

ID BUG-20260828-090732-runs-id-versions Found by bug-runs-id-versions-r2

Fingerprint /runs/[id]/versions/2|version-pin-across-tabs|click-files-or-steps-tab-while-pinned-to-v2|url-and-fetched-data-silently-revert-to-v1-not-just-a-label

Skill detail route (`/library/skills/<id>`) now falls back to the Library listing for EVERY skill id, hyphenated or not — a full regression beyond the previously filed no-hyphen-only scope

`/library/skills/<skillId>` (any id – tested hyphenated multi-segment ids, previously confirmed working)

SUMMARY

BUG-20260828-030430-library-skills-id (filed ~03:04 UTC) characterized this as a narrow defect affecting only the 7 single-word/no-hyphen skill ids, explicitly verifying that hyphenated ids like `windows-desktop-e2e` and `html-deck-to-pptx` rendered their detail page correctly ("byte-for-byte" content match). Retesting now (09:13 UTC, ~6 hours later) shows that characterization no longer holds: **every** skill id tested — including `html-deck-to-pptx` and `windows-desktop-e2e`, the two ids that bug explicitly proved worked — now falls back to the full Library catalog listing instead of the skill detail page. This reproduces identically via direct URL navigation, via a fresh page reload at the same URL, and via a genuine UI card click (scoped to the unique card text, not a global text click) at multiple scroll depths (card index 0, ~100, and a plain two-segment id `motion-patterns`). The URL bar correctly updates to `/library/skills/<id>` in every case; no console error occurs; and the network log shows the only skills-related request fired is `GET /api/skills/library` (the bulk list), never a per-skill resolution — the detail route effectively never attempts to render skill-specific content for ANY id anymore. This is a materially different, far more severe failure (100% of the 186 skills affected, not 7) with an apparent live regression between the two testing windows, so it is filed as a new, distinct issue rather than an update to the closed-scope no-hyphen bug.

REPRODUCTION

1. Sign in as qa-admin, go to `/library?tab=skills`.
2. Click the `html-deck-to-pptx` card (or `android-clean-architecture`, or `windows-desktop-e2e`, or `motion-patterns` — all reproduce identically): URL updates to

/library/skills/html-deck-to-pptx.

1. Observe: instead of the skill's detail page, the exact same Library catalog listing renders (header "94 agents · 186 skills · 8 hooks", tab pills, category pills, full 186-card grid starting at "Accessibility (WCAG 2.2)") — under the detail URL.

1. Reload the same URL directly (browser_navigate to `http://localhost:3000/library/skills/html-deck-to-pptx` from a fresh load): identical fallback, confirming this is not a stale-SPA-state artifact.

1. Repeat with `windows-desktop-e2e` and `accessibility-auditor`: same fallback in every case.
2. Confirm via `browser_network_requests` that only GET `/api/skills/library` (the bulk list) fires — no per-skill request is ever attempted — and `browser_console_messages` shows no error.

EXPECTED

`/library/skills/<id>` should render that skill's own detail page (heading, description, full markdown content, tags, "Copy content" controls) for any valid skill id, exactly as documented working in the earlier bug for `windows-desktop-e2e` and `html-deck-to-pptx`.

ACTUAL

Every tested skill id — hyphenated or not, including the two ids previously proven to work — now renders the Library catalog listing page under the correct detail URL, with no error and no indication the specific skill was never resolved. The skill-detail feature is effectively non-functional for the entire catalog, not just the 7 no-hyphen ids.

EVIDENCE

- Before (listing page, card visible): `bug-hunter/evidence/library-skills/BUG-20260828-091300-library-skills-r2/01-before-listing.png`
- Failure (`html-deck-to-pptx`, URL on detail route, listing rendered): `bug-hunter/evidence/library-skills/BUG-20260828-091300-library-skills-r2/02-failure-detail-url-shows-listing.png`
- After reload, still broken (fresh navigation to same URL): `bug-hunter/evidence/library-skills/BUG-20260828-091300-library-skills-r2/03-after-reload-still-broken.png`

BROWSER SIGNALS

- Console: no errors in any of the ~6 reproductions (click-based and direct-navigation).
- Network: only GET /api/skills/library (bulk list) fires; no per-skill fetch attempt on any

failing navigation, confirmed via browser_network_requests.

- State/URL: location.href correctly reflects /library/skills/<id> in every case while the rendered DOM is the unrelated listing page.

ID BUG-20260828-091300-library-skills-r2 Found by bug-library-skills-r2

Fingerprint /library/skills/<id>|skill-detail-route-match|navigate-to-any-skill-id|library-listing-renders-instead-of-detail-for-100pct-of-skills

21 **HIGH** **"Run once" on the dead-link `/canvas` composer launches a real 9-agent pipeline run, ignoring the visibly-empty 0-agent workflow shown on screen**
`/workflows/<nonexistent-id>/canvas` (e.g. `/workflows/00000000-0000-0000-0000-0000-00000000/canvas`)

SUMMARY

Following up on `BUG-20260828-040400-workflows-nonexistent` (which established that `/workflows/<bad-id>/canvas` silently renders an empty "copy" composer instead of 404ing), this round tested the composer's "Run once" button from that same dead-link entry point. The composer visibly shows "0 agents", an empty canvas with only a "Brief" node, and no workflow content whatsoever — it never fetched a real workflow, since `GET /api/workflows/<bad-id>` 404s. Typing a brief and clicking "Run once" does not fail, does not warn that there is nothing to run, and does not run the empty 0-agent workflow shown on screen. Instead it silently launches a completely different, fully-populated 9-agent pipeline run (Market Research Agent, Strategy Analysis Agent, Roadmap Planning Agent, Security Audit Agent, Test Strategy Agent, Performance Optimization Agent, Documentation Agent, Executive Reporting Agent, Task List Planner) — apparently some client- or server-side default/fallback pipeline entirely unrelated to the dead link the user followed. This is confirmed via the backend: `GET /api/runs/<new-id>` returns a real run record with `agent_count: 9`, `status: "generating"`, and the brief text as `input`. The run genuinely executes against Bedrock — cancelling it after 3 of 9 agents had already completed shows 54.9K tokens (52.2K input / 2.7K output) were consumed for a run the composer gave the user no reason to expect would do anything beyond nothing, given the on-screen "0 agents" state.

REPRODUCTION

1. Sign in as qa-admin, navigate to `http://localhost:3000/workflows/00000000-0000-0000-0000-000000000000/canvas`.
2. Observe the composer header shows "0 agents", "0 review gates", and the canvas has only the "Brief" node — no agents anywhere.
1. Type a brief (e.g. "zz-hunt test brief for dead link canvas run") into the "Brief description"

textbox. The Brief panel updates from "3 more characters to enable Run" to "Ready to run."

despite 0 agents being configured.

1. Click "Run once". The app navigates to `/runs/<new-run-id>/stream` and begins executing a real 9-agent pipeline (visible in the Steps tab: Market Research Agent, Strategy Analysis Agent, Roadmap Planning Agent, Security Audit Agent, Test Strategy Agent, Performance Optimization Agent, Documentation Agent, Executive Reporting Agent, Task List Planner), none of which were ever shown in the composer that launched it.

1. Confirmed via GET `http://localhost:8000/api/runs/<new-run-id>`: `agent_count: 9`, `status: "generating"`, `type: "custom"`.

1. Cancelled the run (Stop button) after 3/9 agents completed to limit token spend; the run detail then showed 54.9K tokens consumed (52.2K input / 2.7K output) for agents the user never selected or saw.

EXPECTED

"Run once" should either be disabled/blocked with a clear message when there is no real workflow behind the composer (0 agents, workflow fetch 404'd), or — at minimum — run exactly the 0-agent pipeline visibly shown on screen (which should itself be a no-op or a clear error), never a different, fully-populated pipeline the user never configured or saw.

ACTUAL

"Run once" silently substitutes and executes an unrelated 9-agent default pipeline, consuming real Bedrock tokens, with no indication anywhere in the composer UI that this would happen.

EVIDENCE

- Before (composer shows 0 agents, empty canvas, "Ready to run"): bug—
hunter/evidence/workflows-nonexistent/BUG-20260828-104200-workflows-nonexistent-r2/01-before-empty-composer-0-agents.png
- Failure (run stream page shows a live 9-agent pipeline actually executing): bug—
hunter/evidence/workflows-nonexistent/BUG-20260828-104200-workflows-nonexistent-r2/02-run-fired-9-agent-pipeline.png
- After cancel (3/9 agents completed, 54.9K tokens consumed): bug—hunter/evidence/workflows-nonexistent/BUG-20260828-104200-workflows-nonexistent-r2/03-cancelled-3-of-9-

agents-ran-54k-tokens.png

BROWSER SIGNALS

- Console: GET /api/workflows/00000000-0000-0000-0000-000000000000 => 404 on composer load (same as the already-filed dead-link-canvas bug); no error at "Run once" time.
- Network: GET http://localhost:8000/api/runs/8b77925f-952d-42ec-b9ed-c767733ffad1 returns a full run record (agent_count: 9, status: "generating", type: "custom") despite the composer that launched it showing "0 agents".
- State/URL: navigates from /workflows/00000000-0000-0000-0000-000000000000/canvas to /runs/8b77925f-952d-42ec-b9ed-c767733ffad1/stream; run id 8b77925f-952d-42ec-b9ed-c767733ffad1, cancelled by tester after 3/9 agents to limit spend.

ID BUG-20260828-104200-workflows-nonexistent-r2 Found by bug-workflows-nonexistent-r2

Fingerprint /workflows/<bad-id>/canvas|run-once|click-run-once-on-empty-copy-composer|launches-unrelated-default-9-agent-pipeline-consuming-real-tokens

SUMMARY

Typing a search term into the library search box (`input[name='library-search']`) that matches no items produces a completely blank content area below the category pills — no "no results found" message, no suggestion to clear the search, no icon, nothing. The header text above still reads "94 agents · 186 skills · 8 hooks · tap any item to see its capabilities" (an unfiltered, static count), giving no indication that the search actually ran or why the list emptied. A user cannot tell whether the app is broken, still loading, or genuinely has zero matches. This reproduces identically on all three tabs (Agents, Skills, Hooks).

REPRODUCTION

1. Sign in as qa-admin and land on `/library` (Agents tab, 94 cards visible).
 2. Type a non-matching string into the search box, e.g. `qqqqnomatch12345`.
 3. Observe: all agent cards disappear, the category pills (All 94, App Builder 15, Retry Loop 0, ...) remain visible and unchanged, but the area below them is entirely blank — no empty-state messaging of any kind.
-
1. Switch to the Skills tab (`[role="tab"]:nth-child(2)`) with a different non-matching string (`zzzznonexistentquery`) — same blank result, same absence of messaging.
-
1. Switch to the Hooks tab (`[role="tab"]:nth-child(3)`) while the search box still holds a non-matching value — same blank result.
-
1. Clear the search box — cards reappear normally, confirming the filter itself works and this is purely a missing-empty-state defect, not a broken search.

EXPECTED

When a search yields zero results, the page should show an explicit empty state (e.g. "No agents match 'qqqqnomatch12345'" plus a way to clear the search), consistent with standard list/search UX and with how the app should communicate a legitimate zero-result state versus a

loading or broken state.

ACTUAL

The results area renders completely blank with no text, icon, or affordance indicating why, on all three library tabs.

EVIDENCE

- Before: bug-hunter/evidence/library/BUG-20260827-221730-library/01-before-agents-loaded.png
- Failure (Agents tab): bug-hunter/evidence/library/BUG-20260827-221730-library/02-failure-agents-no-empty-state.png
- Reproduced (Skills tab): bug-hunter/evidence/library/BUG-20260827-221730-library/03-reproduced-skills-tab.png
- Reproduced (Hooks tab): bug-hunter/evidence/library/BUG-20260827-221730-library/04-reproduced-hooks-tab.png

BROWSER SIGNALS

- Console: no relevant error observed.
- Network: filtering appears client-side; no failed request observed.
- State/URL: URL stays on the respective /library tab throughout; purely a rendering/empty-state gap in the results list component.

ID `BUG-20260827-221730-library` Found by `bug-library-r1`

Fingerprint `/library|search-input|type-nonmatching-query|blank-results-no-empty-state`

SUMMARY

On the Password card of `/settings/profile`, entering a New password and a Confirm new password that differ and clicking "Change Password" correctly shows a client-side "New passwords do not match" error (no network request fires — this validation is a good, working guard). However, if the user then edits the Confirm field so it becomes byte-for-byte identical to the New password field, the "New passwords do not match" message is never cleared. It remains on screen indefinitely (verified via reading both `input[type=password]` DOM values directly — they are identical) even though the stated condition for the error is no longer true. The error is only ever computed at submit time, not reactively on input change, so once shown it is permanently stale until the next submit click. A user who fixes their typo has no way to know, short of clicking submit again, whether the form actually agrees with them.

REPRODUCTION

1. Sign in as qa-admin, land on `/dashboard`, navigate to `/settings/profile`.
2. In the Password card, fill Current password with the real current password, New password with `SecondTry999`, Confirm new password with a different value `DifferentValue1`.
 1. Click "Change Password". Observe the error "New passwords do not match" appears (correct, expected behavior at this point).
 1. Without touching New password, edit Confirm new password to `SecondTry999` (now identical to New password).
 1. Observe: the "New passwords do not match" error is STILL displayed, unchanged, even though `document.querySelectorAll('input[type=password]')` confirms both the New and Confirm fields now hold the exact same string.
 1. Reload and repeat with a different pair of values (`NewPass1234/mismatch` → corrected to

match) — same result: the stale error persists after the fields are made to agree.

EXPECTED

Either the mismatch error should re-evaluate reactively as the user types (clearing as soon as the fields agree), or at minimum should be cleared/hidden once the Confirm field is edited, so the displayed validation state always reflects the current field contents.

ACTUAL

The error message is computed once, at the moment "Change Password" is clicked, and is never recomputed or cleared on subsequent edits to the fields — it stays on screen showing a mismatch that no longer exists until the user clicks submit again.

EVIDENCE

- Before: bug-hunter/evidence/settings-profile/BUG-20260827-222300-settings-profile/01-before-fields-empty.png
- Failure (repro 1): bug-hunter/evidence/settings-profile/BUG-20260827-222300-settings-profile/02-failure-stale-error-repro1.png
- Failure (repro 2, fresh reload, different values): bug-hunter/evidence/settings-profile/BUG-20260827-222300-settings-profile/03-failure-stale-error-repro2.png

BROWSER SIGNALS

- Console: no relevant error observed for this defect (a separate, unrelated 401/501 console error pair appears only when a wrong **current** password is submitted — not part of this bug).
- Network: no additional request fires between the initial failed submit and the field edit; the error is pure client-side state that is never invalidated.
- State/URL: URL stays on /settings/profile throughout; confirmed via direct DOM value reads that the two password fields are identical while the "do not match" message is still shown.

ID `BUG-20260827-222300-settings-profile` Found by `bug-settings-profile-r1`

Fingerprint `/settings/profile|change-password-form|edit-confirm-field-after-mismatch-submit|stale-do-not-match-error-persists`

Sign in button is not disabled during the login request, allowing duplicate submissions

/login

SUMMARY

On the sign-in form, filling in valid credentials and clicking "Sign in" multiple times in rapid succession (e.g. a fast triple click, or any repeat click before the async login request resolves) fires one `POST /api/auth/login` request per click instead of being debounced to a single request. The button never enters a disabled state and its label never changes to a pending/loading state (verified via direct DOM reads of `button.disabled` and `button.textContent` immediately before and after a synchronous triple click) — it stays exactly "Sign in", fully clickable, for the entire duration of the pending request(s). This lets a user accidentally send several concurrent authentication requests from one interaction.

REPRODUCTION

1. Ensure signed out: `localStorage.clear()` then navigate to `/login`.
 2. Fill `input[type="email"]` with `qa-admin@flowinqa.com` and `input[type="password"]` with `flowin-e2e-pass` (valid credentials).
1. Programmatically click the "Sign in" button three times back-to-back with no delay between clicks (simulating a fast repeated/rapid click).
1. Inspect the network log filtered to `auth/login` — observe THREE separate `POST http://localhost:8000/api/auth/login` requests, each returning `200 OK`, instead of one.
1. Inspect `button.disabled` and `button.textContent` captured immediately before and immediately after the three clicks — both are unchanged (`disabled: false`, `text: "Sign in"`) at every point, confirming the button never disables or shows a pending state to prevent the extra clicks.
1. Reload, clear `localStorage`, and repeat steps 1-5 with different fresh state — same result: three duplicate `POST /api/auth/login` requests fire, all succeeding.

EXPECTED

The "Sign in" button should disable itself (and/or show a loading state) the instant the first click starts the login request, so additional clicks before the response arrives are ignored and at most one login request is sent per submission attempt.

ACTUAL

The button remains fully enabled and visually unchanged throughout the pending request, so every extra click before the response returns fires its own independent `POST /api/auth/login` request.

EVIDENCE

- Before (form filled, not yet submitted): `bug-hunter/evidence/login/BUG-20260827-222750-login/01-before-filled-form.png`
- After (triple click resolved, landed on /dashboard): `bug-hunter/evidence/login/BUG-20260827-222750-login/02-after-triple-click-lands-dashboard.png`
- Network log excerpt: `bug-hunter/evidence/login/BUG-20260827-222750-login/network.log`

BROWSER SIGNALS

- Console: no relevant error observed.
- Network: three `POST /api/auth/login` requests fired from a single triple-click interaction,

all `200 OK` (see `network.log`).

- State/URL: ends on `/dashboard` as expected once the (last) login response resolves; the

duplication is in the number of requests fired, not the final navigation outcome.

ID `BUG-20260827-222750-login` Found by `bug-login-r1`

Fingerprint `/login|sign-in-button|rapid-repeated-click-with-valid-credentials|duplicate-login-requests-fired`

For an authenticated user, unmatched `/register/<sub-path>` URLs silently render the Dashboard instead of a 404 or a redirect

`/register/anything`, `/register/abc123xyz` (any nonexistent sub-path under `/register`)

SUMMARY

`/register` itself is a known, intentional redirect stub (out of scope here). But a nonexistent sub-path under it, e.g. `/register/anything`, is a different code path entirely — for a signed-out user it correctly falls through to `/login?redirect=%2Fregister%2Fanything` (an app auth-guard behavior, not the register stub's own `router.replace('/login')`). For an **authenticated** user, however, that same unmatched sub-path does not redirect and does not 404: the browser address bar stays on `/register/anything` (or any other nonexistent sub-path) while the app fully renders the authenticated `/dashboard` catalog — workflow cards, "Jump back in" run history, header nav with "Home" highlighted active — as if the URL were `/dashboard`. This reproduces on a genuine hard/full-page navigation (not just client-side routing residue), confirmed via `location.href` still reading the `/register/...` path while `document.title`, `h1` text, and live API calls (`/api/runs`, `/api/workflows`, `/api/analytics/summary`, etc., all 200 OK) match the dashboard exactly. Distinct from D-03 (`?expired=1` no-op), D-25 (dead validation code) and D-34 (logout 401) — none of which concern route resolution for unmatched paths under `/register`. Also distinct from the ledger's existing `BUG-20260827-223300-login-expired=true` (which is about the `/login` route failing to redirect an authenticated user away from the sign-in form) — here the URL is `/register/...`, no sign-in form is ever shown, and the failure is that the wrong page's content silently renders under the wrong URL rather than the app failing to leave a login screen.

REPRODUCTION

1. Sign in as qa-admin (`localStorage.getItem('auth_token')` is set), confirm landing on `/dashboard`.

1. Navigate directly (full page.`goto`, real navigation) to `http://localhost:3000/register/anything`.

2. Observe: the address bar stays on `/register/anything`, but the rendered page is the full authenticated dashboard — heading "What would you like to build today?", the workflow catalog cards, header with "Home" nav active, "Jump back in" run history — identical to `/dashboard`.

1. Confirm via `location.href` (still `/register/anything`) and `document.querySelector('h1').textContent`

("What would you like to build today?") that this is not a stale screenshot artifact.

1. Repeat with a second, different nonexistent sub-path, `http://localhost:3000/register/abc123xyz`

— same result: URL stays on the bogus `/register/...` path, dashboard content renders.

1. For contrast, sign out (`localStorage.clear()`) and repeat step 2 — the unauthenticated case correctly falls through to `/login?redirect=%2Fregister%2Fanything`, confirming the defect is specific to the authenticated path through this route.

EXPECTED

An authenticated user hitting a nonexistent sub-path under `/register` should either receive a proper 404/not-found page, or be redirected to a real destination (e.g. `/dashboard` via an actual navigation/redirect, or back to `/register` itself) — with the URL and rendered content agreeing with each other in either case.

ACTUAL

The URL bar permanently displays the bogus, nonexistent `/register/<sub-path>` while the dashboard is fully rendered underneath it — an authenticated-only URL/content mismatch that persists across hard reloads and is stable, not a transient loading flash.

EVIDENCE

- After login redirect lands here: `bug-hunter/evidence/register/BUG-20260827-231407-register/01-after-login-redirect-url-mismatch.png`
- Fresh hard reload, still mismatched: `bug-hunter/evidence/register/BUG-20260827-231407-register/02-fresh-hard-reload-still-dashboard.png`

- Second sub-path repro: bug-hunter/evidence/register/BUG-20260827-231407-register/03-repro2-different-subpath.png

BROWSER SIGNALS

- Console: no errors; only routine HMR/devtools info logs.
- Network: dashboard-only API calls (/api/runs, /api/auth/me, /api/workflows, /api/analytics/summary, /api/agents/library, /api/skills/library, /api/hooks/library)

all return 200 OK on this URL, confirming the dashboard is genuinely rendering, not just a stale DOM.

- State/URL: `location.href` remains the nonexistent `/register/<sub-path>` throughout; no navigation event ever corrects it while authenticated.

ID `BUG-20260827-231407-register` Found by `bug-register-r1`

Fingerprint `/register/<sub-path>|route-resolution|authenticated-user-navigates-to-unmatched-register-subroute|url-shows-register-subpath-but-dashboard-content-renders`

Browser Back to `/` after the corrupt-token auth bounce leaves the app permanently blank instead of redirecting to `/login`

/

SUMMARY

Empirical check of the two conflicting docs (the `velocity.json` quirk claiming `/` unconditionally redirects to `/login`, vs. `DEFECTS-OBSERVED.md` correction C-3 claiming it branches on token presence): C-3 is correct — `/` reliably branches (`/dashboard` when a token exists, `/login` when it does not) via a client-side `router.replace`, confirmed in both auth states, with no extra history entry added (replace, not push) across many repeated fresh navigations. However, one specific navigation path breaks this: after an invalid/corrupt token causes the app's normal 401-recovery bounce (`/` → `/dashboard` (401) → `/login?expired=true`, which also clears the bad token from `localStorage`), pressing the browser **Back** button returns to the original `/` URL via a genuine full network reload (GET `/` ⇒ 200, all JS chunks re-fetched, not a bfcache restore) — but the client redirect logic never fires. The page renders a completely empty `<body>` (0 characters of text, an untouched React root) and stays that way indefinitely (verified at events out to 5+ seconds); `location.href` remains `/` and `localStorage.getItem('auth_token')` is confirmed `null`, so the app has every fact it needs to redirect to `/login` and simply doesn't. No console error is logged. A manual `location.reload()` on the exact same stuck URL immediately and correctly redirects to `/login`, proving the redirect logic itself works and this is specifically a Back-navigation trigger gap, not the C-3-corrected core behavior being wrong.

REPRODUCTION

1. Sign in as qa-admin, land on `/dashboard` (real, valid `auth_token` in `localStorage`).
2. Corrupt the token: `localStorage.setItem('auth_token', 'corrupted.invalid.token')`.
3. Navigate to `http://localhost:3000/` (a real, fresh navigation). Observe the app's normal bounce: `/` → briefly resolves `/dashboard` → GET `/api/runs` and POST `/api/auth/refresh` both 401 → lands on `/login?expired=true` with the session-expired banner, and the bad token

is cleared from `localStorage` (confirmed `null`).

1. Press the browser **Back** button.
2. Observe: the URL becomes `http://localhost:3000/` again (a genuine full reload — Network shows `GET / => 200 OK` and every JS chunk re-fetched, not a `bfcache` restore), but the page renders nothing: `document.body.children.length` is non-zero (empty React root mounted) yet `document.body.innerText` is the empty string, and no visible content, form, or banner appears.

1. Wait 5+ seconds — the URL and blank state never change.
`localStorage.getItem('auth_token')` is

`null` the entire time, so the expected outcome (redirect to `/login`) is never triggered.

1. Confirm the redirect logic is otherwise intact: run `location.reload()` on this exact stuck `/` URL — it immediately redirects to `/login` correctly.

1. Repeated the full sequence (steps 1-6) a second time from a fresh `/dashboard` baseline — same result: permanent blank page at `/` after Back, requiring a manual reload to recover.

EXPECTED

Any load of `/` with no valid `auth_token` — whether reached by a fresh navigation, a client-side route change, or the browser Back button — should redirect to `/login`, matching the app's own correctly-working behavior for every other path to `/`.

ACTUAL

Specifically when `/` is reached via browser Back immediately after the corrupt-token bounce chain, the redirect never fires: the page mounts an empty, contentless root and stays there indefinitely with no error, no loading indicator, and no way to recover short of an explicit manual reload or new navigation.

EVIDENCE

- Before (signed in, valid dashboard): `bug-hunter/evidence/root/BUG-20260827-232305-root/01-before-signed-in-dashboard.png`
- Corrupt-token bounce completes normally: `bug-hunter/evidence/root/BUG-20260827-232305-root/02-corrupt-token-bounce-to-login-expired.png`

- Failure (Back navigation, permanently blank /): `bug-hunter/evidence/root/BUG-20260827-232305-root/03-back-navigation-stuck-blank-root.png`

BROWSER SIGNALS

- Console: no errors or warnings logged during the stuck state.
- Network: the Back navigation fires a genuine GET `http://localhost:3000/ => 200 OK` (full chunk

re-fetch, confirmed via `browser_network_requests`), so this is not a stale bfcache snapshot —

the client bundle re-mounts fresh and still fails to redirect.

- State/URL: `location.href` stays `http://localhost:3000/` indefinitely; `auth_token` is confirmed `null` throughout; a manual `location.reload()` on the same URL immediately redirects to `/login`, isolating the defect to the Back-triggered load path specifically.

ID `BUG-20260827-232305-root` Found by `bug-root-r1`

Fingerprint `/|root-redirect|browser-back-after-corrupt-token-bounce-chain|blank-page-stuck-forever-no-redirect`

SUMMARY

The launch panel exposes two separate controls for the same underlying concept — whether an agent's step pauses the pipeline for human review. The top-level "Review gates" popover (a checklist of the workflow's agents, "Checked agents pause the pipeline for your review after they finish") and the "Advanced" modal's per-agent Config tab (Gate combobox: "No gate" / "Conditional gate" / "Human gate"). Checking "Domain Discovery Agent" in the Review gates checklist correctly updates the pill label to "1 agent pause for review" and persists the checked state (re-opening the popover shows it still checked). But opening the Advanced modal, selecting that exact same agent node, and viewing its Config tab shows Gate: "No gate" selected — not "Human gate" as the checklist claims. The two controls disagree about the state of the identical agent within the same page load, with no save/apply step between them (both are live, un-submitted, in-session state on the same composer). This is independent of, and a different mechanism from, D-22/D-23 (Advanced modal Escape/close-button issues) and D-05/D-10 (saved-override agent-count disagreements) — this defect is about a review-gate boolean, not a modal-dismissal control or an agent-count estimate, and it reproduces before anything is ever saved or an override is created.

REPRODUCTION

1. Sign in as qa-admin, navigate to `http://localhost:3000/create/app` (fresh load — confirmed "Review gates" pill reads "no gates" and all 6 checklist checkboxes are unchecked).
1. Click the "Review gates" pill to open its checklist popover.
2. Click the "Domain Discovery Agent" row to check it. Observe the pill label immediately updates to "Review gates — 1 agent pause for review" and the checkbox is checked (confirmed via snapshot).

1. Press Escape / click elsewhere to close the popover (state persists — reopening confirms the checkbox stays checked).

1. Click the "Advanced" button to open "Advanced Workflow Configuration — User Stories".
2. Click the "Domain Discovery Agent" node on the canvas to select it, then click its "Config" tab in the right-hand rail.

1. Observe the Gate combobox for this exact agent: option "No gate" [selected] — not "Human gate" — even though the Review gates checklist (still checked, visible behind/beside the modal) says this agent is a review pause point.

1. Repeated the full sequence (steps 1-7) a second time from a fresh /create/app page load — identical result: checklist shows 1 agent checked and the pill updates, but the modal's per-agent Gate dropdown for that same agent still reads "No gate".

EXPECTED

Checking an agent in the "Review gates" checklist and viewing that same agent's Gate setting in the Advanced modal should show a consistent state (e.g. "Human gate" selected), since both controls describe the same underlying per-agent review-gate property on the same in-session workflow configuration.

ACTUAL

The two controls disagree: the checklist reports the agent as gated ("1 agent pause for review"), while the Advanced modal's own per-agent Config panel for the identical agent reports "No gate" selected — a live, unresolved contradiction about the same field within the same page session.

EVIDENCE

- Before (fresh page, "no gates", all unchecked): bug-hunter/evidence/create-app/BUG-20260828-000100-create-app/01-before-fresh-no-gates.png
- Checklist checked, pill reads "1 agent pause for review": bug-hunter/evidence/create-app/BUG-20260828-000100-create-app/02-checklist-shows-1-agent-checked.png
- Failure (Advanced modal Config tab for the same agent shows "No gate" selected): bug-hunter/evidence/create-app/BUG-20260828-000100-create-app/03-failure-advanced-config-shows-no-gate.png

BROWSER SIGNALS

- Console: no relevant error observed.
- Network: no request involved for either control; both are purely client-side in-session state

on the same composer page.

- State/URL: URL stays `/create/app` throughout; verified via direct DOM query that all six canvas nodes' "Gate" pill buttons render with identical, unstyled classes (no visual "active"/pressed indicator) regardless of the checklist's checked state, and the Config panel's `<select>/combobox` value for the checked agent is confirmed "No gate" via the accessibility tree, not just a screenshot read.

ID `BUG-20260828-000100-create-app` Found by `bug-create-app-r1`

Fingerprint `/create/app|review-gates-checklist-vs-advanced-config-gate-dropdown|check-agent-in-review-gates-popover|advanced-modal-per-agent-gate-still-reads-no-gate`

The Advanced modal's Workflow-level settings (Smart planning, Confirm requirements first, Deliverable strategy, Internet access) are permanently disabled with no explanation

/create/user-stories

SUMMARY

Opening the "Advanced" modal and viewing its right-rail "Workflow" tab (the default view when no agent node is selected on the canvas) renders four workflow-level settings — "Smart planning" (switch), "Confirm requirements first" (switch), "Deliverable" strategy (combobox, plus its dependent "Output file name" text input and "Output format" combobox), and "Internet access" (switch) — all with the HTML `disabled` attribute set, verified via direct DOM reads (`element.disabled === true` for every one of the six form controls). Nothing on the page indicates why: no lock icon, no tooltip, no "upgrade to unlock" copy, no `title` attribute — each control's only affordance is an (i) info icon whose tooltip explains what the setting *does*, not why it cannot be changed. This is not a page-wide disabled state: in the same modal, the "Brief instruction" textarea directly above these controls is fully interactive (`textarea.disabled === false`), and selecting an agent node and switching to its own "Tools" tab shows genuinely interactive switches ("Read files"/"Write files" enabled, only the security-gated "Execute commands" disabled) — proving the app can and does render real, togglable switches elsewhere in the identical modal. The Workflow tab's four settings are simply inert controls masquerading as configurable options, on every fresh page load, with no code path observed that ever enables them.

REPRODUCTION

1. Sign in as qa-admin, navigate to `http://localhost:3000/create/user-stories` (fresh load).
2. Click the "Advanced 6 agents · ~35s" button to open "Advanced Workflow Configuration — User Stories".

1. With no agent node selected (default state), the right-rail "Workflow" tab is active. Observe "Smart planning", "Confirm requirements first", "Deliverable" (dropdown + filename + format),

and "Internet access" are all visually greyed out.

1. Confirm via `document.querySelectorAll(' [role="switch"]')` and the `<select>/<input>` elements under that panel that every one of the six controls has `disabled === true`.

1. For contrast, confirm the "Brief instruction" textarea in the same panel is NOT disabled (`disabled === false`), and that selecting an agent node and opening its own "Tools" tab shows "Read files"/"Write files" switches that ARE enabled — proving this is not a general modal rendering artifact.

1. Closed the modal via "Cancel", did a fresh full navigation to `/create/user-stories` again, reopened "Advanced", and re-checked the same six controls — identical result: all six still `disabled === true`, deterministic across two independent fresh loads.

EXPECTED

Workflow-level settings that are rendered as interactive switches/dropdowns should be editable by an admin/enterprise user (or, if intentionally locked for this workflow, should carry a visible reason — a lock icon, disabled-state tooltip, or explanatory copy — consistent with how the canvas's own "Core = locked" agent badge explains its own lock state).

ACTUAL

All four workflow-level settings and their two dependent fields render as fully-styled, seemingly interactive controls that are permanently disabled, with zero indication anywhere in the UI of why, blocking a user from ever changing "Smart planning", "Confirm requirements first", "Deliverable" output strategy/filename/format, or "Internet access" for this workflow.

EVIDENCE

- Before (fresh launch panel, modal closed): `bug-hunter/evidence/create-user-stories/BUG-20260828-000600-create-user-stories/01-before-launch-panel.png`
- Failure, repro 2 (fresh page load, Workflow tab, all six controls disabled): `bug-hunter/evidence/create-user-stories/BUG-20260828-000600-create-user-stories/02-repro2-fresh-load-workflow-tab-all-disabled.png`
- Failure, repro 1 (same result on the first investigation pass): `bug-hunter/evidence/create-user-stories/BUG-20260828-000600-create-user-stories/03-repro1-workflow-tab-all-disabled.png`

BROWSER SIGNALS

- Console: no relevant error observed.
- Network: no request involved; this is static disabled-attribute rendering with no data-fetch

dependency observed to ever flip it.

- State/URL: URL stays `/create/user-stories` throughout; verified via direct DOM

`element.disabled` reads (not just visual/screenshot inspection) on two independent fresh page

loads, and confirmed non-disabled sibling controls (Brief textarea, agent-level Tools switches)

exist in the same modal session to rule out a page-wide rendering fluke.

ID `BUG-20260828-000600-create-user-stories` Found by `bug-create-user-stories-r1`

Fingerprint `/create/user-stories|advanced-modal-workflow-tab|open-workflow-tab-no-node-selected|all-workflow-level-controls-permanently-disabled-no-affordance`

Advanced modal's "Cancel" button does not discard edits to a conditional-gate outcome's route

/create/ex_A2_branch

SUMMARY

ex_A2_branch ("Branch by Language") is a real branching fixture whose "Pick Language" agent step has a Conditional gate with three outcomes (english/spanish/dutch), each routing to a forward step. In the Advanced modal's per-agent Config tab, changing one outcome's "Type" from "Step" to "Workflow" and picking a target workflow (e.g. "Spanish Greeter") via the "Pick a workflow" dialog is a live, un-saved edit to that route. Clicking the modal's own "Cancel" button is the documented way to discard such in-session edits without persisting them (no network request is ever fired for either the edit or the Cancel — confirmed via `browser_network_requests`, only background GETs are present). Instead, closing the modal via Cancel and reopening "Advanced" shows the edit still applied: the canvas gains a new "Spanish Greeter — Diverts run" node, the step legend changes from english/spanish/dutch to spanish/dutch then back to including english once re-pointed, and the Config tab for "Pick Language" still shows the english outcome's Type as "Workflow" and Target as "Spanish Greeter". This reproduced twice in a row (Cancel → reopen → still present, Cancel again → reopen → still present). A full page reload (`/create/ex_A2_branch` fresh navigation, not just modal close) does correctly reset to the original 5-agent, no-divert state, confirming the edit was never sent to the backend — it is purely un-discarded client-side component state that "Cancel" fails to reset, unlike a real save. This is a different mechanism from the already-filed `create-app/create-user-stories` Advanced-modal defects (per-agent Gate/Review-gates desync, and permanently-disabled Workflow-tab controls) — this is about the modal's own Cancel action failing to roll back an edit made inside the same modal session, specific to the conditional-gate route editor that only branching fixtures like this one expose.

REPRODUCTION

1. Sign in as qa-admin, navigate to http://localhost:3000/create/ex_A2_branch (fresh load).

Confirm "Advanced 5 agents" and no divert node on the canvas.

1. Click "Advanced 5 agents" to open "Advanced Workflow Configuration — Language Branch".
2. Click the "Pick Language" node, then its "Route 3 outcomes" pill (Config tab, conditional

gate route editor with english/spanish/dutch outcomes).

1. For the english outcome, change "Type" from "Step" to "Workflow". The "Target" control becomes a "Pick a workflow..." button.

1. Click it, select "Spanish Greeter" in the "Pick a workflow" dialog, click "Pick workflow" to confirm. Observe the Target now reads "Spanish Greeter" and a new "Spanish Greeter — Diverts run" node appears on the canvas.

1. Click the modal's "Cancel" button (bottom-right) to close it without saving.
2. Re-open "Advanced" (button still reads "5 agents" on the launch panel behind the modal).

Observe the canvas still shows the "Spanish Greeter — Diverts run" node, and selecting "Pick Language" → Config → Route still shows english outcome Type "Workflow", Target "Spanish Greeter".

1. Repeated steps 6-7 a second time (Cancel again, reopen again) — identical result.
2. Did a full page reload (fresh navigation to the same URL, not just modal close/reopen) —

the launch panel correctly resets to "5 agents" with no divert node, confirming the edit was never persisted server-side (no relevant POST/PUT/PATCH observed in `browser_network_requests` at any point in steps 4-8).

EXPECTED

Clicking "Cancel" in the Advanced modal should discard any edits made during that modal session (here, the english outcome's Type/Target change), returning the canvas and Config panel to the state they were in when the modal was opened — consistent with "Cancel" being offered as the alternative to "Save workflow".

ACTUAL

"Cancel" closes the modal but does not roll back the route-editor edit; the changed outcome

Type/Target and the resulting extra canvas node persist across repeated close/reopen cycles

within the same page session, even though nothing was ever saved to the backend.

EVIDENCE

- Before (fresh load, no divert node): bug-hunter/evidence/create-ex-a2-branch/BUG-20260828-001100-create-ex-a2-branch/01-before-fresh-load.png
- Failure, repro 1 (after Cancel + reopen, Spanish Greeter divert node present): bug-hunter/evidence/create-ex-a2-branch/BUG-20260828-001100-create-ex-a2-branch/02-after-cancel-reopen-shows-spanish-greeter-target.png
- Failure, repro 2 (after a second Cancel + reopen, still present): bug-hunter/evidence/create-ex-a2-branch/BUG-20260828-001100-create-ex-a2-branch/03-repro2-after-second-cancel-still-shows-spanish-greeter.png

BROWSER SIGNALS

- Console: no relevant error observed.
- Network: browser_network_requests showed only background GETs (/api/workflows, /api/workflows/ex_A2_branch, /api/agents/library, etc.) throughout the edit/Cancel/reopen cycle — no POST/PUT/PATCH fired, confirming the persisted edit is unsaved client state, not a silent server save.
- State/URL: URL stayed /create/ex_A2_branch throughout steps 1-8; a fresh full navigation in step 9 correctly reset the panel, isolating the defect to the modal's Cancel handler rather than a backend persistence bug.

ID BUG-20260828-001100-create-ex-a2-branch Found by bug-create-ex-a2-branch-r1

Fingerprint /create/ex_A2_branch|advanced-modal-conditional-gate-route-editor|change-outcome-type-and-target-then-click-cancel|edit-persists-across-modal-close-and-reopen

`/workflow/create` never redirects when `mode` is absent or empty, leaving the legacy URL live and uncanonicalized

`/workflow/create` (no query string), `/workflow/create?mode=` (empty value)

SUMMARY

C-1/the documented redirect behavior covers `/workflow/create?mode=ppt` and `?mode=prototype`, both of which correctly router.replace to `/create/ppt` / `/create/prototype` (verified as a working baseline in this investigation — see evidence 01). But the redirect only fires when mode carries a truthy value. Navigating to `/workflow/create` with **no query string at all**, or with `?mode=` (present but empty), never redirects — `location.href` stays on the legacy `/workflow/create` URL indefinitely (confirmed after full navigation and settle, not just a client-side transition). Despite the URL not canonicalizing, the page is not blank or broken: it silently renders the full "Configure your prototype" wizard (h1 "Configure your prototype") with live, successful API calls (GET `/api/workflows/prototype`, `/api/prototype/templates`, `/api/prototype/design-systems` all 200 OK) — i.e. an undocumented default-to-prototype fallback that keeps the legacy URL permanently alive and functional instead of ever canonicalizing to `/create/prototype`. This means any old bookmark, external link, or hand-typed URL missing the mode param becomes a permanent, un-redirected alias into current app functionality — the opposite of what a redirect stub is supposed to guarantee (a single canonical live URL per feature).

REPRODUCTION

1. Signed in as qa-admin (session inherited, auth_token present).
2. Navigate to `http://localhost:3000/workflow/create?mode=ppt` — confirm baseline: URL becomes `/create/ppt` (redirect works correctly for a valid, non-empty mode).
1. Navigate to `http://localhost:3000/workflow/create` (bare, no query string at all, full page.goto navigation). Observe: URL stays `http://localhost:3000/workflow/create` — no

redirect fires. `document.querySelector('h1').textContent` reads "Configure your prototype"; network shows successful GET `/api/workflows/prototype`, `/api/prototype/templates`, `/api/prototype/design-systems` (all 200 OK) — the prototype wizard is fully live under the legacy URL.

1. Navigate to `http://localhost:3000/workflow/create?mode=` (mode present but empty).
Observe

the identical result: URL stays `/workflow/create?mode=`, h1 stays "Configure your prototype", same live wizard renders — reproducing the defect a second time with a slightly different trigger (empty string vs. entirely missing key).

1. Confirmed no console errors or warnings in either state.

EXPECTED

A redirect stub is expected to canonicalize every reachable variant of its route to the real target URL, or otherwise fail cleanly (404/error state). At minimum, missing or empty mode should not silently fall through to rendering full, live application content under the stale legacy URL — either redirect to a sensible default (e.g. `/create/prototype` or `/create`), or show an explicit "missing mode" error, consistent with how `?mode=ppt/?mode=prototype` behave.

ACTUAL

`/workflow/create` with no mode param, or with mode present but empty, never triggers the redirect at all. The legacy URL stays in the address bar while the app quietly renders a fully functional prototype wizard underneath it, giving the legacy route a permanent, working, second identity for the same feature instead of ever being canonicalized away.

EVIDENCE

- Baseline (`?mode=ppt` redirects correctly to `/create/ppt`): `bug-hunter/evidence/workflow-create-ppt/BUG-20260828-001420-workflow-create-ppt/01-baseline-mode-ppt-redirects-correctly.png`
- Failure (bare `/workflow/create`, no query, stays on legacy URL, prototype wizard renders): `bug-hunter/evidence/workflow-create-ppt/BUG-20260828-001420-workflow-create-ppt/02-failure-bare-no-query-stays-on-legacy-url.png`

- Reproduced (?mode= empty value, same result): bug-hunter/evidence/workflow-create-ppt/BUG-20260828-001420-workflow-create-ppt/03-repro2-empty-mode-param-same-result.png

BROWSER SIGNALS

- Console: no relevant error observed in either failing state.
- Network: all API calls made by the fallback-rendered prototype wizard return 200 OK

(/api/workflows/prototype, /api/prototype/templates, /api/prototype/design-systems),

confirming this is a genuinely functional page, not a broken/error render — which is exactly

what makes the missing canonicalization more likely to go unnoticed.

- State/URL: `location.href` never changes away from `/workflow/create` (with or without the empty `mode=`) in either reproduction, contrasted directly against the working `?mode=ppt`

baseline captured in the same session.

ID `BUG-20260828-001420-workflow-create-ppt` Found by `bug-workflow-create-ppt-r1`

Fingerprint `/workflow/create|legacy-wizard-redirect|navigate-with-no-mode-param-or-empty-mode-value|no-redirect-fires-legacy-url-serves-live-prototype-content`

Invalid `mode` value redirects into a permanently dead-end 0-agent composer with the underlying 404 silently swallowed

`/workflow/create?mode=nonsense` (any unrecognized, non-empty `mode` value)

SUMMARY

This is a distinct failure mode from the already-filed `BUG-20260828-001420-workflow-create-ppt` (missing/empty mode never redirects and silently falls back to a fully functional prototype wizard under the stale legacy URL). Here the trigger is different — an unrecognized but **non-empty** mode value, e.g. `mode=nonsense` — and the symptom is different too: the redirect DOES fire (`/workflow/create?mode=nonsense` → `/create/nonsense`, treating "nonsense" as a workflow slug), but `/create/nonsense` is not a real workflow. `GET /api/workflows/nonsense` returns `404 Not Found` (confirmed twice via console), yet the page renders a normal-looking "Provide the brief" generic workflow composer with zero visible indication that anything failed — no error banner, no toast, no "workflow not found" message anywhere in `document.body.innerText`.

The composer is permanently stuck at "0 agents": `Save workflow`, `Save as my version`, and `Add agents first` all render with the `HTML disabled` attribute set and stay disabled even after typing a full, valid, non-trivial brief into the textbox (verified via direct DOM `button.disabled` reads before and after filling the brief). There is no way to add an agent, save, or run anything — a genuine, silent dead end reachable via a completely ordinary-looking URL typo (a stray character in `mode=`), with the only evidence of failure being a console 404 a typical user would never see.

REPRODUCTION

1. Signed in as qa-admin (session inherited).
2. Navigate to `http://localhost:3000/workflow/create?mode=nonsense`. Observe the redirect fires to `/create/nonsense` (URL bar changes, confirming this is treated as a workflow-slug lookup, not the `mode=ppt/mode=prototype` special-cased redirect).

1. Observe the console: GET `http://localhost:8000/api/workflows/nonsense` returns 404 Not Found.
 2. Observe the rendered page: heading "Provide the brief", "Advanced 0 agents" button, and no error text anywhere in `document.body.innerText` — the 404 is completely invisible in the UI.
1. Confirm via DOM that `Save workflow`, `Save as my version`, and `Add agents first` all have `disabled === true`.
1. Type a full, valid brief ("Build a test app with enough characters to satisfy validation") into the "Brief description" textbox. Re-check the same three buttons — all three remain `disabled === true`, unchanged. There is no action on this page that can ever be completed.
1. Did a fresh, independent page navigation (not just a state re-check) to the same URL a second time — identical result: 404 in console, 0-agent dead-end composer, all three action buttons disabled, no error message, deterministic across both attempts.

EXPECTED

An unrecognized mode/workflow slug should either show an explicit "workflow not found" error (with a way back to a valid start point), or fall back to a real, usable default (as the missing-mode case does, however incorrectly, by defaulting to a working prototype wizard) — either way, the resulting page should not silently present a normal-looking but permanently non-functional composer with no indication that anything is wrong.

ACTUAL

The page renders what looks like an ordinary, empty workflow composer, giving no indication that `/api/workflows/nonsense 404'd`. Every possible action (save, save-as, add agents) is permanently disabled regardless of what the user types into the brief field, so the page is an unrecoverable dead end reachable via nothing more than a mistyped mode query value.

EVIDENCE

- Before: `bug-hunter/evidence/workflow-create-prototype/BUG-20260828-001720-workflow-create-prototype/01-before-mode-nonsense.png`
- Failure (brief filled, buttons still disabled): `bug-hunter/evidence/workflow-create-prototype/BUG-20260828-001720-workflow-create-prototype/02-failure-brief-`

filled-still-disabled.png

- Reproduced (fresh independent navigation): bug-hunter/evidence/workflow-create-prototype/BUG-20260828-001720-workflow-create-prototype/03-repro2-fresh-load.png

BROWSER SIGNALS

- Console: GET http://localhost:8000/api/workflows/nonsense → 404 Not Found, on both reproduction attempts, with no corresponding UI error surfaced.

- Network: same 404 confirmed via console entries; no other failed requests.
- State/URL: location.href settles on /create/nonsense (the redirect itself works); the defect

is entirely in the resulting page's failure to reflect the 404 or offer any usable action.

ID BUG-20260828-001720-workflow-create-prototype Found by bug-workflow-create-prototype-r1

Fingerprint /workflow/create|mode-param-redirect|navigate-with-unrecognized-nonempty-mode-value|redirects-to-generic-0-agent-composer-permanently-disabled-no-error-shown

Leaving the new-workflow composer via its own "Back" control silently discards all unsaved work with no confirmation

`/workflows/new`

SUMMARY

On `/workflows/new`, after naming the workflow and adding an agent to the canvas (real, unsaved, in-progress composer state — never persisted, since "Save workflow" was never clicked), clicking the composer's own button "Back" control (top-left, immediately left of the workflow name field) navigates straight to `/workflows` with zero confirmation prompt. All in-progress work — the workflow name and every added agent node — is silently and irrecoverably discarded. Returning to `/workflows/new` afterward shows a completely blank "Untitled workflow" canvas with 0 agents; there is no draft-recovery mechanism. This reproduces identically via a plain client-side route navigation (`page.goto`) to another in-app route while unsaved work is present — neither the in-page Back button nor a direct navigation away triggers any "unsaved changes" warning (no `beforeunload` prompt, no in-app confirm dialog).

REPRODUCTION

1. Sign in as qa-admin, navigate to `/workflows/new` (empty composer, 0 agents).
2. Type a name into the "Workflow name" textbox, e.g. "Bug Hunter Throwaway Draft".
3. Click "Add agent", add "Domain Discovery Agent" from the library modal, press Escape to close the modal. Canvas now shows the named workflow with 1 agent (1 agents in the header summary) — none of this has been saved (Save workflow was never clicked).

1. Click the composer's button "Back" (top-left, next to the workflow name field).
2. Observe: navigation to `/workflows` happens immediately with no confirmation dialog of any kind.

1. Navigate back to `/workflows/new`. Observe: the composer is completely reset — "Untitled workflow" placeholder, 0 agents, no trace of the name or the added agent.

EXPECTED

Either the "Back" control (and any other away-navigation while the composer has unsaved,

uncommitted agents/name) should prompt the user to confirm discarding their in-progress work (a "You have unsaved changes — leave anyway?" dialog), or the composer should preserve/restore a draft so returning to `/workflows/new` does not start from a totally blank canvas.

ACTUAL

The composer's own Back button discards all unsaved state (name + every added agent) with no warning whatsoever, and there is no draft persisted anywhere the user can recover.

EVIDENCE

- Before leaving (named workflow with 1 agent, unsaved): `bug-hunter/evidence/workflows-new/BUG-20260828-003800-workflows-new/01-before-leave-with-work.png`
- After returning to `/workflows/new` (work gone, no warning was ever shown): `bug-hunter/evidence/workflows-new/BUG-20260828-003800-workflows-new/02-after-back-work-gone.png`

BROWSER SIGNALS

- Console: no relevant error observed.
- Network: no request fired for the discarded work (never persisted, confirming it was pure

client-side composer state that had no save/draft path at all).

- State/URL: URL transitions cleanly from `/workflows/new` to `/workflows` and back to

`/workflows/new`; no dialog/alert intercepted the navigation at any point.

ID `BUG-20260828-003800-workflows-new` Found by `bug-workflows-new-r1`

Fingerprint `/workflows/new|composer-back-button|click-back-with-unsaved-agents-and-name|work-discarded-no-confirmation`

Simple view's per-agent "Configure →" panel has no Tools section at all, hiding an existing tool-grant override (e.g. Write files disabled) with no visibility or edit path
 /workflows/656ca387-e69c-474d-b7ff-5fd9eb017cc0/edit (composer PATCH mode, applies to any saved workflow)

SUMMARY

The composer's Canvas view exposes a full "Tools" tab in the per-agent config rail (Overview/Skills/Hooks/Tools/Config) with three real, togglable switches — "Read files", "Write files", "Execute commands" — that persist correctly in-session (verified: toggling "Write files" off for one agent, switching to the other agent's Tools tab, and switching back shows no leak and no reversion). The same composer's Simple view offers an equivalent per-agent "Overrides" row with a "Configure →" button that opens an inline "Advanced — <Agent Name>" panel. That panel's own subtitle explicitly enumerates its scope as "Validator · Gate · Model · Retry" plus a Skills library below it — there is no Tools sub-section anywhere in it. Searching the panel's full rendered text for "Tool grants" or "Read files" returns no match, for either agent, on both a fresh page load and after toggling a tool grant in Canvas view first. This means: (1) Simple view provides no way to view or edit an agent's tool-grant restrictions at all — a user working only in Simple view cannot discover or set them; (2) after a tool-grant override is set in Canvas view (e.g. disabling an agent's ability to write files, a meaningful capability restriction that changes what artifacts that step can produce for later steps), Simple view's "Overrides" pill row is identical for both agents (Validator Gate Retry Skills Configure →) with no "Tools" pill or any other indicator that one agent now has a restricted tool set — the two views disagree about what state exists for the same agent on the same in-session workflow.

REPRODUCTION

1. Sign in as qa-admin, navigate to <http://localhost:3000/workflows/656ca387-e69c-474d-b7ff-5fd9eb017cc0/edit>

(fresh load, Canvas view, "My prototype" — 2 agents: Documentation Agent, Spec Writer Agent).

1. Click the "Documentation Agent" node, open its "Tools" tab in the rail. Observe three switches:

Read files (on), Write files (on), Execute commands (disabled/off). Toggle "Write files" off.

Confirm via [role=switch] DOM reads: Write files now aria-checked="false".

1. Click the "Spec Writer Agent" node — confirm its own Tools tab independently still shows "Write files" true (no state leak between nodes). Click back to "Documentation Agent" — confirm "Write files" is still false there (no reversion).

1. Switch to "Simple" view. Observe both agent cards' "Overrides" row reads identically Validator | Gate | Retry | Skills | Configure → — no visual difference between the now-restricted Documentation Agent and the unrestricted Spec Writer Agent, and no "Tools" pill exists at all.

1. Click "Configure →" on the Documentation Agent card. Observe the inline "Advanced — Documentation Agent" panel opens with subtitle "Validator · Gate · Model · Retry" followed by a Skills library browser — confirmed via document.body.innerText search: no occurrence of "Tool grants" or "Read files" anywhere in the panel.

1. Reloaded fresh (/workflows/656ca387-e69c-474d-b7ff-5fd9eb017cc0/edit, no prior edit this time), switched straight to Simple view, and opened "Configure →" for the Spec Writer Agent — identical result: no Tools section, confirming this is a structural gap in the Simple-view panel, not contingent on the specific override made in step 2.

EXPECTED

Either the Simple view's per-agent "Configure →" panel should expose the same Tools grants (Read/Write files, Execute commands) that Canvas view's rail does — consistent with "Overrides" already listing Validator/Gate/Retry/Skills as editable categories — or, at minimum, the Simple view's Overrides row should visibly flag when an agent has a non-default tool-grant restriction set elsewhere in the same composer session, so the two views never silently disagree about what capabilities an agent actually has.

ACTUAL

Simple view has no Tools section anywhere in its per-agent configuration UI. A tool-grant override made in Canvas view is real (persists correctly, is not lost) but is completely invisible and unreachable from Simple view — no pill, no tab, no indicator of any kind — even though Simple view is otherwise a full editing surface for the same in-session workflow.

EVIDENCE

- Canvas view, Tools tab, "Write files" toggled off for Documentation Agent: `bug-hunter/evidence/workflows-id-edit/BUG-20260828-005500-workflows-id-edit/01-canvas-tools-tab-write-files-off.png`
- Simple view, both agents' Overrides row identical, no Tools pill: `bug-hunter/evidence/workflows-id-edit/BUG-20260828-005500-workflows-id-edit/02-simple-view-overrides-row-no-tools-pill.png`
- Simple view, "Configure →" panel for Documentation Agent — no Tools section present: `bug-hunter/evidence/workflows-id-edit/BUG-20260828-005500-workflows-id-edit/03-simple-configure-panel-no-tools-section-doc-agent.png`
- Simple view, "Configure →" panel for Spec Writer Agent (fresh reload, second repro) — same gap: `bug-hunter/evidence/workflows-id-edit/BUG-20260828-005500-workflows-id-edit/04-simple-configure-panel-no-tools-section-specwriter.png`

BROWSER SIGNALS

- Console: no relevant error observed.
- Network: no request involved; both the Canvas Tools toggle and the Simple "Configure →" panel

are purely client-side in-session composer state, confirmed via `document.body.innerText` substring search for "Tool grants"/"Read files" returning no match in the Simple panel on two independent fresh-page-load reproductions.

- State/URL: URL stays `/workflows/656ca387-e69c-474d-b7ff-5fd9eb017cc0/edit` throughout; no

changes were ever saved (Save workflow was never clicked), and the underlying

GET `/api/user-workflows/656ca387-e69c-474d-b7ff-5fd9eb017cc0` was confirmed unchanged (`updated_at` unchanged) after this investigation.

ID `BUG-20260828-005500-workflows-id-edit` Found by `bug-workflows-id-edit-r1`

Fingerprint `/workflows/{id}/edit|simple-view-agent-configure-panel|toggle-tool-grant-in-canvas-then-inspect-simple-view|no-tools-section-or-indicator-exists-in-simple-view`

Loading a saved-workflow launch panel with a nonexistent or malformed workflow id silently renders the Dashboard under the unchanged bogus URL

`/workflows/<nonexistent-uuid>/run`, `/workflows/<malformed-id>/run`

SUMMARY

Navigating directly to a saved workflow's dedicated launch route (`/workflows/{id}/run`) with an id that does not resolve to any saved workflow — either a well-formed but nonexistent UUID (`00000000-0000-0000-0000-000000000000`) or a syntactically invalid id (`not-a-valid-uuid`) — triggers a GET `/api/user-workflows/<id>` request that correctly 404s (`ApiError: Saved workflow not found`, logged to console as `cold-mount workflow-run fetch failed`). Instead of showing a not-found state, an error message, or performing an actual navigation to `/dashboard` or `/workflows`, the page's error-handling path silently renders the full authenticated Dashboard catalog content (`h1: "What would you like to build today?"`, workflow cards, "Jump back in") in place while `location.href` remains the original, unresolvable `/workflows/<id>/run` URL. The address bar and the rendered content permanently disagree — confirmed via direct `location.href` reads immediately after the swap and again after a subsequent full page load of a different URL and back. This reproduces identically for both a valid-shaped-but-nonexistent UUID and a completely malformed id string, meaning the run panel's cold-mount fetch-failure handler falls back to dashboard content unconditionally rather than surfacing the actual 404. This is a different mechanism from the ledger's `BUG-20260827-231407-register` (which is Next.js route resolution silently falling through to dashboard content for an **unmatched sub-path** under `/register`, with no API call involved at all): here the route itself matches correctly, an actual API request fires and genuinely 404s, and it is this page's own client-side error/not-found handling that discards the failure and renders the wrong content without ever updating the URL. It is also unrelated to D-05/D-10 (agent-count/estimate disagreements) and D-28 (the saved-workflow

detail view being unreachable) since a *valid* saved workflow's /run route (confirmed in the same session against 656ca387-e69c-474d-b7ff-5fd9eb017cc0, "My prototype") loads and binds correctly —

this defect is specific to the 404/not-found path for an id that does not resolve.

REPRODUCTION

1. Sign in as qa-admin. Confirm a real saved workflow's launch panel loads correctly:

navigate to `http://localhost:3000/workflows/656ca387-e69c-474d-b7ff-5fd9eb017cc0/run` — the

prototype wizard shell renders normally (h1 "Configure your prototype", Advanced button, etc.).

1. Navigate to `http://localhost:3000/workflows/00000000-0000-0000-0000-000000000000/run` (a

well-formed UUID that matches no saved workflow).

1. Observe the console logs `Failed to load resource: ... 404 ... /api/user-workflows/00000000-...`

and `cold-mount workflow-run fetch failed: ... ApiError: Saved workflow not found.`

1. Observe the rendered page: `document.querySelector('h1').textContent` reads "What would you

like to build today?" and the body shows the full Dashboard catalog (workflow cards, "Jump back in"), while `location.href` still reads

`http://localhost:3000/workflows/00000000-0000-0000-0000-000000000000/run` — confirmed via

direct `location.href` read, not a stale screenshot.

1. Repeat with a syntactically malformed id: navigate to

`http://localhost:3000/workflows/not-a-valid-uuid/run` — identical result: the same 404 + `cold-mount workflow-run fetch failed` console pair, Dashboard content rendered, URL stays on the bogus `/workflows/not-a-valid-uuid/run` path.

1. Repeated step 2 a second time from a fresh navigation — deterministic, same mismatch both times.

EXPECTED

An id that does not resolve to a real saved workflow should produce an explicit not-found state (e.g. "This workflow no longer exists" plus a link back to /workflows) or an actual navigation to a real destination, with the URL and the rendered content agreeing either way — consistent with how a valid id's launch panel correctly renders its own content at its own URL.

ACTUAL

The page's own client-side handling of the /api/user-workflows/<id> 404 discards the error and renders the Dashboard catalog's content in place, while location.href remains the original, non-existent /workflows/<id>/run URL indefinitely — a persistent URL/content mismatch specific to this route's not-found path.

EVIDENCE

- Before (valid saved workflow's own /run panel, for contrast): bug-hunter/evidence/workflows-id-run/BUG-20260828-010600-workflows-id-run/01-before-valid-workflow-run-panel.png
- Failure (nonexistent UUID, Dashboard renders under the bogus URL): bug-hunter/evidence/workflows-id-run/BUG-20260828-010600-workflows-id-run/02-failure-nonexistent-id-dashboard-mismatch.png
- Reproduced (malformed id, same mismatch): bug-hunter/evidence/workflows-id-run/BUG-20260828-010600-workflows-id-run/03-repro2-malformed-id-dashboard-mismatch.png
- Console excerpt: bug-hunter/evidence/workflows-id-run/BUG-20260828-010600-workflows-id-run/console.log

BROWSER SIGNALS

- Console: GET /api/user-workflows/<id> returns 404 in both cases; app logs

cold-mount workflow-run fetch failed: <id> ApiError: Saved workflow not found for each,

confirming the failure is caught but mishandled rather than silent/uncaught.

- Network: GET http://localhost:8000/api/user-workflows/00000000-0000-0000-0000-000000000000 and

GET http://localhost:8000/api/user-workflows/not-a-valid-uuid both return 404.

- State/URL: location.href remains the requested, unresolvable /workflows/<id>/run path

throughout in both cases; the rendered DOM (h1, catalog cards) matches /dashboard exactly, confirmed via direct DOM reads rather than a screenshot alone.

ID `BUG-20260828-010600-workflows-id-run` Found by `bug-workflows-id-run-r1`

Fingerprint `/workflows/<id>/run|workflow-run-panel-cold-mount|navigate-with-nonexistent-or-malformed-workflow-id|api-404-caught-then-dashboard-silently-rendered-under-stale-url`

35

MEDIUM

Run detail header's relative timestamp is stuck on "just now" for a run that finished 11+ hours ago, disagreeing with the version picker's own age label
`/runs/b9feac1c-ec21-4531-8ba7-bb391786993e` (and its Preview URL)

SUMMARY

The run detail page's left-hand header (below the run title, `data-testid="lane-run-meta"` area) shows a relative timestamp for when the run happened, followed by duration and token count (e.g. "just now · 18m 54s · 3.1M tokens"). For run `b9feac1c-ec21-4531-8ba7-bb391786993e` (`created_at: 2026-08-27T13:48:45Z`, `completed_at: 2026-08-27T14:07:40Z`, confirmed via `GET /api/runs/{id}`), the browser's own clock at the time of testing was `2026-08-28T01:16:56Z` — roughly 11 hours 9 minutes after the run completed. The header still reads "just now", which is wrong by over 11 hours. In the same page, in the same header row's "Version v1, choose version" dropdown, the app computes and displays the correct relative age for the identical run: "Version v1 · 11h ago". Two labels on the same page, both meant to express how long ago this run happened, disagree by 11+ hours — one is a live, correctly-computed value, the other is a stale/frozen "just now" that never updates. Duration ("18m 54s") and token count ("3.1M tokens") next to the broken timestamp are both individually correct against the API (`duration: 1134.4s`, `token_usage.total_tokens: 3105278`), so only the relative-time portion of the header is defective, not the whole meta line.

REPRODUCTION

1. Sign in as qa-admin, navigate to `http://localhost:3000/runs/b9feac1c-ec21-4531-8ba7-bb391786993e` (a completed run from the prior day).
1. Observe the header meta line directly under the run title/status pill: "just now · 18m 54s · 3.1M tokens".
1. Read `GET http://localhost:8000/api/runs/b9feac1c-ec21-4531-8ba7-bb391786993e — created_at`

is 2026-08-27T13:48:45.894220+00:00, completed_at is 2026-08-27T14:07:40.345371+00:00.

Read the browser's own clock via new Date().toISOString() — confirmed far later (2026-08-28T01:16:56Z at the time of this test), i.e. the run is genuinely ~11 hours old, not seconds old.

1. Click the "Version v1, choose version" button in the same header row to open the version listbox. Observe its own entry for this exact run/version: "Version v1 · 11h ago" — the correct, freshly-computed relative age for the same underlying completed_at.

1. Close the version picker, hard-reload the page fresh (page.goto to the same URL, not a soft navigation). Observe the header still reads "just now" immediately on load — this is not a transient state that later corrects itself; it is the header's steady-state value.

1. Repeat step 4 after the fresh reload — the version picker again correctly shows "11h ago" for the same run, reconfirming the two labels disagree deterministically, not as a one-off race.

EXPECTED

The header's relative timestamp should reflect the same real elapsed time the rest of the page computes correctly (e.g. "11h ago" or an equivalent accurate relative/absolute time), consistent with the version picker's own correct label for the identical run.

ACTUAL

The header is frozen on "just now" regardless of how much real time has actually elapsed since the run finished, while a different control in the same header row (the version picker) computes and displays the correct ~11-hour age for the same run.

EVIDENCE

- Before: bug-hunter/evidence/runs-id/BUG-20260828-011700-runs-id/01-before-run-detail-loaded.png
- Failure (header "just now" next to version picker showing "11h ago"): bug-hunter/evidence/runs-id/BUG-20260828-011700-runs-id/02-failure-header-vs-version-mismatch.png
- Reproduced on a fresh hard reload: bug-hunter/evidence/runs-id/BUG-20260828-011700-runs-id/03-repro-fresh-reload-still-just-now.png

BROWSER SIGNALS

- Console: no relevant error observed.
- Network: GET /api/runs/b9feac1c-ec21-4531-8ba7-bb391786993e returns 200 OK with accurate

created_at/completed_at timestamps; the API itself is correct, so this is a client-side rendering/formatting defect (likely the header's relative-time value being computed once from a stale/incorrect source, e.g. session-start time instead of completed_at, while the version picker correctly derives its label from the run/version's own timestamp).

- State/URL: URL stays on /runs/b9feac1c-ec21-4531-8ba7-bb391786993e throughout; verified via

direct new Date().toISOString() read that the browser's system clock itself is far past the run's actual completion time, ruling out a client-clock-skew explanation.

ID BUG-20260828-011700-runs-id Found by bug-runs-id-r1

Fingerprint /runs/[id]|run-detail-header-relative-time|load-completed-run-11h-old|header-reads-just-now-while-version-picker-reads-11h-ago

Steps tab footer's "input" token figure is ~20x too low and its own breakdown does not sum to the stated total

`/runs/b9feac1c-ec21-4531-8ba7-bb391786993e/steps`

SUMMARY

The Steps tab's bottom-left "TOKEN USAGE" footer (`3.1M total · 141.3K input · 192.8K output`) is

meant to break the run's total token count down into input vs. output. Its own numbers do not add up: $141.3K + 192.8K = 334.1K$, nowhere close to the stated `3.1M total` — a discrepancy of roughly 2.77M tokens. Cross-checking against `GET /api/runs/b9feac1c-ec21-4531-8ba7-bb391786993e`

confirms the real figures are `total_input_tokens: 2,915,058 (2.9M)`, `total_output_tokens: 190,220 (190.2K)`, `total_tokens: 3,105,278 (3.1M)` — input + output does correctly sum to total in the API data. So the "3.1M total" and "192.8K output" figures shown are each individually close to correct, but the displayed "input" figure (141.3K) is wrong by roughly 20x. Tellingly, the "141.3K input" element's own accessible/tooltip text reads "Total context sent: 2.9M (2.4M cached)" — the component has access to the correct 2.9M value in its own tooltip copy, but renders the wrong number (141.3K, which is closer to $\text{input} - \text{cache_read} - \text{cache_write} = 2,915,058 - 2,443,873 - 332,250 = 138,935 \approx 141.3K$ after rounding/estimation) as the primary visible label with no "excludes cached context" qualifier. A user reading only the visible text sees a self-contradictory breakdown: a stated total that its own two visible parts cannot produce.

REPRODUCTION

1. Sign in as qa-admin, navigate to `http://localhost:3000/runs/b9feac1c-ec21-4531-8ba7-bb391786993e/steps`

(a completed 4-agent PPT run).

1. Observe the "TOKEN USAGE" line pinned to the bottom-left of the Steps panel: `3.1M total · 141.3K input · 192.8K output`.

1. Compute $141.3K + 192.8K = 334.1K$ — confirm it does not approach `3.1M total`.

2. Fetch GET /api/runs/b9feac1c-ec21-4531-8ba7-bb391786993e with the current auth token and read

token_usage: total_input_tokens: 2915058, total_output_tokens: 190220,
total_tokens:

3105278 — confirms the real input figure is ~2.9M, not 141.3K, and that input+output does sum to total in the underlying data.

1. Hover/inspect the "141.3K input" element directly — its own tooltip/aria text reads "Total context sent: 2.9M (2.4M cached)", i.e. the component itself holds the correct 2.9M value but displays a different, much smaller number as the visible label.

1. Reload the page fresh (page.goto to the same URL) and re-read the footer via direct DOM text (element.innerText) — identical result: TOKEN_USAGE\n3.1M total\n\n141.3K input\n\n192.8K

output, confirming this is deterministic, not a transient render race.

EXPECTED

The visible "input" figure should either show the real total input tokens (2.9M, matching the API and the element's own tooltip) so that $\text{input} + \text{output} \approx \text{total}$, or, if the intent is to show only non-cached "new" input tokens, the label should say so explicitly (e.g. "141.3K new input") rather than being presented unqualified next to a total it cannot reconstruct.

ACTUAL

The footer shows 3.1M total · 141.3K input · 192.8K output — a self-contradictory breakdown where input+output (334.1K) is roughly 9x smaller than the stated total (3.1M), while the actual API-reported input token count (2.9M) matches what the same element's own tooltip already says.

EVIDENCE

- Before: bug-hunter/evidence/runs-id-steps/BUG-20260828-012500-runs-id-steps/01-before-steps-tab.png
- Failure (footer visible, tooltip shows the correct 2.9M contradicting the 141.3K label): bug-hunter/evidence/runs-id-steps/BUG-20260828-012500-runs-id-steps/02-failure-input-141k-vs-tooltip-2.9M.png

- After reload, still wrong: `bug-hunter/evidence/runs-id-steps/BUG-20260828-012500-runs-id-steps/03-after-reload-still-wrong.png`

BROWSER SIGNALS

- Console: no relevant error observed.
- Network: GET `/api/runs/b9feac1c-ec21-4531-8ba7-bb391786993e` returns 200 OK with

`token_usage.total_input_tokens = 2915058`, directly contradicting the rendered "141.3K input".

- State/URL: URL stays on `/runs/b9feac1c-ec21-4531-8ba7-bb391786993e/steps` throughout; confirmed

via direct DOM `innerText` reads (not just screenshot) both before and after a fresh reload.

ID `BUG-20260828-012500-runs-id-steps` Found by `bug-runs-id-steps-r1`

Fingerprint `/runs/[id]/steps|token-usage-footer|load-completed-run-steps-tab|input-token-figure-far-below-actual-breakdown-does-not-sum-to-total`

Deep-linking a specific agent's step URL never opens that agent's detail pane; it renders the plain unfiltered lane list instead

`/runs/b9feac1c-ec21-4531-8ba7-bb391786993e/steps/<agentId>` (e.g. ``/steps/ppt-brief-analyst``, ``/steps/ppt-composer``)

SUMMARY

The Steps tab's URL scheme includes a per-agent segment (`/runs/{id}/steps/{agentId}`), and the app itself generates and links to these URLs (e.g. the "Answer in Steps"/"Open in Steps" chat buttons, and the breadcrumb "Steps / <Agent Name>" shown once an agent is selected). The clear implication is that this URL should deep-link directly to that agent's own detail pane — its reasoning, full input prompt, agent output, and context-received sources. It does not. A fresh `page.goto` to any such URL (tested with all four real agent ids from this run — `ppt-brief-analyst`, `ppt-composer` — plus a nonexistent id, `nonexistent-agent-xyz`) renders the Steps tab correctly selected in the top tablist, but the content pane shows only the plain, unfiltered lane list of all four agent buttons (Presentation Strategist Agent, Deck Engineer Agent, Deck QA Agent, PPTX Code Generator) with no agent expanded, no breadcrumb, and no detail content — identical regardless of which agent id (real or fake) is in the URL. The only way to see an agent's detail is to manually click its row after the page has already loaded, at which point the breadcrumb "Steps / Presentation Strategist Agent" appears and the detail pane (reasoning, full input prompt, agent output, context sources) renders correctly — but this in-page click never changes the URL, so the state that was just achieved cannot be reloaded, shared, or bookmarked: reloading the exact same URL that is currently showing full agent detail throws that detail away and reverts to the bare list.

REPRODUCTION

1. Sign in as qa-admin. Resolve real agent ids for run `b9feac1c-ec21-4531-8ba7-bb391786993e` via `GET /api/runs/{id}/events?after=0` (agent_start events give `ppt-brief-analyst`, `ppt-composer`, `ppt-deck-qa-v2`, `ppt-code-generator`, displayed as "Presentation Strategist

Agent", "Deck Engineer Agent", "Deck QA Agent", "PPTX Code Generator" respectively).

1. Fresh page. goto to `http://localhost:3000/runs/b9feac1c-ec21-4531-8ba7-bb391786993e/steps/ppt-brief-analyst`.

Wait 2+ seconds for full settle. Observe: "Steps" tab is selected in the tablist, but the pane shows only the plain lane list of all 4 agent buttons — no breadcrumb, no expanded detail for "Presentation Strategist Agent" even though it is named directly in the URL.

1. Manually click the "Presentation Strategist Agent" button in that same list. Observe: the breadcrumb "Steps / Presentation Strategist Agent" now appears, and the detail pane renders ("Reasoning" toggle, full JSON spec output, "Full input prompt 7,804 chars", "Agent output 9.7k chars", "Context received — 2 sources fed in: prompt.md, Template: html-ppt-zhangzara-scatterbrain"). Confirm via the address bar that `location.href` did NOT change — it is still the exact same `/steps/ppt-brief-analyst` URL from step 2.

1. Fresh page. goto to that identical, unchanged URL again (equivalent to a reload). Observe: the detail pane just shown in step 3 is gone; the pane is back to the bare 4-button list with no breadcrumb and no selection — the same starting state as step 2, despite the URL never having changed and still literally naming this agent.

1. Repeated steps 2-4 on a second, different agent id, `/steps/ppt-composer` — identical result: fresh load shows only the bare list, manual click opens detail without changing the URL.

1. For contrast, navigated to a nonexistent agent id, `/steps/nonexistent-agent-xyz` — renders identically to the real ids on fresh load (bare list, Steps tab selected, no error, no 404), confirming the URL segment has no effect on the rendered content at all, valid or invalid.

EXPECTED

Loading `/runs/{id}/steps/{agentId}` — whether via a fresh navigation, a reload, or a shared/bookmarked link — should deep-link directly to that agent's own detail pane (matching what a manual click on that agent's row produces), since the app itself both names this URL shape and generates links into it. At minimum, the manual "select an agent" client action should push/

replace the URL to reflect the selection it just made, so the resulting state is reloadable.

ACTUAL

The {agentId} URL segment is inert on load: every fresh navigation or reload renders the same undifferentiated lane list regardless of which (or whether a valid) agent id is present. Detail can only be reached by an in-page click, and that click never syncs the URL, so the detail view is unreachable by direct navigation and is lost on reload.

EVIDENCE

- Before (fresh deep-link to /steps/ppt-brief-analyst, no detail shown): bug-hunter/evidence/runs-id-steps-agent/BUG-20260828-012700-runs-id-steps-agent/01-before-fresh-deeplink-no-detail.png
- Manual click on the same page reveals the detail, URL unchanged: bug-hunter/evidence/runs-id-steps-agent/BUG-20260828-012700-runs-id-steps-agent/02-manual-click-shows-detail.png
- Reloading that identical URL loses the detail again: bug-hunter/evidence/runs-id-steps-agent/BUG-20260828-012700-runs-id-steps-agent/03-after-reload-still-list-only.png

BROWSER SIGNALS

- Console: no relevant error observed on any of the fresh loads (real or fake agent id).
- Network: all page-load requests (/api/runs/{id}, /api/runs/{id}/events?after=0, /api/runs/{id}/family, /api/runs/{id}/sandbox, /api/runs/{id}/gate-events) return 200 OK identically for ppt-brief-analyst, ppt-composer, and nonexistent-agent-xyz — the data layer never distinguishes the URL's agent segment, confirming the gap is purely in client-side routing/selection logic, not a failed fetch.
- State/URL: location.href retains the full /steps/{agentId} path through every step, including immediately after the manual click that opens the detail pane — the URL is written once by navigation and never updated or read back by the selection logic.

ID `BUG-20260828-012700-runs-id-steps-agent` Found by `bug-runs-steps-agent-r1`

Fingerprint `/runs/[id]/steps/[agentId]|steps-tab-lane-detail-panel|fresh-navigation-or-reload-of-agent-deep-link|no-agent-detail-renders-only-list-view`

Workspace tab's "Code" file viewer is a live, freely-editable text editor with no read-only indication for a completed run

`/runs/b9feac1c-ec21-4531-8ba7-bb391786993e/workspace`

SUMMARY

First established that the Workspace tab correctly bounds the already-filed Files-tab

deliverable bug (BUG-20260828-012900-runs-id-files): the run's real `presentation.pptx`

binary deliverable IS listed here (under "Deliverables · 2", alongside `presentation.html`),

opens a "binary file — use Download" message, and downloads correctly (confirmed via

`GET /api/runs/{id}/sandbox/file?path=presentation.pptx => 200` and an actual browser download

event). The "All files" tree also matches the backend sandbox listing exactly — 36 files across

`ROOT/.agents/.browser/.verify/conversation_history/tmp`, matching

`GET /api/runs/{id}/sandbox's` file count exactly. So the run's own deliverable IS reachable somewhere in the UI (via Workspace), just not via Files.

While probing the file viewer itself, however, a separate defect surfaced. Selecting any text

file (`.json`, `.md` in its "Code" view — the raw-text mode toggled via the "Code"/"Preview"

button next to Download) renders the content inside a CodeMirror editor whose content root

(`.cm-content`) has `contentEditable === true` and a live text cursor. Clicking into it and

typing is accepted exactly like a real code editor: new characters are inserted at the cursor

position and the rendered text updates live. This reproduces on two independent files of two

different types — `.browser/deck-styles.json` (typing `BUGHUNTER_TEST_EDIT` inserted it

mid-line at the click position) and `PLANNER.md` in Code view (typing `MD_EDIT_TEST` inserted it

at the start of a line). Nothing on the page indicates the content is supposed to be read-only:

no lock icon, no "read-only" badge, no disabled/greyed styling, and critically no "unsaved

changes" indicator or Save/Discard control appears after typing — the UI gives no sign that

anything unusual just happened. The only two buttons present are "Code"/"Preview" (view-mode

toggle) and "Download". Switching to a different file and back re-fetches the original (unmodified) content from the server — confirmed via GET `/api/runs/{id}/sandbox/file?path=...`

firing again on return — and no PUT/POST/PATCH request was ever observed after typing, so the edit is not persisted server-side. But the client-side editing session itself has no read-only guard, meaning a user who types into what is presented as an inert "file preview" for a Done (immutable, completed) run gets no feedback that their keystrokes went anywhere, and loses them the instant they navigate to a sibling file or leave the tab — a misleading, silently-discarding "editor" masquerading as a passive viewer.

REPRODUCTION

1. Sign in as qa-admin, navigate to `/runs/b9feac1c-ec21-4531-8ba7-bb391786993e/workspace` (a completed "Ppt V2" run, header status "Done").

1. Click "All files" tab, expand `.browser`, click `deck-styles.json` to open it. Observe the Code-view pane renders with line numbers and syntax highlighting.

1. Run `document.querySelector('.cm-content').isContentEditable` in the console — confirmed `true`.

1. Click into the code pane at an arbitrary position and type `BUGHUNTER_TEST_EDIT`. Observe the text is inserted live into the rendered line (verified both via a DOM `textContent` check and a screenshot: `"borderRadiusPx": 0,BUGHUNTER_TEST_EDIT` appears on line 24). No read-only indicator, warning, or Save/Discard control appears anywhere before or after typing.

1. Click a different file (`PLANNER.md`), then click back to `deck-styles.json` — the injected text is gone; the file was silently re-fetched fresh from the server with no prompt about the discarded edit.

1. Repeated on a second, different file: open `PLANNER.md`, click its "Code" toggle button (next to Download) to switch from the default rendered-markdown Preview into raw Code view, click into `.cm-content` (confirmed `isContentEditable === true`), type `MD_EDIT_TEST` — inserted

live at the cursor position (confirmed via DOM text and screenshot).

1. Confirmed via `browser_network_requests` across both repro steps that only GET `/api/runs/{id}/sandbox/file?path=...` calls fired — no PUT/POST/PATCH request was ever made after typing, so the app never attempts to persist the edit; the defect is purely the unguarded client-side editable state and its silent-discard behavior.

EXPECTED

A file-preview pane for a completed, immutable run's workspace should render content read-only (e.g. `contentEditable !== true`, or an explicit read-only CodeMirror configuration), or if editing is an intentional feature, it should be clearly indicated (an "editing" label, a visible Save/Discard affordance, and a warning before the edit is discarded by switching files).

ACTUAL

The Code-view file pane is a fully live, keystroke-accepting text editor with no read-only enforcement, no visual indication that content is (or should be) immutable, and no warning that typed edits are silently discarded the instant the user selects a different file — reproduced on two different files and file types (`.json`, `.md`).

EVIDENCE

- Before (Deliverables list, `presentation.pptx` correctly present — establishes Workspace is NOT the materially-worse "nowhere in the UI" case): `bug-hunter/evidence/runs-id-workspace/BUG-20260828-013500-runs-id-workspace/01-before-deliverable-pptx-listed.png`
- Before (JSON file opened in Code view, unmodified): `bug-hunter/evidence/runs-id-workspace/BUG-20260828-013500-runs-id-workspace/02-before-json-code-view.png`
- Failure (typed text inserted into `deck-styles.json`, no read-only indication): `bug-hunter/evidence/runs-id-workspace/BUG-20260828-013500-runs-id-workspace/03-failure-json-accepts-typed-edit.png`
- Failure, second file/type (typed text inserted into `PLANNER.md` Code view): `bug-hunter/evidence/runs-id-workspace/BUG-20260828-013500-runs-id-workspace/04-failure-md-code-view-accepts-typed-edit.png`

BROWSER SIGNALS

- Console: no relevant error observed.

- Network: GET `/api/runs/{id}/sandbox/file?path=...` fires on every file selection (200 OK);

no write-type request (PUT/POST/PATCH) was ever observed following a typed edit, in

either repro.

- State/URL: URL stays on `/runs/{id}/workspace` throughout; `document.querySelector('.cm-content').isContentEditable`

confirmed `true` on both files tested; re-selecting a previously-edited file re-fetches and

shows the original, unmodified server content.

ID `BUG-20260828-013500-runs-id-workspace` Found by `bug-runs-id-workspace-r1`

Fingerprint `/runs/[id]/workspace|workspace-file-viewer-code-mode|click-into-cm-content-and-type|editor-accepts-arbitrary-keystrokes-no-readonly-lock-no-save-affordance`

For a failed run with no Preview tab, `/preview/full` silently renders the Audit trail instead of an honest "no deliverable" state

`/runs/{id}/preview/full`

SUMMARY

`/runs/{id}/preview/full` is the dedicated route whose entire purpose is to force-select and show the run's Preview tab (the deliverable view). For a run whose pipeline failed before producing any deliverable, no Preview tab exists at all — confirmed the run detail page's tablist for this run is exactly `Steps | Files | Workspace | Audit`, no Preview entry. Two different code paths handle "the tab this route wants doesn't exist" completely differently: plain `/runs/{id}` (no query/sub-path) shows **no tab selected** and a neutral placeholder, "Output will appear here" — an honest, uncommitted default. But `/runs/{id}/preview/full` instead silently auto-selects the **Audit** tab and renders its full content (32 records, a governance/security log unrelated to a "preview") with no banner, message, or visual cue explaining that the requested Preview view isn't available and something else is being shown instead. This is a different mechanism from the already-known D-13 (blank pane when an incompatible renderer is force-selected on an existing Preview tab) and from the cancelled-run case on this same route (which correctly shows an honest "This run was cancelled ... stopped before producing a deliverable" empty state on the still-present Preview tab) — here there is no Preview tab to show an empty state on, and the route quietly substitutes an entirely unrelated tab's content instead of degrading honestly the way the cancelled-run case does one state over.

REPRODUCTION

1. Sign in as qa-admin. Navigate to `http://localhost:3000/runs/d6e425b6-df0d-4f54-9bba-f4a771e33812`

(a failed PPT run, seeded fixture per the worker brief). Observe the tablist reads

`Steps | Files | Workspace | Audit` (no Preview tab exists for this run), no tab shows [selected], and the content area shows the neutral placeholder text "Output will appear

here".

1. Navigate to `http://localhost:3000/runs/d6e425b6-df0d-4f54-9bba-f4a771e33812/preview/full`

(same run, the dedicated full-preview route). Observe: the tablist is identical

(Steps | Files | Workspace | Audit), but this time **Audit is [selected]** and the pane below renders a full Audit trail — "32 records", a governance/security/activity summary, Gate 32, and a scrollable list of 32 "audit logger" entries — with no text anywhere on the page indicating this is a fallback, or that a Preview was requested but is unavailable.

1. Reloaded (page.go to) the same `/preview/full` URL a second time — identical result: Audit tab selected, same 32-record trail rendered, deterministic across reloads.

1. For contrast, the cancelled run (`a8dfa959-e233-4ddf-87ce-d9a942cefde3`) on this exact same `/preview/full` route DOES keep the Preview tab selected and shows an honest inline message ("This run was cancelled / The run was stopped before producing a deliverable / Open the Thinking tab to view details") — confirming the route is capable of degrading honestly when a Preview tab exists; the failed-run case fails specifically because no Preview tab exists at all and the fallback logic picks Audit instead of mirroring the plain route's neutral, nothing-selected placeholder.

EXPECTED

When the requested Preview view has no tab to render (no deliverable was ever produced), `/preview/full` should degrade the same way the plain `/runs/{id}` route already does — no tab selected, neutral "nothing to show" placeholder — or explicitly explain why a different tab is shown instead. It should not silently substitute a materially different tab's real content (Audit) with no explanation.

ACTUAL

`/preview/full` auto-selects and fully renders the Audit tab for a failed run, with no visual or textual indication that this happened, while the equivalent plain route for the identical run

correctly shows no tab selected and a neutral empty placeholder.

EVIDENCE

- Before (plain `/runs/{id}` route, no tab selected, neutral placeholder): bug-hunter/evidence/runs-id-preview-full/BUG-20260828-014416-runs-id-preview-full/01-before-plain-route-no-tab-selected.png
- Failure (`/preview/full`, Audit tab silently selected with full trail content): bug-hunter/evidence/runs-id-preview-full/BUG-20260828-014416-runs-id-preview-full/02-failure-preview-full-shows-audit.png
- Reproduced on a second fresh navigation: bug-hunter/evidence/runs-id-preview-full/BUG-20260828-014416-runs-id-preview-full/03-repro2-preview-full-shows-audit.png

BROWSER SIGNALS

- Console: no relevant error observed.
- Network: no failed request; both routes fetch the same run data successfully, the divergence

is purely in client-side tab-selection/fallback logic.

- State/URL: `location.href` correctly stays on `/runs/{id}/preview/full` throughout; the tablist DOM confirmed via accessibility snapshot (tab "Audit" [selected]) on both reproduction attempts, versus no [selected] tab and a distinct placeholder string on the plain route for the identical run id.

ID `BUG-20260828-014416-runs-id-preview-full` Found by `bug-runs-id-preview-full-r1`

Fingerprint `/runs/{id}/preview/full|preview-tab-fallback|navigate-to-preview-full-for-failed-run-with-no-preview-tab|silently-renders-audit-tab-instead-of-empty-state`

Diverted run's Preview tab shows the "waiting for live output" placeholder even though the run is permanently terminal

`/runs/{id}/stream`

SUMMARY

Loading `/stream` for a run whose status is `diverted` (the API confirms `status: "diverted"` with `completed_at` already set — a permanently terminal state, not paused-and-resumable; only 2 of the run's 3 agents ever executed and `output/deliverable_filename` are both `null`) auto-selects the Preview tab and renders the plain text "Output will appear here". That is the exact same neutral placeholder a genuinely in-progress/live run shows while streaming output has not arrived yet — it implies output is still forthcoming. For this run, nothing further will ever arrive: the pipeline stopped for good at the divert point with no deliverable and no path to auto-resume. Contrast with the sibling terminal state on the identical route: the cancelled run (`a8dfa959-e233-4ddf-87ce-d9a942cefde3`) instead shows an explicit, run-status-aware message ("This run was cancelled / The run was stopped before producing a deliverable...") rather than the generic live placeholder — proving the app is capable of degrading honestly for a terminal state without a deliverable, and simply fails to do so for `diverted`. This is a different mechanism from the already-filed `BUG-20260828-014416-runs-id-preview-full` (which is about `/preview/full` silently substituting the Audit tab's content for a run with `*no Preview tab at all*`); here the Preview tab genuinely exists, is correctly selected, and the bug is purely that its empty-state copy is state-unaware and misleadingly implies the run is still live.

REPRODUCTION

1. Sign in as `qa-admin`. Navigate to `http://localhost:3000/runs/940ca699-b21b-4666-8e44-3370a08a4561/stream` (seeded diverted run "Ex A4 Human Divert", per the worker brief's resolved instances).
1. Observe the header badge reads "Diverted", "2/3 agents", and the transcript ends with "Run

started" / "Review approved — build continues" with no further activity and no chat composer (consistent with D-19, not re-filed here).

1. Observe the Preview tab is [selected] in the tablist and the content pane shows only the plain-text placeholder "Output will appear here" — identical wording/markup to what an actively-streaming, in-progress run shows while awaiting its first output chunk.

1. Confirmed via GET /api/runs/940ca699-b21b-4666-8e44-3370a08a4561: "status": "diverted", "completed_at": "2026-08-24T13:27:57.322032+00:00" (already set — terminal), "output": null, "deliverable_filename": null. Waited 30s on the page and re-checked browser_network_requests: no repeated polling of /events occurred, ruling out an active "still connecting" state client-side — the UI has already given up polling, yet still shows the "waiting" copy.

1. Reloaded the page (fresh page.goto) and repeated steps 2-4 — identical result both times.
2. For contrast, loaded /runs/a8dfa959-e233-4ddf-87ce-d9a942cefde3/stream (cancelled run): Preview tab is also [selected], but the pane instead shows a status-specific message, "This run was cancelled — The run was stopped before producing a deliverable. Open the Thinking tab to view details." (the "Thinking tab" phrasing is the already-known D-08, not re-filed).

EXPECTED

A run in a permanently terminal state with no deliverable should show a state-aware empty message on its Preview tab (as the cancelled run does), not the same "Output will appear here" copy used for an actively streaming run whose output has simply not arrived yet.

ACTUAL

The diverted run's Preview tab renders the generic live-streaming placeholder text, giving no indication that the run is over and no output is ever coming.

EVIDENCE

- Cancelled run's honest, status-specific empty state (for contrast):

bug-hunter/evidence/runs-id-stream/BUG-20260828-014937-runs-id-stream/01-cancelled-run-honest-empty-state.png

- Diverted run showing the misleading generic "Output will appear here" placeholder:

bug-hunter/evidence/runs-id-stream/BUG-20260828-014937-runs-id-stream/02-diverted-run-live-placeholder.png

BROWSER SIGNALS

- Console: none observed.
- Network: no SSE/WebSocket connection is opened for any tested run (completed, failed,

cancelled, diverted) — all use polling GET /api/runs/{id}/events?after=N, and none of the four statuses showed a reconnect loop; polling correctly stops once settled.

- State/URL: GET /api/runs/940ca699-b21b-4666-8e44-3370a08a4561 confirms status: "diverted",

completed_at set, output: null, deliverable_filename: null.

ID `BUG-20260828-014937-runs-id-stream` Found by `bug-runs-id-stream-r1`

Fingerprint `/runs/{id}/stream|preview-tab-empty-state|load-stream-for-a-diverted-run|shows-generic-live-waiting-placeholder-instead-of-honest-terminal-message`

Version picker label is stuck on "Version v1" while viewing a real, distinct v2

`/runs/77f74563-fa19-4f0b-84fd-d0224f89a54a/versions/2` (root run has a genuine 2-member

SUMMARY

This is distinct from the already-known D-12 (a NONEXISTENT version like `/versions/99` silently serving v1 with no error). Here the requested version genuinely exists and the CORRECT content is served — the chat transcript, deliverable filename

(`luxury-asset-marketplace-jets-yachts-cars.html`), 5-agent pipeline, and rendered prototype all

correctly reflect the v2 revision (a "replace flat gray placeholders with inline SVG" pass). But

the version-picker control in the run-detail chrome — both its collapsed button (`aria-label`

"Version v1, choose version", visible text "Version v1") and its expanded listbox (option "Version

v1" marked `[active]` `[selected]`, option "Version v2" NOT selected) — falsely claims the page is

showing v1, when it is actually showing v2. So the app doesn't just fail to error on a bad

request (D-12); when given a GOOD request for a real second version, it renders the right content

but mislabels which version that content is, in the one UI element whose entire job is to tell the

user which version they're looking at. This reproduced identically on the initial navigation and

again after a full page reload of `/versions/2`, so it is not a one-off render race.

REPRODUCTION

1. Sign in as qa-admin. Navigate directly to

`http://localhost:3000/runs/77f74563-fa19-4f0b-84fd-d0224f89a54a/versions/1`.

Confirm the

picker button reads "Version v1" and the chat transcript shows the original "Luxury Asset

Marketplace" prototype run (1 revision, "Delivered" once).

1. Navigate to `http://localhost:3000/runs/77f74563-fa19-4f0b-84fd-d0224f89a54a/versions/2`

(fresh navigation, not a picker click).

1. Observe the chat transcript, deliverable filename, and 5-agent pipeline summary — all correctly reflect the v2 revision ("Replace every flat gray image placeholder..." run).

1. Observe the version-picker button next to Share/Download: it still reads "Version v1". Open it — the listbox shows "Version v1" as the selected/active option and "Version v2" as unselected, even though v2's content is what's on screen.

1. Reload the page at the same `/versions/2` URL — the mislabel persists identically.

EXPECTED

When `/versions/2` renders v2's content, the version-picker button and listbox should read/select "Version v2", consistently with the URL and the content actually shown.

ACTUAL

The picker button and its listbox both claim "Version v1" is selected/current, while the visible page content is unambiguously v2's.

EVIDENCE

- Before (v1, correct): `bug-hunter/evidence/runs-id-versions/BUG-20260828-020900-runs-id-versions/01-before-versions-1.png`
- Failure (v2 content, picker says v1): `bug-hunter/evidence/runs-id-versions/BUG-20260828-020900-runs-id-versions/02-failure-versions-2-picker-says-v1.png`
- After reload, still broken: `bug-hunter/evidence/runs-id-versions/BUG-20260828-020900-runs-id-versions/03-after-reload-still-broken.png`

BROWSER SIGNALS

- Console: none observed (0 errors)
- Network: `/api/runs/{root_id}` and `/api/runs/{root_id}/family` both return correct data

(family confirms v1/v2 with correct `revision_index`); the version-number mismatch is a frontend label/state bug, not a backend data problem

- State/URL: URL correctly stays at `/versions/2` throughout; only the picker's own label/selection state is wrong

Fingerprint /runs/[id]/versions/2|version-picker-label|load-existing-second-version|header-and-picker-both-claim-v1-while-serving-v2-content

Library agent detail's "Save agent" Config override silently discards the change with no API call

`/library/agents/<agentId>` (e.g. `/library/agents/material-analyzer`)

SUMMARY

The agent detail drawer's Config tab (reached via any agent's "Configure →" affordance on `/library`, e.g. the Architecture Agent at `/library/agents/material-analyzer`) presents a "Model" override selector, a "Reset" button, and a "Save agent" button, with the caption "Overrides for this agent. Defaults inherit from the workflow." Changing the Model from "Default" to any other option (e.g. "Claude Sonnet 4.5") and clicking "Save agent" fires no network request whatsoever — no PUT/POST/PATCH to any agent or config endpoint. The button simply closes the drawer, navigating back to `/library`. Reopening the same agent's detail and Config tab shows the Model reverted to "Default" — the edit was silently discarded. The button gives no error, no toast, and no visual difference from a successful save, so a user has no way to know their override was never applied. This was reproduced twice (once via UI click sequence capture, once via a clean isolated repro run) with identical results both times.

REPRODUCTION

1. Sign in as qa-admin, go to `/library`, Agents tab.
2. Open any agent via its "Configure →" affordance (e.g. Architecture Agent, `/library/agents/material-analyzer`).
 1. Click the "Config" tab in the drawer.
 2. Click the "Model for <Agent>" dropdown and select any non-Default option (e.g. "Claude Sonnet 4.5").
 1. Click "Save agent".
 2. Observe: the drawer closes, URL returns to `/library`; check Network requests since the click — none were fired.
1. Navigate back to `/library/agents/material-analyzer`, open Config tab again.
2. Observe: Model shows "Default" again — the change from step 4 is gone.

EXPECTED

Either the Model override is actually persisted (a network call is made and the value survives a reload), or, if per-agent overrides genuinely only make sense inside a specific workflow context (as the caption "Defaults inherit from the workflow" suggests), the library's read-only agent-detail view should not present editable Model/Validator/Gate/Retry controls with a functioning-looking "Save agent" button that implies the change takes effect.

ACTUAL

Selecting a new Model and clicking "Save agent" makes no API call, discards the edit, and silently closes the drawer with no feedback of any kind — success or failure.

EVIDENCE

- Before: bug-hunter/evidence/library-agents-id/BUG-20260828-025950-library-agents-id/01-before-model-default.png
- Selected, before save: bug-hunter/evidence/library-agents-id/BUG-20260828-025950-library-agents-id/02-selected-sonnet45-before-save.png
- After Save (drawer closed): bug-hunter/evidence/library-agents-id/BUG-20260828-025950-library-agents-id/03-after-save-drawer-closed.png
- After reload (reverted to Default): bug-hunter/evidence/library-agents-id/BUG-20260828-025950-library-agents-id/04-after-reload-still-default.png

BROWSER SIGNALS

- Console: none observed
- Network: zero requests fired by the "Save agent" click (verified via

browser_network_requests immediately after each of two repro runs) — only the routine

GET /library?_rsc=... navigation request from the drawer closing

- State/URL: URL returns to /library; reopening /library/agents/material-analyzer → Config tab shows Model back at "Default"

ID `BUG-20260828-025950-library-agents-id` Found by `bug-library-agents-id-r1`

Fingerprint `/library/agents/<id>|agent-detail-config-tab-save|change-model-then-save|change-not-persisted-no-network-call`

Skill IDs with no hyphen (single-word slug) silently fall back to the Library listing instead of rendering the skill detail page

```
/library/skills/<skillId> (single-segment ids: `accessibility`, `benchmark`, `seo`)
```

SUMMARY

Navigating directly to `/library/skills/<skillId>` renders the skill detail page correctly for hyphenated ids (e.g. `windows-desktop-e2e`, `html-deck-to-pptx` — verified via a full markdown render matching the API's content field byte-for-byte at the tail, plus tag chips at the bottom). However, for the 7 skills whose id has no hyphen at all (single-word slug — `accessibility`, `benchmark`, `emoji`, `joke`, `poet`, `rhyming`, `seo`, `per` GET `/api/skills/library`), the exact same route pattern silently renders the full Library catalog listing (Agents/Skills/Hooks tabs, all 186 skill cards, category pills) instead of that skill's detail content. The URL bar stays on `/library/skills/accessibility` (or `/benchmark`, `/seo`) throughout — confirmed via `location.href` — and no console error or failed network request occurs; GET `/api/skills/library` (the list endpoint, not a per-skill fetch) is the only skills-related request fired, meaning the page never even attempts to resolve `accessibility` as a specific skill id — it just renders the list route's content under the detail route's URL. This is a distinct failure shape from Emerging bug class 4 (unmatched/404 route silently rendering the Dashboard): here the id is a real, valid resource that exists in the API response, the fallback target is the Library list (not the Dashboard), and the trigger is a hyphen-free segment specifically, not an arbitrary bad path.

REPRODUCTION

1. Sign in as qa-admin. From a fresh `about:blank`, navigate directly to `http://localhost:3000/library/skills/windows-desktop-e2e` (a hyphenated id). Confirm the detail page renders correctly: heading, `## Related Skills` section, "Copy content"/"Copy" buttons, and the page's `innerText` tail matches the API's content field tail exactly, plus

the skill's tags (windows, desktop, e2e) rendered as chips at the bottom.

1. From a fresh `about:blank`, navigate directly to

`http://localhost:3000/library/skills/accessibility` (a single-word, hyphen-free id that is a real skill per the API: `display_name: "Accessibility (WCAG 2.2)"`).

1. Observe: the URL stays `/library/skills/accessibility` (confirmed via `location.href`), but the rendered content is the full Library catalog page — "94 agents · 186 skills · 8 hooks ·

tap any item to see its capabilities" header, the Agents/Skills/Hooks tab pills, all 9

category pills, and a full grid of skill cards starting with "Accessibility (WCAG 2.2)" as

just the first list card, not a detail view.

1. Confirm no console error and no failed network request; the only skills-related request fired on this load is `GET /api/skills/library` (the bulk list endpoint), not a per-skill lookup.

1. Repeated the exact sequence (fresh `about:blank` → direct navigation) with two more independent single-word ids: `benchmark` and `seo`. Both reproduce identically — URL stays on the `/library/skills/<id>` detail path, content is the full Library listing page.

EXPECTED

`/library/skills/accessibility`, `/library/skills/benchmark`, and `/library/skills/seo` should

render each skill's own detail page (heading, description, full markdown content, tags,

"Copy content"/"Copy" controls), exactly as every hyphenated skill id does.

ACTUAL

For every skill whose id contains no hyphen, the detail route silently renders the Library catalog listing page instead, under the correct detail URL, with no error, no redirect, and no indication to the user that the specific skill they intended to view was never resolved or rendered.

EVIDENCE

- Before (hyphenated id `windows-desktop-e2e`, detail renders correctly): `bug-hunter/evidence/library-skills-id/BUG-20260828-030430-library-skills-id/01-`

before-hyphenated-id-works.png

- Failure (accessibility, library listing renders under the detail URL): bug-hunter/evidence/library-skills-id/BUG-20260828-030430-library-skills-id/02-failure-accessibility-falls-back.png
- Reproduced independently (seo, same fallback): bug-hunter/evidence/library-skills-id/BUG-20260828-030430-library-skills-id/03-repro2-seo-falls-back.png

BROWSER SIGNALS

- Console: no errors logged in any of the three failing cases.
- Network: only GET /api/skills/library (the bulk list) fires; no failed request, no per-skill

fetch attempt observed, confirmed via browser_network_requests on each failing navigation.

- State/URL: location.href remains the correct /library/skills/<id> detail path in every case

(confirmed via direct location.href reads), while the rendered DOM content is the Library

listing page's markup, not the detail page's.

ID BUG-20260828-030430-library-skills-id Found by bug-library-skills-id-r1

Fingerprint /library/skills/<id>|skill-detail-route-match|navigate-directly-to-a-no-hyphen-skill-id|library-listing-renders-instead-of-detail

Hook detail's "Compatible agents" list references nonexistent agent ids not found in the Agents library

`/library/hooks/<hookId>` (reproduces on 6 of the 8 hooks: ``post-design-quality``, ``session-start``, ``stop-console-log``, ``stop-format-typecheck``, and more)

SUMMARY

Each hook detail page renders a "Compatible agents" section (sourced directly from the backend's `compatible_agents` array on `GET /api/hooks/library`, verified byte-for-byte against the rendered chips) that is meant to tell a user which agents this hook can meaningfully be attached to. For most hooks, one or more of the listed agent ids do not correspond to any real agent in `GET /api/agents/library` — the ground-truth catalog of the app's 94 agents. Cross-checking all 8 hooks' `compatible_agents` against the real agent id set shows: `post-design-quality` lists `html-prototype-builder`, `prototype-polisher`, and `ppt-slide-architect` — none exist (the app has no agent by any of those ids); `session-start` and `stop-session-end` both list `requirements-analyst` — does not exist (the closest real agent is `chat-requirements`); `stop-console-log` lists `html-prototype-builder` — does not exist; `stop-format-typecheck` lists both `html-prototype-builder` and `prototype-polisher` — neither exists. Only `post-quality-gate`, `pre-config-protection`, and `pre-gateguard` have compatible-agent lists that are fully valid. The chips themselves are plain non-interactive `<div>`s (confirmed via DOM inspection: no `<a>` wrapper, no `onclick`, default `cursor: auto`) — a user cannot click through to verify, so the stale/fictional names are simply presented as fact. This directly misleads a user deciding which agent to attach a hook to via the documented "Advanced → Skills & Hooks panel" workflow described in the same page's "How to use" section.

REPRODUCTION

1. Sign in as qa-admin. Fetch the ground truth: `GET /api/agents/library` (94 real agent ids) and `GET /api/hooks/library` (8 hooks, each with a `compatible_agents` array).

1. Navigate to `http://localhost:3000/library/hooks/post-design-quality`. Observe the "Compatible

agents" section renders four chips: `app-ux-design`, `html-prototype-builder`, `prototype-polisher`, `ppt-slide-architect`.

1. Confirm via the API dump that `app-ux-design` is a real agent, but `html-prototype-builder`, `prototype-polisher`, and `ppt-slide-architect` are absent from the full 94-agent list — no agent by those ids exists anywhere in the app.

1. Navigate to `http://localhost:3000/library/hooks/session-start`. Observe "Compatible agents"

renders `domain-analyst`, `requirements-analyst`, `app-user-stories`, `epic-architect`, `app-code-generator`. Confirm `requirements-analyst` does not exist in the real agent list (verified: only `chat-requirements` exists, a different id).

1. Confirm the chips are non-interactive: `document.evaluate` on the chip container's children shows plain DIV elements with no `<a>` ancestor, no `onclick` handler, and `cursor: auto` — there is no way to click through and discover the mismatch from the UI itself.

1. Repeated the cross-check programmatically for all 8 hooks: `stop-console-log` and `stop-format-typecheck` also reference `html-prototype-builder/prototype-polisher`, the same

nonexistent ids — a systemic data-integrity issue in the hooks library's seed data, not a one-off typo.

EXPECTED

Every agent id listed in a hook's "Compatible agents" section should correspond to a real, resolvable agent in the Agents library, so the guidance is actionable and trustworthy.

ACTUAL

Six of the eight hooks list one or more agent ids (`html-prototype-builder`, `prototype-polisher`, `ppt-slide-architect`, `requirements-analyst`) that do not exist anywhere in the 94-agent Agents library, presented as plain unlinked text with no way for the user to discover the mismatch short

of independently checking the API.

EVIDENCE

- post-design-quality detail showing the 3 fictional agent chips: bug-hunter/evidence/library-hooks-id/BUG-20260828-030700-library-hooks-id/01-post-design-quality-fake-agents.png
- session-start detail showing the requirements-analyst fictional chip (second, independent repro): bug-hunter/evidence/library-hooks-id/BUG-20260828-030700-library-hooks-id/02-session-start-fake-agents-repro.png

BROWSER SIGNALS

- Console: no relevant error observed.
- Network: GET /api/hooks/library and GET /api/agents/library both return 200 OK; the mismatch is a data-content issue between the two payloads, not a request failure.
- State/URL: URL stays on the respective /library/hooks/<id> detail path throughout; confirmed via direct DOM query that the rendered chips are non-interactive plain <div>s.

ID BUG-20260828-030700-library-hooks-id Found by bug-library-hooks-id-r1

Fingerprint /library/hooks/<id>|compatible-agents-list|render-hook-detail-page|list-contains-agent-ids-absent-from-agents-library

Pipeline model selector enforces no tier restriction; a basic-tier account can select and persist the most expensive "powerful" models

`/settings/ai-model`

SUMMARY

The Pipeline Model selector on `/settings/ai-model` presents all 9 catalog models — including the two "powerful" (premium/most expensive) tier models, Claude Opus 4.5 and Claude Opus 4.6 — to every account with no differentiation by subscription tier whatsoever: no lock icon, no "upgrade to unlock" copy, no disabled option, no tier badge on the option itself (contrast each option's own tier metadata: "fast"/"balanced"/"powerful" is shown as descriptive text, not as a gate).

Confirmed both at the API and the UI: GET `/api/settings/preferences` for a qa-basic tier account (tier: "basic") returns the identical 9-model `available_models` catalog as the qa-admin/enterprise account, and PUT `/api/settings/preferences` with

`preferred_model: "eu.anthropic.claude-opus-4-6-v1"` (the single most expensive/most capable

model in the catalog) is accepted with a plain 200 OK and persists — no 403, no validation

error, no downgrade. The same is reproducible end-to-end through the real UI as the basic-tier

user: selecting "Claude Opus 4.5 · powerful" from the dropdown and clicking "Save" succeeds

identically to selecting any other model, shows the same "Model preference saved" confirmation,

and survives a reload. This is a genuine gap in the app's own stated tier model (the app

elsewhere gates capabilities by tier, e.g. `security_gated/user_allowed` flags on

`/api/capabilities`, and the login response itself carries tier: "basic") — the single most

consequential, cost-driving setting on this whole page has no tier enforcement at all, contrary

to what a "Pipeline model" picker on a subscription product would be expected to gate.

REPRODUCTION

1. Sign in as qa-admin, confirm `/settings/ai-model` lists all 9 models including "Claude Opus 4.5 · powerful" and "Claude Opus 4.6 · powerful" with no lock/tier indicators.

1. Via API, log in as `qa-basic@flowinqa.com / flowin-e2e-pass` (tier: "basic" per the login response) and GET `/api/settings/preferences` — the `available_models` array returned is byte-for-byte identical to the admin/enterprise account's, all 9 models, no per-tier filtering.

1. PUT `/api/settings/preferences` as the same basic-tier account with `{"preferred_model": "eu.anthropic.claude-opus-4-6-v1"}` — response is 200 OK, `preferred_model` is set to the Opus 4.6 id; a follow-up GET confirms it persisted.

1. Switch the same basic-tier session into the real UI: sign in as qa-basic in the browser, navigate to `/settings/ai-model` — the dropdown shows "Claude Opus 4.6 · powerful" already selected (the value from step 3), with no restriction badge, warning, or disabled state anywhere on the page.

1. From the UI, select a second premium model, "Claude Opus 4.5 · powerful", and click "Save" — the button is enabled (not gated), the request succeeds, and "Model preference saved" displays exactly as it does for any other model choice.

1. Reload `/settings/ai-model` as the same basic-tier account — "Claude Opus 4.5 · powerful" remains selected, confirming the change is genuinely persisted, not a stale client state.

1. Restored: basic-tier account's `preferred_model` reset to `null` (its original state) and qa-admin's `preferred_model` reset to `eu.anthropic.claude-haiku-4-5-20251001-v1:0` (its original selection) via the same API, both confirmed via a follow-up GET.

EXPECTED

A subscription-tiered product should restrict access to its most expensive/most capable models by account tier — e.g. a "basic" tier account should not be able to select or persist "powerful" tier models (Opus 4.5/4.6) without an upgrade path, mirroring the tier-gating pattern already present elsewhere in the app (`/api/capabilities's security_gated/user_allowed` flags, and the account object's own `tier` field).

ACTUAL

There is no tier restriction anywhere in this flow — not in the API's model catalog, not in the save endpoint's validation, and not in the UI's rendering of the selector. A basic-tier account can select, save, and persist the single most expensive model in the catalog exactly as freely as an enterprise admin account.

EVIDENCE

- Basic-tier account with Opus 4.6 already selected (set via API), no restriction UI present: bug-hunter/evidence/settings-ai-model/BUG-20260828-031200-settings-ai-model/01-basic-tier-opus46-preselected-no-restriction.png
- Basic-tier account selects Opus 4.5 via the real UI and Save succeeds normally: bug-hunter/evidence/settings-ai-model/BUG-20260828-031200-settings-ai-model/02-basic-tier-opus45-saved-successfully.png
- Reload confirms Opus 4.5 persisted for the basic-tier account: bug-hunter/evidence/settings-ai-model/BUG-20260828-031200-settings-ai-model/03-basic-tier-reload-opus45-persisted.png

BROWSER SIGNALS

- Console: no relevant error observed.
- Network: GET /api/settings/preferences (both accounts) and PUT /api/settings/preferences

(basic-tier account, preferred_model: eu.anthropic.claude-opus-4-6-v1 and then eu.anthropic.claude-opus-4-5-20251101-v1:0) all return 200 OK with no tier-based rejection.

- State/URL: URL stays /settings/ai-model throughout; verified via direct API GET calls

before and after each PUT that the persisted preferred_model for the basic-tier account matches the just-saved premium model, both via API round-trip and a full browser reload.

ID BUG-20260828-031200-settings-ai-model Found by bug-settings-ai-model-r1

Fingerprint /settings/ai-model|pipeline-model-select|basic-tier-account-selects-powerful-tier-model-and-saves|no-tier-gating-anywhere-ui-or-api

"Manage plan" button on Usage & Limits has no click handler at all; a basic-tier account has no way to upgrade

/settings/usage

SUMMARY

The /settings/usage page's only interactive control besides the settings tabs is the "Manage plan" button, rendered next to the plan-name heading ("Enterprise plan" for qa-admin, "Basic plan" for qa-basic). The button is fully enabled, styled with hover/brand-accent affordances (text-brand, hover:bg-surface-warm), and not disabled. Clicking it does nothing: no network request fires (confirmed via browser_network_requests before/after — identical request list), no modal/dialog opens, no navigation occurs, and the page's rendered document.body.innerHTML is byte-identical before and after the click. Inspecting the element's React fiber props directly shows no onClick handler is bound to it at all (typeof props.onClick === 'undefined') — this is not a silently-failing async handler, the button has never been wired to any handler.

Reproduces identically for both an enterprise-tier admin account (where "nothing to upgrade to" might arguably explain a no-op, though the button should still open a plan-management view) and, more consequentially, for a **basic-tier account** — whose plan explicitly restricts it to 2 of 8 deliverable types (per TIER_PIPELINES["basic"] in backend/app/core/entitlements.py, which also defines a real UPGRADE_PATH: basic -> pro) — meaning a basic-tier user who wants to act on the page's own stated purpose ("Your plan determines which deliverables you can run... Manage plan") has literally no UI path to do so anywhere in the app. This is a distinct control and distinct claim from the already-filed BUG-20260828-031200-settings-ai-model (model tier-gating on a different page/component); here the defect is that the plan page's own primary CTA is inert, not that a restriction is unenforced.

REPRODUCTION

1. Sign in as qa-admin (enterprise tier), navigate to /settings/usage. Observe "Enterprise

plan" heading and an enabled "Manage plan" button.

1. Click "Manage plan". Observe: no dialog, no navigation (`location.href` unchanged), and `browser_network_requests` shows no new request fired by the click.

1. Sign out, sign in as `qa-basic@flowinqa.com / flowin-e2e-pass` (basic tier), navigate to `/settings/usage`. Observe "Basic plan" heading, "Deliverable access" listing only 2 items (Product Requirements, Presentation) — the other 6 pipeline types are simply absent (no lock badge, no explanation), and the same enabled "Manage plan" button.

1. Click "Manage plan" as the basic-tier account. Same result: no request, no modal, no navigation — confirmed via direct DOM read that `document.body.innerText` is unchanged and via `browser_network_requests` that the request list before and after the click is identical.

1. Reload to a fresh `/settings/usage` load and repeat the click a second time (basic tier) — identical no-op result, confirming determinism, not a one-off race.

1. Inspected the button's bound React props directly

`(Object.keys(btn).find(k=>k.startsWith('__reactProps')) → the click handler): onClick` is undefined — the button has no handler wired to it whatsoever, it is a static, inert control styled to look actionable.

EXPECTED

Clicking "Manage plan" should open some plan-management/upgrade affordance (a modal, a dedicated page, or at minimum a "contact sales" / "request upgrade" flow) — the button's own label and the page's own copy ("Your plan determines which deliverables you can run... Manage plan") promise this. At minimum, a basic-tier account should have some in-app path toward the `UPGRADE_PATH` the backend already models (`basic → pro`).

ACTUAL

The button is fully rendered as an enabled, clickable, styled call-to-action but has no click handler bound to it at all. Clicking it is a complete no-op for every account tier tested, leaving a basic-tier user with no in-app mechanism to act on the page's stated purpose.

EVIDENCE

- Before (enterprise admin, "Manage plan" enabled): bug-hunter/evidence/settings-usage/BUG-20260828-031600-settings-usage/01-before-enterprise-admin.png
- Before (basic tier, only 2 deliverable-access items, "Manage plan" enabled): bug-hunter/evidence/settings-usage/BUG-20260828-031600-settings-usage/02-before-basic-tier.png
- After click (basic tier, identical state, no dialog/navigation): bug-hunter/evidence/settings-usage/BUG-20260828-031600-settings-usage/03-after-click-no-change.png

BROWSER SIGNALS

- Console: no relevant error observed.
- Network: browser_network_requests request list is identical immediately before and after the

click, for both accounts — no request is fired by this control.

- State/URL: `location.href` stays `/settings/usage` throughout; `document.body.innerText` unchanged before/after click; direct React-fiber prop inspection confirms `onClick` is `undefined` on the button element.

ID `BUG-20260828-031600-settings-usage` Found by `bug-settings-usage-r1`

Fingerprint `/settings/usage|manage-plan-button|click|no-onclick-handler-bound-no-request-no-modal-no-navigation`

Constitution's "X / 4000 chars" limit is display-only; content well over 4000 chars saves and persists with no truncation or rejection

`/settings/constitution`

SUMMARY

The Constitution editor shows a character counter styled as a hard cap ("X / 4000 chars"), and the page copy states this constitution "is prepended to every agent on every run." Despite that, the underlying `<textarea>` has no `maxLength` attribute, "Save constitution" never disables when the counter exceeds 4000, and `PUT /api/settings/constitution` accepts and persists the full, un-truncated content server-side with no length validation at all. This was reproduced twice with different content and lengths: 4515 chars (`"A".repeat(4500) + "MARKER-END-4500"`) and 6015 chars (`"B".repeat(6000) + "MARKER-END-6000"`) — both were accepted by the backend, both returned `200 OK`, and both persisted verbatim (confirmed via a direct `GET /api/settings/constitution` read of the saved content field, and via a full page reload showing the identical over-limit text still loaded into the textarea with the counter reading e.g. "4515 / 4000 chars"). The "4000 chars" figure is pure decoration — it communicates a limit that does not exist anywhere in the save path, client or server.

REPRODUCTION

1. Sign in as qa-admin, navigate to `/settings/constitution`. Note the existing content and the counter (starts at a normal in-range value, e.g. "44 / 4000 chars").
1. Programmatically set the textarea's native value to a 4515-character string (via the native `HTMLTextAreaElement` value setter + an `input` event, to mimic real typing) and observe the counter update to "4515 / 4000 chars" — over the stated limit.
1. Confirm `textarea.maxLength === -1` (no HTML enforcement) and `Save constitution` button's `disabled` is `false` (no client-side block).
1. Click "Save constitution". Observe `PUT http://localhost:8000/api/settings/constitution`

returns 200 OK and the UI shows "Constitution saved — active on all future runs."

1. Call GET /api/settings/constitution directly with the auth token — the returned content is the full 4515-character string, unmodified, not truncated to 4000.

1. Reload the page from scratch — the textarea loads the same 4515-character content and the counter still reads "4515 / 4000 chars", confirming the over-limit save is durable, not a client-side artifact.

1. Repeated with a second, independent value (6015 characters, different filler character and marker) — identical result: 200 OK, full 6015 characters persisted and readable back from the API.

1. Restored the original constitution content ("E2E S-09-12: answer in exactly one sentence.")

and saved, confirmed via GET /api/settings/constitution that the original text is back exactly.

EXPECTED

Either the client should prevent saving past 4000 characters (disable Save, or truncate/reject locally) and the server should enforce the same limit (422/400 on an over-limit payload), so the displayed counter reflects a real constraint — or, if there genuinely is no limit, the counter should not be presented as "X / 4000 chars" in a way that implies a hard cap.

ACTUAL

The counter is purely cosmetic: content far beyond 4000 characters (tested up to 6015) is accepted by both the client (Save stays enabled) and the server (200 OK, persisted verbatim, survives reload), with no truncation, warning, or rejection at any layer.

EVIDENCE

- Before (original content, 44/4000 chars): bug-hunter/evidence/settings-constitution/BUG-20260828-031900-settings-constitution/01-before.png
- Failure (4515 chars entered, counter over limit, Save still enabled): bug-hunter/evidence/settings-constitution/BUG-20260828-031900-settings-constitution/02-over-limit-save-not-disabled.png

- After reload, still 4515 chars persisted: `bug-hunter/evidence/settings-constitution/BUG-20260828-031900-settings-constitution/03-after-reload-still-4515.png`
- Second independent repro, 6015 chars saved: `bug-hunter/evidence/settings-constitution/BUG-20260828-031900-settings-constitution/04-repro2-6015-chars-saved.png`

BROWSER SIGNALS

- Console: no relevant error observed.
- Network: `PUT /api/settings/constitution` returns `200 OK` for both over-limit payloads (4515

and 6015 chars); `GET /api/settings/constitution` echoes the full un-truncated content back in both cases.

- State/URL: URL stays `/settings/constitution` throughout; `textarea.maxLength` confirmed `-1` (unset) via direct DOM read; original content restored and verified via API before finishing.

ID `BUG-20260828-031900-settings-constitution` Found by `bug-settings-constitution-r1`

Fingerprint `/settings/constitution|global-instructions-textarea|type-past-4000-char-counter-and-save|no-limit-enforced-content-persists-unbounded`

Security tab tells break-glass admin their credentials are "managed outside the application" when this app manages them itself

`/settings/security`

SUMMARY

For the `qa-admin` account (a local/break-glass account, not Cognito-backed), the Security tab shows "Not available for this account" with the explanation "This account's credentials are managed outside the application, so two-factor authentication is configured separately." That claim is false for this account: its password IS stored and managed by this application (bcrypt hash in the app's own database), and can be changed from this very same settings shell (Profile tab's "Change Password" form, backed by `POST /api/auth/change-password`'s `auth_provider == "local"` branch). The copy is written for genuinely Cognito-backed accounts (where credentials really are held by an external IdP) and is reused verbatim for break-glass accounts, where the real reason 2FA is unavailable is simply "this account has no Cognito MFA factors to manage" — a materially different, and more concerning, situation: this account has no MFA at all, is not offloading that responsibility elsewhere, and the UI actively tells an admin it is. The frontend source's own code comment states the real reason correctly (SecuritySection.tsx: "A break-glass/local account has no Cognito factors to manage. Say so plainly instead of rendering controls that would 501.") — but the copy actually shown does not say that.

REPRODUCTION

1. Sign in as `qa-admin@flowinqa.com` (a local/break-glass admin account) and go to `/settings/security`. Observe: "Not available for this account" / "This account's credentials are managed outside the application, so two-factor authentication is configured separately."

1. `GET /api/auth/mfa` for this account returns `{"supported": false, ...}` — the backend's own docstring for that route states this response is specifically for "a non-Cognito

(break-glass) account," not for an externally-managed one.

1. Go to `/settings/profile` on the same account. Observe a full "Password" section with Current password / New password / Confirm new password fields and a "Change Password" button — proving this application itself is the credential store for this account.

1. Cross-reference `backend/app/api/auth.py` POST `/change-password:` for `user.auth_provider == "local"`, it verifies `request.current_password` against `user.password_hash` and writes the new bcrypt hash back into this app's own User row — confirming credentials are managed entirely inside this application, not "outside" it.

EXPECTED

The Security tab's unavailable-MFA message should be accurate for a break-glass/local account: something to the effect of "This is a local admin account with no two-factor authentication configured" — not a claim that credentials are managed by an external system, since they demonstrably are not.

ACTUAL

The UI shows the Cognito-account explanation ("credentials are managed outside the application") for a local/break-glass account whose credentials are, in fact, managed entirely inside this application — a materially false security claim on the account type this app's own break-glass invariant depends on.

EVIDENCE

- Profile tab, confirming this app owns password storage/change for this account:

`bug-hunter/evidence/settings-security/BUG-20260828-032424-settings-security/01-before-profile-change-password-form.png`

- Security tab, showing the false "managed outside the application" claim:

`bug-hunter/evidence/settings-security/BUG-20260828-032424-settings-security/02-failure-security-tab-false-claim.png`

- API responses + source correlation: `bug-hunter/evidence/settings-security/BUG-20260828-032424-settings-security/network.log`,

bug-hunter/evidence/settings-security/BUG-20260828-032424-settings-security/notes.md

BROWSER SIGNALS

- Console: no relevant error observed.
- Network: GET /api/auth/mfa → 200, {"supported": false, ...} for the local break-glass

account (same response shape used to trigger the misleading copy).

- State/URL: no state change is possible here (page is read-only); observed on

/settings/security with a cold deep-link and a tab-click navigation, same result both ways.

ID BUG-20260828-032424-settings-security Found by bug-settings-security-r1

Fingerprint /settings/security|mfa-unavailable-message|view-as-local-break-glass-account|copy-falsely-claims-external-credential-management

SUMMARY

The "Create New User" dialog on /admin performs no email-format validation, neither client-side nor server-side. The "Create User" button only checks that the Email and Password fields are non-empty (it stays disabled while either is blank, and becomes enabled the moment both fields have any text). Typing a string with no @ and no domain (e.g. `not-an-email`) into the Email field is enough to enable submission, and the backend's POST to create the user accepts it without complaint: GET `/api/admin/users` afterward shows a real, persisted user row with `"email": "not-an-email"`. The row renders normally in the table (avatar initial "N", plan "Basic", role "User", a real UUID and creation timestamp) as if it were a legitimate account. This account can never be used to log in through `/login`, since nothing resembling an email was stored, so it becomes permanently orphaned unless an admin notices and deletes it by hand.

REPRODUCTION

1. Sign in as qa-admin, navigate to /admin.
 2. Click "Add user" to open the "Create New User" dialog.
 3. Enter `not-an-email` (no @, no domain) in the Email field and any 8+ character string (e.g. `abc123456`) in the Password field. Leave Plan as "Basic" and "Grant admin access" unchecked.
1. Observe the "Create User" button is enabled (not blocked by the malformed email).
 2. Click "Create User".
 3. Observe the toast "User not-an-email created" and a new row in the table with that literal string as the email. Confirmed via GET `/api/admin/users` that the record is persisted server-side with `"email": "not-an-email"`.
1. Cleanup performed: deleted the throwaway user via its row's Delete action and confirmed via GET `/api/admin/users` that only the 4 original seeded users remain.

EXPECTED

The Email field should be validated as a well-formed email address before "Create User" is enabled (client-side), and the backend's user-creation endpoint should independently reject a value with no @/domain, returning a validation error rather than persisting the record.

ACTUAL

Both layers accept the malformed string. A new user is created and persisted with a plainly invalid, unusable "email" value, with no error anywhere in the flow.

EVIDENCE

- Before: bug-hunter/evidence/admin/BUG-20260828-033400-admin/01-before-invalid-email-entered.png
- Failure: bug-hunter/evidence/admin/BUG-20260828-033400-admin/02-failure-user-created-with-invalid-email.png

BROWSER SIGNALS

- Console: none observed
- Network: POST to admin user-creation endpoint returned success (200-series); GET

/api/admin/users confirmed persisted row {"email":"not-an-email", "tier":"basic", "is_admin":false, ...} before cleanup.

- State/URL: stayed on /admin throughout; no client-side validation error surfaced.

ID `BUG-20260828-033400-admin` Found by `bug-admin-r1`

Fingerprint `/admin|create-user-dialog|submit-malformed-email|user-persisted-with-invalid-email`

`/workflows/<bad-id>/canvas` silently opens an empty "copy" composer instead of 404ing, letting a user save a brand-new persisted workflow from a dead link
 /workflows/<nonexistent-or-malformed-id>/canvas (e.g. /workflows/00000000-0000-0000-0000-000000000000/canvas, /workflows/totally-bogus-id-12345/canvas)

SUMMARY

Unlike the bare workflow detail route and /edit, both of which correctly render the generic 404 for a nonexistent or malformed workflow id (verified in this same investigation — both are hardened), the /canvas sub-route does not. Loading /workflows/<bad-id>/canvas fires GET /api/workflows/<bad-id> which genuinely 404s (confirmed in the console), but the frontend silently swallows that failure and renders a fully interactive, empty workflow composer instead of the 404 page — header reads <bad-id> · copy, "0 agents", a blank Brief field, and a "Save as copy" button. This is not inert: typing a name into the Workflow name field, adding a real agent via "+ Add", and clicking "Save as copy" fires POST /api/user-workflows, which succeeds (201 Created) and returns a brand-new, real, persisted workflow with its own fresh UUID. The new workflow then appears in "My Workflows" as a normal saved entry — a completely legitimate user-owned resource was created by starting from a URL that should not resolve to anything at all. This is a different mechanism from BUG-20260828-010600-workflows-id-run (the already-filed class-4 defect where /run on a bad id silently falls back to rendering the *Dashboard*, with no composer and no way to create anything) and from BUG-20260828-005700-workflows-ppt-canvas (canvas on the real, existing ppt built-in losing its manifest on copy — that bug requires a valid workflow to begin with; this one requires the workflow to *not* exist).

REPRODUCTION

1. Sign in as qa-admin, navigate to `http://localhost:3000/workflows/00000000-0000-0000-0000-000000000000/canvas`

(the ledger's own resolved nonexistent uuid).

1. Observe: no 404 page. Instead a full composer renders — header

"00000000-0000-0000-0000-000000000000 · copy", "0 agents", an empty "Workflow name" field, a

"Save as copy" button, and a working canvas/Brief panel. Console shows

GET /api/workflows/00000000-0000-0000-0000-000000000000 => 404, proving the frontend received

the correct 404 and chose to render the copy-composer shell anyway.

1. Fill the "Workflow name" field with a real value (e.g. "DEAD LINK COPY TEST").
2. Click "Add agent", add any real agent (e.g. "Domain Discovery Agent"), close the picker.
3. Click "Save as copy". Observe POST /api/user-workflows returns 201 Created with a real response body: a fresh UUID, the entered name, agent_ids: ["domain-analyst"].

1. Navigate to /workflows (My Workflows). Observe the new workflow, "DEAD LINK COPY TEST", listed as a normal saved entry with "1 agent" — a genuine, persisted, user-owned resource.

1. Reproduced independently on a second, differently-shaped bad id:

http://localhost:3000/workflows/totally-bogus-id-12345/canvas (not a uuid at all) — same result: the empty "copy" composer renders instead of a 404, with a working "Save as copy" button.

1. Cleanup: deleted the created throwaway workflow (DELETE /api/user-workflows/<new-id> → 204) so no stray data was left behind.

EXPECTED

/workflows/<bad-id>/canvas, like the bare detail route and /edit for the same bad id, should render the app's 404 page when the source workflow does not exist — the frontend already receives a 404 from GET /api/workflows/<bad-id> and simply needs to act on it instead of falling through to an empty composer shell.

ACTUAL

The frontend ignores the 404 from the workflow fetch and renders a fully functional "copy" composer for a workflow that does not exist, and that composer's "Save as copy" action successfully creates and persists a brand-new real workflow via a legitimate backend call.

EVIDENCE

- Before (empty copy composer renders for the all-zeros uuid, no 404): bug-hunter/evidence/workflows-nonexistent/BUG-20260828-040400-workflows-

nonexistent/01-before-canvas-empty-composer.png

- Failure (filled + agent added, "Save as copy" fired 201 Created): bug-hunter/evidence/workflows-nonexistent/BUG-20260828-040400-workflows-nonexistent/02-failure-saved-as-copy-201-created.png
- My Workflows shows the new entry persisted: bug-hunter/evidence/workflows-nonexistent/BUG-20260828-040400-workflows-nonexistent/03-my-workflows-shows-persisted-entry.png
- Reproduced on a second, non-uuid bad id: bug-hunter/evidence/workflows-nonexistent/BUG-20260828-040400-workflows-nonexistent/04-repro2-malformed-id-canvas.png

BROWSER SIGNALS

- Console: GET `http://localhost:8000/api/workflows/00000000-0000-0000-0000-000000000000` => 404

fires and is logged as an error, but no code path reacts to it by redirecting to the 404 page.

- Network: POST `http://localhost:8000/api/user-workflows` returns 201 Created with a real new

workflow id and the entered name/agents — the backend has no reason to reject this since it is, from its perspective, a completely ordinary "create a new workflow" call with no reference back to the original bad id.

- State/URL: `location.href` stays on the bogus `/workflows/<bad-id>/canvas` throughout, both before and after the successful save (no navigation to the newly created workflow's own URL) — matching the app's documented pattern elsewhere of not navigating after a composer save, but here compounding the defect since the user has no visible confirmation their dead link just created a real object, only discoverable via My Workflows or the network log.

ID `BUG-20260828-040400-workflows-nonexistent` Found by `bug-workflows-nonexistent-r1`

Fingerprint `/workflows/<bad-id>/canvas|copy-composer|navigate-to-canvas-subroute-for-nonexistent-workflow|empty-composer-renders-and-save-as-copy-persists-a-real-workflow`

`?error=quota` always shows the "too large" dead-end even when a fully valid preview is sitting in sessionStorage`

`/preview-fullscreen?error=quota`

SUMMARY

`frontend/src/app/preview-fullscreen/page.tsx` checks `params.get("error") === "quota"` and, if

true, immediately renders the dead-end "Project too large for full screen" message and returns — before ever looking at `sessionStorage["__app_preview__"]`. The plain (no-param) route by contrast reads that same key and renders the full working IDE preview when it holds valid data.

Because `AppBuilderPreview.handleFullscreen()` opens the fullscreen tab via `window.open(...,`

`"_blank")` **without** `"noopener"` specifically so the new tab shares the opener's browsing context (confirmed by the code's own comment), `sessionStorage` is shared between the two tabs. `sessionStorage.setItem` is atomic — a `QuotaExceededError` throw leaves any previously-stored value untouched. So the realistic sequence is: a user successfully opens Full Screen once (`__app_preview__` now holds valid, renderable file data), then triggers a second Full Screen attempt whose write throws quota-exceeded (larger project) — the catch block opens a new tab at `?error=quota`. That new tab shares the exact same `sessionStorage` that still contains the **earlier, fully valid and renderable** preview payload, yet the page shows only the unhelpful, inapplicable "too large" message and never offers or falls back to the working cached preview it has direct access to. This is demonstrated deterministically below by seeding `__app_preview__` with a valid payload and comparing the plain route (renders it correctly) against the same route with `?error=quota` appended (discards it, shows the dead end, while the payload is still verifiably present and parseable in `sessionStorage` the whole time).

REPRODUCTION

1. Sign in as qa-admin. In `sessionStorage`, set a valid payload:

```
sessionStorage.setItem('__app_preview__', JSON.stringify({files:
[{'path': 'index.html',
content: '<h1>hello</h1>' }], projectName: 'ValidCachedProject'})).
```

1. Navigate to `http://localhost:3000/preview-fullscreen` (no query params). Observe the page correctly renders the full IDE preview: title becomes "ValidCachedProject – IDE Preview", file explorer shows `index.html`, content is visible. This confirms the cached payload is valid and renderable.

1. Navigate to `http://localhost:3000/preview-fullscreen?error=quota` (same tab/session, payload untouched). Observe the page instead shows "Project too large for full screen" / "The generated project exceeds the browser session storage limit (~5MB)..." — the dead-end error state, not the preview.

1. Confirm in the same page load that the valid data was never touched or consulted:

```
!!sessionStorage.getItem('__app_preview__') is true and
```

```
JSON.parse(sessionStorage.getItem('__app_preview__')).files.length is 1 — the exact data
```

from step 1, fully intact and parseable, while the UI shows only the generic error.

1. Repeated with a second, independent payload (`{files: [{path: 'app.js', ...}, {path: 'style.css', ...}], projectName: 'SecondRepro'}`) — identical result: `?error=quota` shows the dead end while `sessionDataStillPresent` reads true for the fresh, different payload.

EXPECTED

When `?error=quota` is present, the page should still check whether `__app_preview__` holds a valid, parseable payload (the same check the plain route already performs) before giving up — either rendering that last-known-good preview, or at minimum clarifying that a previous preview is still available rather than unconditionally declaring the "too large" state on every load of this URL regardless of what is actually sitting in the shared `sessionStorage` the two tabs

intentionally share.

ACTUAL

The error `=== "quota"` branch returns unconditionally before any `sessionStorage` check, so a tab that legitimately still holds a valid, viewable preview payload (from an earlier successful Full Screen attempt in the same shared browsing context) is shown the generic, dead-end "too large" message instead — discarding a working preview the app itself proves it can render, with no way to reach it from this URL.

EVIDENCE

- Before (plain `/preview-fullscreen`, valid cached payload renders correctly): `bug-hunter/evidence/preview-fullscreen-quota/BUG-20260828-040915-preview-fullscreen-quota/01-before-valid-preview-renders.png`
- Failure (`?error=quota` with the same valid payload still in `sessionStorage`, dead end shown instead): `bug-hunter/evidence/preview-fullscreen-quota/BUG-20260828-040915-preview-fullscreen-quota/02-failure-quota-shown-despite-valid-cache.png`
- Reproduced with a second, independent payload: `bug-hunter/evidence/preview-fullscreen-quota/BUG-20260828-040915-preview-fullscreen-quota/03-repro2-different-cached-data.png`

BROWSER SIGNALS

- Console: no relevant error observed; the page renders its own hard-coded quota message

entirely client-side.

- Network: no requests involved — this is a pure client-side branch order issue in a `useEffect`.
- State/URL: `location.href` stays on `/preview-fullscreen?error=quota` throughout;

`sessionStorage.getItem('__app_preview__')` verified non-null and correctly parseable via `direct browser_evaluate` reads in both reproductions, proving the data was available and simply never consulted once the `error=quota` param was present.

ID `BUG-20260828-040915-preview-fullscreen-quota` Found by `bug-preview-fullscreen-quota-r1`

Fingerprint `/preview-fullscreen|error-quota-flag-precedence|navigate-with-error=quota-while-valid-__app_preview__-payload-present-in-sessionStorage|quota-dead-end-shown-instead-of-working-preview-that-is-actually-cached`

A genuine network/fetch failure on the handoff page is misreported as "Handoff not found", even for a real, valid, owned token `/handoff/{token}`

SUMMARY

The `/handoff/{token}` page's "not found" error surface (Handoff not found) is shown any time the `GET /api/handoff/{token}` request fails to complete — including a pure network failure (DNS/connection error, backend unreachable, request aborted) — not only on an actual HTTP 404. Reproduced against a real, currently-valid, owned handoff token minted via `POST /api/handoff/receive` (task/repo present, PAT step correctly rendered on a healthy request): when the browser's fetch to `/api/handoff/{token}` is made to fail at the network layer, the page renders the exact same "Handoff not found" heading/body used for a genuinely nonexistent token, with a secondary "Failed to fetch" line and only a "Back to dashboard" link — no retry action, no indication that the session might still exist. A user whose network blips or whose backend briefly restarts while opening a legitimate handoff link is told their handoff does not exist, which is misleading (the session is fine) and gives no way to recover other than manually reloading.

REPRODUCTION

1. Sign in as qa-admin. Mint a real handoff token via the API:

`POST /api/handoff/receive` with `X-Flowin-API-Key` → token `vEv29RqYXaZwB96qpEfkoYDBIwEMBRnqZ1sAT84_VXU`.

1. Navigate to

`http://localhost:3000/handoff/vEv29RqYXaZwB96qpEfkoYDBIwEMBRnqZ1sAT84_VXU` with a healthy network — confirm it renders the real session ("One more step before this can run" / task description / repo URL), proving the token and ownership are valid.

1. Install a Playwright route handler that aborts every `/api/handoff/` request

(`route.abort('failed')`), then reload the same real-token URL.

1. Observe the page renders Handoff not found / Failed to fetch / Back to dashboard —

identical in shape to the genuinely-nonexistent-token case — for a token that is valid and still exists server-side.

1. Reproduced the same conflated rendering on a synthetic nonexistent token

(/handoff/network-fail-test-token) under the same aborted-network condition.

1. Remove the route interception and reload — the real token renders correctly again

immediately, confirming the session/token itself was never actually invalid.

EXPECTED

A network/fetch failure while loading a handoff session should be distinguished from a confirmed 404 from the server — e.g. a "Couldn't reach VelocityAI — check your connection and retry" state with a retry action, not the same copy used for "this token doesn't exist."

ACTUAL

Any failed GET /api/handoff/{token} request (network-layer failure, not just a real 404) renders the identical "Handoff not found" error surface, even for a token that is real, owned, and unexpired.

EVIDENCE

- Before (real token, healthy network, renders correctly): bug-hunter/evidence/handoff-invalid/BUG-20260828-042900-handoff-invalid/01-before-real-token-loads-correctly.png
- Failure (same real token, network request forced to fail, shows "Handoff not found"): bug-hunter/evidence/handoff-invalid/BUG-20260828-042900-handoff-invalid/02-failure-network-error-shown-as-not-found.png

BROWSER SIGNALS

- Console: one error logged per failed lookup (Failed to fetch surfaces directly in the page body as a secondary line under the heading — the raw fetch rejection message is shown to the end user verbatim).
- Network: the GET /api/handoff/{token} request never completes (aborted at the network layer via Playwright route interception, simulating an unreachable backend/connection drop) — no HTTP status is ever received, yet the UI renders the same state as an explicit 404.

- State/URL: URL stays `/handoff/{token}` throughout; unrouting and reloading immediately recovers the correct content, confirming this is a client-side error-handling gap, not a server-side data issue.

ID `BUG-20260828-042900-handoff-invalid` Found by `bug-handoff-invalid-r2`

Fingerprint `/handoff/{token}|token-lookup-fetch|network-request-fails-or-is-unreachable|frontend-renders-generic-not-found-instead-of-a-network-error-state`

"Coming Soon" agents are fully browsable and configurable via direct URL, bypassing the catalog's own disabled/unreachable gating

`/library/agents/{id}` (e.g. `/library/agents/dotnet-inventory`, `/library/agents/mulesoft-springboot-scaffold`)

SUMMARY

30 of the Agents tab's 94 catalog entries (the .NET to Azure and Mulesoft-to-Spring-Boot migration pipelines, plus a handful of revision agents) are rendered as visibly disabled cards — `opacity-60` `cursor-not-allowed`, a "Coming Soon" badge, and no `Configure →` link — so there is no click path in the UI that reaches their detail page. However, the detail route itself enforces none of this: navigating directly to `/library/agents/{id}` for any of these "Coming Soon" agent ids opens the exact same fully-populated detail drawer as a released agent — Overview, Skills, Hooks, and an interactive **Config** tab with live Model/Validator/Gate/Retry controls and an enabled "Save agent" button — with nothing in the drawer indicating the agent is unreleased. The catalog's "Coming Soon" gating is therefore cosmetic only: it blocks the card click, not the route, so any user who knows or guesses an id (bookmark, shared link, browser history, or simply incrementing through the API's own agent list) gets full access to an agent the catalog is actively telling every other user isn't available yet.

REPRODUCTION

1. Sign in as qa-admin, go to `/library` (Agents tab). Scroll to a .NET to Azure card, e.g. ".NET Solution Inventory Agent" — confirm it renders with `opacity-60` `cursor-not-allowed`, a "Coming Soon" badge, and no "Configure →" link (DOM-confirmed: no click target reaches it).
1. Navigate directly to `http://localhost:3000/library/agents/dotnet-inventory`.
2. Observe: the full agent detail dialog opens — dialog ".NET Solution Inventory Agent details" — with Overview/Skills/Hooks/Config tabs, exactly as for any released agent.
1. Repeat for a second, unrelated "Coming Soon" pipeline: navigate to

<http://localhost:3000/library/agents/mulesoft-springboot-scaffold> ("Spring Boot Scaffold

Agent", Mulesoft-to-Spring-Boot pipeline). Same result: full dialog opens.

1. Click into the Config tab for the second agent: renders live Model / Validator / Before-execute gate / After-execute gate / Retry controls, a "Reset" button, and an enabled "Save agent" button — fully interactive, not a read-only/locked view.

EXPECTED

An agent the catalog marks "Coming Soon" and deliberately blocks from being opened by click should either be unreachable by direct URL too (redirect back to /library or show a "not yet available" state), or at minimum the detail view should visibly communicate the same unavailable/locked state the card already shows — not render a fully interactive Config panel with a live Save button.

ACTUAL

The detail route performs no availability check at all: any "Coming Soon" agent id opens the identical, fully interactive detail drawer as a released agent, with no visual indication that the agent is unreleased.

EVIDENCE

- Before (card in grid showing "Coming Soon", opacity-60 cursor-not-allowed, no Configure link): <bug-hunter/evidence/library/BUG-20260828-050115-library/01-before-coming-soon-card-in-grid.png>
- Failure (direct URL opens full detail dialog, agent 1): <bug-hunter/evidence/library/BUG-20260828-050115-library/02-failure-direct-url-opens-full-detail.png>
- Repro 2 (direct URL opens full detail dialog, unrelated agent 2 on a different pipeline): <bug-hunter/evidence/library/BUG-20260828-050115-library/03-repro2-second-agent-direct-url.png>

BROWSER SIGNALS

- Console: no errors on either navigation.
- Network: GET /api/agents/library 200 OK; the drawer renders from the same already-fetched catalog payload, confirming the gap is purely a missing route-level/UI-level availability check, not a missing-data issue.

- State/URL: URL correctly reflects /library/agents/{id} in both cases; the underlying grid

(still /library in the background) continues to show the same card as disabled the whole time.

ID BUG-20260828-050115-library Found by bug-library-r2

Fingerprint /library/agents/{id}|agent-detail-drawer|direct-url-navigation-to-coming-soon-agent-id|full-detail-and-editable-config-render-despite-catalog-marking-agent-unavailable

54 **MEDIUM** **Run detail chat header shows a token total that does not match the run's own API `token_usage`, disagreeing with the correct figure shown on the Run History list card for the same run**
`/runs/a0693fcd-9451-4c19-a9e7-61474ea03516`

SUMMARY

Filtered Run History to "Presentation" and searched, landing on a set of 5 runs that all share the exact same (truncated) title text — a good stress case for confirming the correct run is opened. Clicking the card reading "22m 1s / 10.4K / Done" correctly navigated to run `a0693fcd-9451-4c19-a9e7-61474ea03516` (verified via GET `/api/runs/{id}:duration:1320.7s ≈ 22m1s, token_usage.total_tokens: 10428 ≈ 10.4K` — both match the list card, so the navigation itself is NOT the bug). However, the run-detail page's own chat header — the line under the run title reading "just now · 22m 0s · N tokens" — displays **17.9K tokens**, not 10.4K. This is reproducible on a fresh hard reload, so it is not a stale-render artifact from the click. This is a distinct component/mechanism from the already-filed Steps-tab footer token bug (BUG-20260828-012500-runs-id-steps, which is ~20x too low and lives in the Steps tab footer on a different run `b9feac1c...`); this one is the top-of-page chat header on the Preview tab, overstates by roughly 1.7x, and is verified against the API's `token_usage.total_tokens` field directly rather than a tooltip.

REPRODUCTION

1. Sign in as qa-admin, go to `/runs`, click the "Presentation" filter chip (URL becomes `/runs?type=ppt`).
1. In the filtered list, locate the completed card reading "22m 1s" / "10.4K" / "Done" (title: "A 5-slide deck pitching a coffee subscription service to inv..." — 5 cards share this exact truncated title, distinguished only by duration/tokens/status).
1. Click that card. Land on `/runs/a0693fcd-9451-4c19-a9e7-61474ea03516`.
2. Read the chat header directly under the run title: it reads "just now · 22m 0s · 17.9K

tokens".

1. Reload the page directly (fresh `page.goto` to the same URL, not a click-through) — the header still reads "17.9K tokens".

1. Cross-check GET `/api/runs/a0693fcd-9451-4c19-a9e7-61474ea03516` with the qa-admin token:

`token_usage.total_tokens` is 10428 (10.4K) — matching the Run History list card, not the detail header.

EXPECTED

The run detail header's token total should equal the run's actual `token_usage.total_tokens` from the API (10.4K here), and should agree with the figure already shown for the same run on the Run History list.

ACTUAL

The run detail chat header shows "17.9K tokens" for a run whose API-reported total is 10,428 tokens (10.4K) — a ~72% overstatement, and a direct disagreement with the correct value the user already saw one click earlier on the Run History card.

EVIDENCE

- List card showing the correct 10.4K: `bug-hunter/evidence/runs/BUG-20260828-051900-runs/01-list-card-shows-10.4K.png`
- Detail header showing the incorrect 17.9K (fresh reload): `bug-hunter/evidence/runs/BUG-20260828-051900-runs/02-detail-header-shows-17.9K.png`
- API response excerpt (`token_usage.total_tokens = 10428`): `bug-hunter/evidence/runs/BUG-20260828-051900-runs/api-response-excerpt.json`

BROWSER SIGNALS

- Console: none observed
- Network: GET `/api/runs/a0693fcd-9451-4c19-a9e7-61474ea03516` returns 200 OK with

`token_usage.total_tokens: 10428`; no error status on any request during reproduction.

- State/URL: URL stays on `/runs/a0693fcd-9451-4c19-a9e7-61474ea03516` throughout; confirmed via

a fresh direct `page.goto` (not just a click), ruling out stale client-side state from the

filter+search navigation.

ID `BUG-20260828-051900-runs` Found by `bug-runs-r2`

Fingerprint `/runs/[id]|run-detail-chat-header-token-total|open-completed-run-from-filtered-search|header-token-figure-disagrees-with-api-and-list-card`

SUMMARY

Clicking Delete in a saved-workflow card's "Workflow actions" menu opens a confirmation dialog whose entire text is generic and static — "Delete workflow" / "This cannot be undone" / "The saved workflow will be permanently removed from your saved workflows." — with no workflow title, id, or any other identifying detail anywhere in the dialog (verified via the full accessibility tree, not just the visible screenshot). The list contains many near-identical rows (four cards titled "Signoff Stop-Resume Test" with only a trailing numeric suffix distinguishing three of them, four "Signoff Composer Test (API)..." cards, etc.), so a user who opens the menu on the wrong card among lookalikes has no way to catch the mistake before confirming an irreversible delete — the dialog itself provides zero disambiguating evidence, and by the time it is open the source card is no longer visible/highlighted either.

REPRODUCTION

1. Go to /workflows as qa-admin.
2. Locate two or more cards with very similar or identical titles (e.g. the four "Signoff Composer Test (API)..." cards, or the four "Signoff Stop-Resume Test..." cards).
3. Click "Workflow actions" on one such card, then click "Delete" in the menu.
4. Observe the confirmation dialog text in full (via accessibility snapshot or by reading every visible string).
5. Repeat on a second, differently-named lookalike card.
6. Observe: both dialogs render byte-identical generic text with no workflow name/id.

EXPECTED

The delete confirmation should state which workflow is about to be deleted (e.g. "Delete '<workflow title>'" or include the title/id in the body), so a user can verify the correct row is targeted before an irreversible action, especially given the list's many near-duplicate names.

ACTUAL

The dialog heading and body are fixed, generic strings identical regardless of which card triggered it — no title, id, or other identifying detail is present anywhere in the dialog's accessible text.

EVIDENCE

- Before: bug-hunter/evidence/workflows/BUG-20260828-052400-workflows/01-before.png
- Failure (card 1, "Signoff Composer Test (API) 1786741346946"): bug-hunter/evidence/workflows/BUG-20260828-052400-workflows/02-delete-dialog-no-name.png

- Second repro (card 2, "Signoff Stop-Resume Test"): `bug-hunter/evidence/workflows/BUG-20260828-052400-workflows/03-delete-dialog-second-card-still-no-name.png`

BROWSER SIGNALS

- Console: none observed
- Network: none observed (dialog is client-rendered before any delete request fires; both repros were cancelled, no deletion performed)
- State/URL: URL stays `/workflows` throughout; dialog `accessible-name/heading` text confirmed identical across both cards via full a11y snapshot, not just visual screenshot

ID `BUG-20260828-052400-workflows` Found by `bug-workflows-r2`

Fingerprint `/workflows|delete-workflow-confirmation-dialog|open-workflow-actions-then-delete|dialog-text-contains-no-workflow-name-or-identifier`

SUMMARY

Attaching a plain-text-type file (`.txt/.md/.json/.csv`) whose content exceeds the 450,000-character `ATTACH_MAX_CHARS` limit gets its content silently sliced to that limit before being sent in the run payload, with absolutely no indication anywhere in the UI (chip, toast, or the sent brief text itself) that truncation occurred. The sibling binary path (`.pdf/.docx/.pptx`, extracted server-side via `extractFileText`) hits the exact same limit but explicitly appends a visible `"[Content truncated to 450,000 chars]"` note into the content block that is sent to the agent. The two code paths in the same `handleFiles()` function apply identical truncation but only one of them tells anyone it happened — a user (or the downstream agent reading the brief) has no way to know a large `.txt/.md/.json/.csv` attachment lost data.

REPRODUCTION

1. Sign in as `qa-admin`, go to `http://localhost:3000/create/app` (fresh load).
 2. Type any brief text, e.g. "Summarize the attached notes into user stories."
 3. Click "+ Attach file", select a `.txt` file whose content is far larger than 450,000 characters (repro used a ~31MB huge `.txt` of repeated `a` characters).
1. Observe the attachment chip: it shows only the filename `huge.txt`, no size, no warning.
 2. Click "Run workflow". Intercepted the outgoing `POST /api/runs` request with `page.route()` and aborted it client-side before it reached the backend (so no real run was created) purely to inspect the payload the browser had already assembled.
1. Inspect the `message` field of the intercepted body: it contains the file block
`=== Attached: huge.txt ===\n` followed by content sliced to exactly 450,000 characters, ending directly at `=== End: huge.txt ===` with no truncation marker anywhere in the string.
1. Repeated the full sequence (steps 1–6) a second time from a fresh `/create/app` page load, both with `huge.txt` alone and combined with a small `valid.md` attachment — identical result

both times: `containsTrunc` (payload contains the word "truncat") is false in both runs.

EXPECTED

When a text-type attachment's content is cut to fit `ATTACH_MAX_CHARS`, the user should see the same visible signal the binary path already provides — either a truncation note appended to the sent content (matching the PDF/DOCX/PPTX behaviour), a visible warning near the file chip, or outright rejection of files over the limit with a clear error.

ACTUAL

The content is silently sliced with `content.slice(0, ATTACH_MAX_CHARS)` and stored with no truncation flag or note at all. The file chip shows only the filename. Nothing in the UI or the payload sent to the agent indicates data was dropped.

EVIDENCE

- Before (huge.txt attached, chip shows filename only, no size/warning):

bug-hunter/evidence/create-app/BUG-20260828-053745-create-app-r2/01-before-huge-attached-chip.png

- Network payload evidence (two independent intercept-and-abort repros, exact body lengths and tails showing truncation with no marker):

bug-hunter/evidence/create-app/BUG-20260828-053745-create-app-r2/network.log

BROWSER SIGNALS

- Console: none relevant (the two `Failed to fetch` errors seen during the repro are from this investigation's own `route.abort()` calls, not application-caused).

- Network: `POST http://localhost:8000/api/runs` body length 450,236–450,427 chars (~450,000

from the file content plus the brief/wrapper text) with "truncat" absent from the string in

both reproductions; no run was actually created (request aborted client-side before dispatch).

- State/URL: stayed on `/create/app` throughout; no server-side effect.

Fingerprint `/create/app|brief-attach-file-text-type|attach-text-file-over-450000-chars-and-run|content-silently-truncated-no-warning-anywhere`

The "Add agent" library's own "User Stories" category tab always shows "No agents found", even for a bare text search

`/create/user-stories`

SUMMARY

On `/create/user-stories`, opening Advanced → Agents → any "+ Add agent" placeholder opens the "Add agent" library dialog defaulted to the **"User Stories"** category tab (the page's own pipeline). That tab renders "No agents found" immediately on open, with no interaction needed to reproduce it. Every sibling category tab (PPT, Prototype, App Builder, MuleSoft to Spring Boot, .NET to Azure, Custom, and All) returns a populated list of addable agents when selected. Typing a maximally generic search term ("agent") into the search box while the User Stories tab is active still returns "No agents found" — this isn't a narrow filter miss, the category has zero catalog entries tagged for it at all, so the "+ Add agent" affordance on this specific launch panel's canvas can never actually add anything from the panel's own pipeline category.

REPRODUCTION

1. Sign in as qa-admin, navigate to `http://localhost:3000/create/user-stories` (fresh load).
2. Click "Advanced 7 agents" to open the Advanced Workflow Configuration modal, Agents tab.
3. Click any "+ Add agent" canvas placeholder to open the "Add agent" library dialog. Observe it opens with "User Stories" already the active/highlighted tab, and the body immediately reads "No agents found" — no click needed to trigger it.

1. Click the "All" tab: a full multi-category list of addable agents renders (App Builder, Prototype, PPT, MuleSoft, Custom entries all present).

1. Click ".NET to Azure": populated with its own agents (not empty), confirming other category-specific tabs work correctly.

1. Click back to "User Stories": empty again ("No agents found"), reproducing the same state a second time from a fresh tab reselection.

1. With "User Stories" still active, type the generic term agent into "Search agents": still "No agents found" — ruling out an overly narrow default filter, since a search this broad

would match almost anything in a working category.

EXPECTED

The "Add agent" library's category tab matching the current pipeline (User Stories) should list addable User-Stories-tagged agents (or, if none exist by design, the tab should not be offered / should not default-select as the opening tab, and should not silently imply that no agents exist to add rather than that the category is empty by design). At minimum, a generic search should surface something if any catalog agent anywhere is loosely relevant.

ACTUAL

The "User Stories" tab — the modal's own default/home category — is unconditionally empty, on first open and on every reselection, with or without a search term. No agent can be added to a User Stories workflow from its own category tab; a user must know to switch to "All" or another pipeline's tab to add anything at all.

EVIDENCE

- Default open state (User Stories tab pre-selected, empty): `bug-hunter/evidence/create-user-stories/BUG-20260828-054500-create-user-stories/01-dialog-opens-defaulted-to-user-stories-tab-empty.png`
- Reproduced after switching away and back: `bug-hunter/evidence/create-user-stories/BUG-20260828-054500-create-user-stories/02-repro-reselect-user-stories-tab-still-empty.png`
- Generic search term still empty: `bug-hunter/evidence/create-user-stories/BUG-20260828-054500-create-user-stories/03-search-agent-term-still-empty.png`
- Full dialog context with sidebar: `bug-hunter/evidence/create-user-stories/BUG-20260828-054500-create-user-stories/04-dialog-view-with-sidebar.png`

BROWSER SIGNALS

- Console: no error logged on any tab switch or search.
- Network: no failed request observed; the agent catalog appears to be loaded client-side/already

fetches (switching tabs re-renders instantly with no new request), so this is a client-side category-filter/tagging gap, not a fetch failure.

- State/URL: URL stays `/create/user-stories` throughout; verified via repeated tab switches and a direct accessibility-tree read (not screenshot alone) showing the "No agents found" paragraph

present under the User Stories tab specifically, absent under All/.NET to Azure.

ID `BUG-20260828-054500-create-user-stories` Found by `bug-create-user-stories-r2`

Fingerprint `/create/user-stories|advanced-modal-add-agent-dialog|select-user-stories-category-tab|no-agents-found-for-own-pipelines-category-even-on-search`

A step's seeded "before-human" gate is invisible in both the Gate combobox and the Review-gates checklist on fresh load

`/create/ex_A4_human_gate`

SUMMARY

GET `/api/workflows/ex_A4_human_gate` returns the "Pick Language" step with `"gates": ["before-human", "conditional"]` — the manifest genuinely declares this step as both a human-review pause point and a conditional router. Neither UI surface that is supposed to expose review-gate state reflects the human-gate half of that on a completely fresh page load, with zero user interaction: the "Review gates" checklist popover shows "no gates" with every one of the 5 agent checkboxes unchecked (including "Pick Language"), and the Advanced modal's per-agent Config tab for that same "Pick Language" node renders Gate as a single-select combobox pre-set to "Conditional gate" only — there is no indication anywhere in the UI that this step also carries a human-review gate. This is a distinct mechanism from the already-filed `/create/app` bug (checking an agent in the Review-gates checklist doesn't sync to the Advanced modal's Config Gate dropdown): that defect requires a user action (checking a box) to trigger the desync. Here, no interaction is needed — the seeded, already-gated fixture data is silently misrepresented the instant the page loads, and the Gate control's single-select model appears structurally unable to represent a step that carries two gates simultaneously, so the human-review portion has no path to be displayed, edited, or unset via this UI at all.

REPRODUCTION

1. Sign in as qa-admin, navigate to `http://localhost:3000/create/ex_A4_human_gate` (fresh load, no interaction beyond page load).
1. Click the "Review gates" pill. Observe the popover: pill reads "no gates", and all 5 checklist checkboxes (Ask For Language, Pick Language, Say Hello, Say Hola, Say Hallo) are unchecked — despite "Pick Language" genuinely carrying a before-human gate per the backend manifest.

1. Close the popover, click "Advanced 5 agents" to open "Advanced Workflow Configuration — Human Gate".

1. Click the "Pick Language" node, then its "Config" tab. Observe the Gate combobox reads option "Conditional gate" [selected] with no other indicator of a human gate anywhere in the panel.

1. Confirmed via API (GET /api/workflows/ex_A4_human_gate) that this exact step's gates array is ["before-human", "conditional"], not ["conditional"] alone.

1. Repeated steps 1-4 on a second fresh page load — identical result both times.

EXPECTED

A step the backend manifest declares as carrying a before-human gate should surface that fact in at least one of the two UI controls built to expose gate state — the Review-gates checklist should show that agent checked, and/or the Config Gate control should indicate the human-review gate is active (or fall back to something other than silently showing only the other gate).

ACTUAL

Both controls report the step as ungated for human review. The Review-gates checklist shows "no gates" / all unchecked, and the Config Gate dropdown shows only "Conditional gate" with no trace of the before-human gate the backend manifest actually carries for that step — a real, seeded, dual-gate configuration is silently reduced to a single gate in the UI on first load, before any user interaction.

EVIDENCE

- Review gates checklist, fresh load, "no gates"/unchecked: bug-hunter/evidence/create-ex-a2-branch/BUG-20260828-055300-create-ex-a2-branch/01-review-gates-nogates.png
- Advanced modal, Pick Language Config tab, Gate shows only "Conditional gate": bug-hunter/evidence/create-ex-a2-branch/BUG-20260828-055300-create-ex-a2-branch/02-gate-dropdown-conditional-only.png

BROWSER SIGNALS

- Console: no relevant error observed.
- Network: GET /api/workflows/ex_A4_human_gate (200) confirmed via direct API call that the

"Pick Language" step's gates field is ["before-human", "conditional"]; the UI never issues a request for gate state beyond this initial load (both controls are purely derived client-side from the same fetched manifest).

- State/URL: URL stays /create/ex_A4_human_gate throughout; reproduced twice from independent fresh page loads with zero prior interaction.

ID `BUG-20260828-055300-create-ex-a2-branch` Found by `bug-create-ex-a2-branch-r2`

Fingerprint `/create/ex_A4_human_gate|advanced-modal-gate-config-and-review-gates-checklist|load-fixture-whose-manifest-step-carries-two-simultaneous-gates|human-review-gate-invisible-in-both-ui-surfaces`

"Save workflow" silently no-ops when the name field is empty, with zero visible feedback and no API request fired

`/workflows/new`

SUMMARY

On `/workflows/new`, with at least one agent added to the canvas but the "Workflow name" field left empty (showing only the "Untitled workflow" placeholder), clicking "Save workflow" does nothing observable: no `POST` request is issued at all, no toast or error banner appears, and the name field gets no visual error state (no red border, no inline message) — the only side-effect is the name input silently regaining DOM focus, which is indistinguishable from any unrelated focus change. The exact same click, with a name typed into that field first, fires `POST /api/user-workflows` immediately and returns `201 Created`. So the button is clearly validating client-side before submitting, but communicates that validation failure to the user in no way whatsoever — a user with an empty name has no way to know the click did anything short of noticing nothing changed.

REPRODUCTION

1. Sign in as qa-admin, navigate to `/workflows/new` (empty composer, "Untitled workflow" placeholder, 0 agents).
1. Click "Add agent" → "+ Add" on any agent (e.g. Domain Discovery Agent) → Escape to close the modal. Canvas now shows 1 agent; the header confirms "1 agents". Name field remains empty.
1. Click "Save workflow". Observe: no new network request fires (confirmed via `browser_network_requests`, filtered to `workflow`), no toast/banner appears anywhere in the DOM, no red border or inline error on the name field — the page looks completely unchanged.
1. Type a name (e.g. `zz-hunt-savetest`) into the same "Workflow name" field, keep the same agent on the canvas, click "Save workflow" again: `POST /api/user-workflows` fires immediately and returns `201 Created` — proving the click handler and the agent state were both fine; only the empty-name case was gated with no feedback.

1. Clear the name field back to empty, click "Save workflow" a third time (repro #2): again, no new request fires, no feedback of any kind.

EXPECTED

Attempting to save a workflow with an empty/missing name should give the user a clear, observable signal — an inline validation message near the field, a red border, a toast, or at minimum a disabled "Save workflow" button when the name is empty — so the user understands why nothing was saved.

ACTUAL

The save action is silently blocked client-side with absolutely no user-visible indication: button stays enabled and clickable, no request is sent, no error text or styling appears anywhere in the DOM, and the only trace is the name input regaining focus (which looks identical to a stray click).

EVIDENCE

- Before (1 agent added, name field empty, about to click Save): `bug-hunter/evidence/workflows-new/BUG-20260828-060700-workflows-new-r2/01-before-1agent-no-name.png`
- After clicking Save workflow — page unchanged, no toast/error, no request fired: `bug-hunter/evidence/workflows-new/BUG-20260828-060700-workflows-new-r2/02-after-click-save-nothing-happens.png`
- Network log contrasting the empty-name no-op against the named-save 201: `bug-hunter/evidence/workflows-new/BUG-20260828-060700-workflows-new-r2/network.log`

BROWSER SIGNALS

- Console: no errors or warnings logged on either the no-op click or the successful save.
- Network: empty-name click → no new request; named click (same session, same agent) →

POST `http://localhost:8000/api/user-workflows` → 201 Created.

- State/URL: stays on `/workflows/new` throughout; only DOM change on the no-op click is the name `<input>` regaining `[active]/focus` state, with no error text anywhere in `document.body.innerHTML`.

Fingerprint /workflows/new|save-workflow-button|click-save-with-empty-name-field|no-request-fired-no-feedback-shown

Preview toolbar's "Full Screen" button renders entirely outside a narrow (mobile) viewport with no scroll affordance to reach it

`/runs/{id}/preview/full`

SUMMARY

On `/runs/{id}/preview/full` for a completed run, the Preview tab's toolbar row (Renders-as toggle area's sibling row holding a secondary "Download" button and the "Full Screen" button) does not wrap or scroll at narrow viewport widths. Its immediate wrapper div carries `flex-shrink-0`, so at a 375px-wide viewport (verified via `window.innerWidth`, not screenshot pixels) the row's total content width grows to ~493px while the row container is capped at 375px. Nothing in the ancestor chain sets `overflow-x: auto/scroll`, and the document itself reports no horizontal scroll (`document.documentElement.scrollWidth === clientWidth === 375`).

The result: the "Full Screen" button's entire bounding box (`left: 400`, `right: 493`) sits outside the visible/interactive viewport permanently — `document.elementFromPoint` at the viewport's right edge resolves to a different, unrelated button, confirming there is no way for a real pointer/touch user to see or reach "Full Screen" at this width. Part of the secondary toolbar "Download" button is also clipped (right edge at 386.6 vs 375px viewport), though it remains mostly clickable; "Full Screen" is completely unreachable.

REPRODUCTION

1. Sign in as qa-admin, navigate to `http://localhost:3000/runs/b9feac1c-ec21-4531-8ba7-bb391786993e/preview/full` (completed run, Preview tab auto-selected).

1. Resize the browser viewport to 375×700 (verify with `window.innerWidth` — confirmed 375, not just a screenshot artifact).

1. Inspect the toolbar row containing "Download" and "Full Screen" (siblings of the Preview tablist). Run `document.querySelectorAll('button')` to find the "Full Screen" button and read

`getBoundingClientRect()`.

1. Observe: `left: 400.02`, `right: 493.38`, while `window.innerWidth: 375` — the button's box is entirely to the right of the viewport. `document.elementFromPoint` at the clamped right edge of the viewport returns a different button (the composer's send button), not "Full Screen".

No horizontal scrollbar exists anywhere in the ancestor chain to scroll it into view.

1. Reproduced on a fresh full page reload/navigation to the same URL at the same viewport size — identical rect, identical unreachability.

EXPECTED

At narrow/mobile viewport widths, all toolbar controls (including "Full Screen") should either wrap onto a new line, shrink to fit, or live behind a horizontally scrollable/overflow-safe container, so every control remains visible and clickable.

ACTUAL

The toolbar row's "Full Screen" button is pushed completely outside the viewport's horizontal bounds with `flex-shrink-0` preventing any shrink and no scroll mechanism provided — the control is permanently invisible and unclickable for any user on a ~375px-wide viewport (a standard mobile width).

EVIDENCE

- Before (default/desktop width, "Full Screen" fully visible and clickable): `bug-hunter/evidence/runs-id-preview-full/BUG-20260828-084621-runs-id-preview-full/01-before-desktop-fullscreen-visible.png`
- Failure (375px viewport, "Full Screen" text/button clipped off the right edge): `bug-hunter/evidence/runs-id-preview-full/BUG-20260828-084621-runs-id-preview-full/02-failure-mobile-fullscreen-offscreen.png`
- Reproduced on a second, fresh navigation at the same viewport: `bug-hunter/evidence/runs-id-preview-full/BUG-20260828-084621-runs-id-preview-full/03-repro2-fresh-nav-still-offscreen.png`

BROWSER SIGNALS

- Console: none observed (0 errors related to this)
- Network: none relevant
- State/URL: `location.href` stays on `/runs/{id}/preview/full`; measured via

`getBoundingClientRect()` (left: 400.02, right: 493.38) against `window.innerWidth`: 375
(`viewportUnreliable` quirk avoided by reading `window.innerWidth` directly, not screenshot
pixel dimensions)

ID `BUG-20260828-084621-runs-id-preview-full` Found by `bug-runs-id-preview-full-r2`

Fingerprint `/runs/{id}/preview/full|preview-toolbar-fullscreen-button|resize-viewport-to-375px-
width|button-bounding-box-left-edge-exceeds-window-innerWidth-with-no-scroll-mechanism`

61 **MEDIUM** **Completed run's "Download the deliverable" toolbar button stays permanently disabled when the backend never populates `deliverable\_filename`, even though the deliverable is fully rendered**
`/runs/{id}/stream`

SUMMARY

Launched a genuinely live run (the "Playwright Smoke Test" workflow, ~3s/1-agent pipeline — first live pipeline execution this hunt has been able to exercise now that Bedrock credentials work) and watched it run to completion in `/runs/{id}/stream`. The run reaches status: "completed", the Preview tab renders the deliverable at "100%" with a working file explorer (`index.html`, viewable, "Download ZIP" button in the file panel works), yet the toolbar's top-level "Download the deliverable" button (`aria-label="Download the deliverable"`) is `[disabled]` and stays disabled indefinitely — confirmed still disabled after 10s+ of polling and after a full page reload. `GET /api/runs/{id}` shows why: `deliverable_mimetype: "application/zip"` is set, but `deliverable_filename: null` — the backend marks the run complete and typed the deliverable's mimetype, but never wrote the filename the frontend's top toolbar button apparently gates on. For contrast, the seeded completed run (`b9feac1c-ec21-4531-8ba7-bb391786993e`) has both `deliverable_mimetype: "text/html"` AND `deliverable_filename: "presentation.html"` populated, and its toolbar Download button is enabled (`cursor=pointer`, not disabled). This is a different mechanism from the already-filed `BUG-20260828-014937-runs-id-stream` (a diverted run with NO deliverable at all — output and `deliverable_filename` both null, no rendered content, wrong Preview placeholder copy): here the deliverable genuinely exists, is fully rendered, and is downloadable through a different UI path (the file panel's "Download ZIP"), yet the primary/expected Download affordance is silently broken for this run.

REPRODUCTION

1. Sign in as qa-admin. From `/dashboard`, launch "Playwright Smoke Test" (~1 agent · ~3s) with

brief "Smoke test run for bug hunt round 2". Run id:

649e56cf-ce0f-4a0f-91fa-4e75a971a980.

1. Watch `/runs/649e56cf-ce0f-4a0f-91fa-4e75a971a980/stream` progress from "Running · 0/1" to

"Done" (~21s wall clock, single agent). Preview tab auto-shows the deliverable at "100%",

file explorer lists `index.html` (2 lines) with working Copy/Download-ZIP controls.

1. Observe the toolbar: "Version v1" / "Share" / "Download the deliverable" — the Download button carries `[disabled]` in the accessibility tree despite the run being fully complete with a rendered, viewable deliverable.

1. Waited 5-10s and re-checked — still disabled (rules out a brief post-completion race).
2. Reloaded the page fresh (`page.goto` to the same URL) — Download button still `[disabled]`.
3. Confirmed via GET `/api/runs/649e56cf-ce0f-4a0f-91fa-4e75a971a980`: `status: "completed"`,

`deliverable_mimetype: "application/zip"`, `deliverable_filename: null`.

1. For contrast, loaded `/runs/b9feac1c-ec21-4531-8ba7-bb391786993e/stream` (seeded completed

run): toolbar Download button is enabled (`cursor=pointer`, no `disabled` attribute). Its API response has `deliverable_filename: "presentation.html"` populated.

EXPECTED

A completed run with a fully rendered, viewable deliverable should offer a working top-level Download button, matching the deliverable that is demonstrably downloadable via the file panel's "Download ZIP".

ACTUAL

The toolbar's primary "Download the deliverable" button is permanently disabled whenever the backend completes a run without populating `deliverable_filename`, even though `deliverable_mimetype` is set and the deliverable content is fully present and independently downloadable through the file explorer's "Download ZIP" control — an inconsistent, silently broken primary affordance with no error shown to the user.

EVIDENCE

- Live run in progress: bug-hunter/evidence/runs-id-stream/BUG-20260828-085047-runs-id-stream/01-live-run-in-progress.png
- Completed, deliverable rendered, Download disabled: bug-hunter/evidence/runs-id-stream/BUG-20260828-085047-runs-id-stream/02-live-run-completed-download-disabled.png
- Still disabled after reload: bug-hunter/evidence/runs-id-stream/BUG-20260828-085047-runs-id-stream/03-reload-still-disabled.png
- Seeded completed run for contrast, Download enabled: bug-hunter/evidence/runs-id-stream/BUG-20260828-085047-runs-id-stream/04-comparison-seeded-run-download-enabled.png

BROWSER SIGNALS

- Console: none observed.
- Network: GET /api/runs/649e56cf-ce0f-4a0f-91fa-4e75a971a980 → 200, deliverable_filename:

null, deliverable_mimetype: "application/zip", status: "completed".

- State: reproduced twice (initial completion, and again after a full page reload).

ID BUG-20260828-085047-runs-id-stream Found by bug-runs-id-stream-r2

Fingerprint /runs/{id}/stream|download-deliverable-toolbar-button|complete-a-live-run-whose-deliverable_filename-is-null|download-button-permanently-disabled-despite-fully-rendered-deliverable

SUMMARY

Clicking "Reopen & fix from the failed step" on the seeded failed run d6e425b6... genuinely resumes the run (confirmed via GET /api/runs/{id} — status flips to generating and agent_outputs regenerate; this is a real live run, not a no-op). The page correctly reflects this in the header ("Running") and in the chat/pipeline widget (agents move through Thinking → Writing states, elapsed time and token counts climb). However the Steps tab's status summary panel keeps displaying the stale terminal label **"Run failed"** the entire time the run is actively progressing (1/4 → agents streaming, tokens accumulating), directly contradicting the "Running" badge and the "Deck Engineer Agent · streaming" sub-header shown a few pixels away on the same screen.

REPRODUCTION

1. Sign in as qa-admin, navigate to `http://localhost:3000/runs/d6e425b6-df0d-4f54-9bba-f4a771e33812`.
2. Confirm run count via GET `http://localhost:8000/api/runs?limit=50` (baseline, run status failed).
3. Click "Reopen & fix from the failed step" in the Resume options panel.
4. Page navigates to `/runs/{id}/stream`. Confirm via GET `/api/runs/{id}` that status is now generating (a genuine resume, not a no-op).

1. Observe the header badge reads "Running", the sub-header under the run title cycles through "Running · streaming" → "Deck Engineer Agent · streaming", and the pipeline chips in the chat transcript show agents moving to "Thinking..." then "Writing..." with rising token counts.

1. In the Steps tab (right panel), observe the status summary line directly above the agent list still reads **"Run failed"** with a stale 1 / 4 agents · 3s count that never updates as the run genuinely progresses to agent 2 and beyond.

1. Reproduced continuously across ~20s of live progress (0/4 → 1/4 agents, Presentation

Strategist Agent completing at 51s/18.4K tokens, Deck Engineer Agent starting) — the "Run failed" label never once updated to "Running" or cleared, even as every other live indicator on the same page updated correctly.

EXPECTED

While a run is actively generating/running, the Steps panel's status summary should show a state consistent with "Running" (matching the header badge and sub-header), not the terminal "Run failed" label left over from before the reopen.

ACTUAL

The Steps panel's status summary is frozen at "Run failed" for the entire observed live-run duration, directly contradicting the "Running" badge and the actively-streaming agent sub-header rendered simultaneously on the same page.

EVIDENCE

- Before (failed run, Resume options visible): bug-hunter/evidence/runs-failed/BUG-20260828-085400-runs-failed/01-before-reopen.png
- Just after reopen (header "Running" vs Steps panel "Run failed"): bug-hunter/evidence/runs-failed/BUG-20260828-085400-runs-failed/02-reopened-running-vs-failed-badge.png
- Mid-progress (agent 2 "Writing...", tokens climbing, panel still "Run failed"): bug-hunter/evidence/runs-failed/BUG-20260828-085400-runs-failed/03-stale-run-failed-label-during-progress.png
- Persisted (same stale label ~15s later): bug-hunter/evidence/runs-failed/BUG-20260828-085400-runs-failed/04-stale-run-failed-label-persists.png

BROWSER SIGNALS

- Console: none observed
- Network: GET /api/runs/{id} confirmed status: "generating" while the UI showed the contradictory "Run failed" label — not a stale API response, the frontend simply doesn't re-derive this one label from live stream state.
- State/URL: URL settles at /runs/{id}/stream; run stopped via the "Stop" control at the end of the investigation to avoid abandoning a live run (the seeded fixture d6e425b6... itself was not deleted or altered).

ID `BUG-20260828-085400-runs-failed` Found by `bug-runs-failed-r2`

Fingerprint `/runs/{id}/stream|steps-panel-status-badge|reopen-and-fix-from-failed-step|stale-run-failed-label-persists-during-live-progress`

A run whose "Run Again" pipeline actually failed keeps reporting `status: "cancelled"` everywhere, permanently misrepresenting its real outcome

`/runs/{id}`, `/runs`

SUMMARY

Run `a8dfa959-e233-4ddf-87ce-d9a942cefde3` was seeded cancelled. A prior worker used "Run Again" on it; the backend's re-run pipeline genuinely failed (GET `/api/runs/{id}/events` shows two `agent_error` events followed by `pipeline_failed` at seq 29 and a `chat_reply` "What went wrong" at seq 30 — all 4 agents ended in `agents_failed`). This is a durable, persisted state change, not a transient rendering glitch: GET `/api/runs/{id}` still returns `"status": "cancelled"`, `"error": null` on every fresh fetch since, the run detail page's own badge still reads "Cancelled" / "Cancelled by you" on a full reload, and the `/runs` history list still tags the same run "cancelled". Yet the same detail page's body — derived from the event stream rather than the `status` field — simultaneously renders "Failed agents: Presentation Strategist Agent, PPTX Code Generator, Deck Engineer Agent, Deck QA Agent" directly under a "This run was cancelled" heading. The run's canonical `status` field never gets corrected to `failed`, so the badge, the history list, and the API all permanently misreport a failed pipeline run as a user-cancelled one, while the body of the very same page contradicts that badge by listing failed agents. This is a different, independently-actionable defect from the already-filed `BUG-20260828-015930-runs-cancelled` (the live `/stream` view freezing mid-render before any reload) — that bug is about the live page failing to *render* an update; this one is about the run's persisted `status` field itself being permanently wrong, confirmed via direct API calls with no live view or stream involved at all, and visible on an entirely different page (the `/runs` history list).

REPRODUCTION

1. Sign in as qa-admin. Load `http://localhost:3000/runs/a8dfa959-e233-4ddf-87ce-d9a942cefde3`

(a fresh page load, no live view).

1. Observe the header badge reads "Cancelled" / "Cancelled by you", and the resume-panel-turned result panel shows "This run was cancelled — The run was stopped before producing a deliverable." directly above a "Failed agents" list naming all 4 agents.

1. Independently call `GET /api/runs/a8dfa959-e233-4ddf-87ce-d9a942cefde3` — response has `"status": "cancelled", "error": null`, with no field indicating the pipeline actually failed.

1. Independently call `GET /api/runs/a8dfa959-e233-4ddf-87ce-d9a942cefde3/events` — tail of the stream shows `agent_error x2, pipeline_failed (seq 29), chat_reply "What went wrong" (seq 30)`, confirming the pipeline genuinely failed rather than being cancelled.

1. Load `http://localhost:3000/runs` (Run History list) — the same run's card is still tagged "cancelled", matching the wrong API status, not the true failed outcome.

EXPECTED

Once a run's pipeline fails (`pipeline_failed` emitted, all agents in `agents_failed`), the run's own persisted status should update to `failed` (or an equivalent terminal state that reflects the real outcome), and every surface reading that field — the detail page badge, the Run History list, and the API — should agree with each other and with the failed-agents content already rendered on the same page.

ACTUAL

The run's status field is permanently stuck at `cancelled` after a re-run pipeline failure.

The detail page badge, the Run History list card, and the raw API response all report "cancelled" indefinitely, while the same detail page's own body simultaneously lists "Failed agents" — a self-contradictory page, and a run-outcome record that is durably wrong across every surface that reads it.

EVIDENCE

- Detail page (badge "Cancelled" + "Failed agents" list, same view):

bug-hunter/evidence/runs-cancelled/BUG-20260828-085830-runs-cancelled/01-detail-badge-cancelled-but-failed-agents-listed.png

- Run History list (same run still tagged "cancelled"):

bug-hunter/evidence/runs-cancelled/BUG-20260828-085830-runs-cancelled/02-runs-list-still-shows-cancelled.png

- Raw API response (status: "cancelled", error: null):

bug-hunter/evidence/runs-cancelled/BUG-20260828-085830-runs-cancelled/api-run-status.json

- Network excerpt: bug-hunter/evidence/runs-cancelled/BUG-20260828-085830-runs-cancelled/network.log

BROWSER SIGNALS

- Console: no relevant error observed.
- Network: GET /api/runs/{id} → status: "cancelled", error: null; GET

/api/runs/{id}/events tail → agent_error ×2, pipeline_failed, chat_reply.

- State/URL: reproduced on a fresh page load of /runs/{id} (no live /stream view involved)

and independently on /runs; not a rendering/polling artifact.

ID `BUG-20260828-085830-runs-cancelled` Found by `bug-runs-cancelled-r2`

Fingerprint `/runs/{id}|run-status-field|run-again-on-a-cancelled-run-that-then-fails|status-permanently-reports-cancelled-instead-of-failed`

Library agent detail's Skills tab "Add" gives success feedback (checkmark + badge count) but persists nothing — silently discarded on reload, no Save affordance exists at all

`/library/agents/<agentId>` (e.g. `/library/agents/material-analyzer`)

SUMMARY

On the Library agent detail drawer's Skills tab, clicking the "+" (Add) button on any catalog skill immediately renders full success feedback — the button becomes a filled checkmark and a count badge (e.g. "1") appears on the "SKILLS" section header — implying the skill was attached to the agent. No network request fires for this action (confirmed via the network log: zero new requests appear between before/after states), and unlike the Config tab this Skills tab has no Save/Cancel button at all — there is no way to persist the change even in principle. A reload of the exact same URL silently wipes the selection back to zero, with no warning, toast, or indication anything was lost. This is a materially different (and arguably worse) gap than the already-filed `BUG-20260828-025950-library-agents-id` ("Save agent" Config override silently discards with no API call): that bug at least has a Save button whose click fails silently; this Skills tab has no persistence path whatsoever, yet gives the same convincing "it worked" visual confirmation.

REPRODUCTION

1. Sign in as qa-admin, navigate to `http://localhost:3000/library/agents/material-analyzer`

(Architecture Agent detail drawer opens).

1. Click the "Skills" tab. Observe the full skill catalog with "+" add buttons and no count badge on the "SKILLS" header.

1. Click "Add .NET Backend Expert" (or any skill's "+"). Observe the button becomes a filled checkmark and a "1" badge appears next to "SKILLS".

1. Confirm no new request appears in the network log after the click (GET

`/api/agents/library`, `/api/skills/library`, `/api/hooks/library` are the only agent/skill

calls, all from initial page load).

1. Reload `http://localhost:3000/library/agents/material-analyzer` (same URL), reopen the Skills tab. Observe: the badge is gone and ".NET Backend Expert" shows an empty "+" again — the addition is completely lost.

1. Repeated with a second skill (".NET Development Patterns") on a fresh load: same result — checkmark + badge appear immediately, zero network calls, reload wipes it.

EXPECTED

Either the "Add" action persists the skill grant (firing a save request, surviving reload), or the UI does not present unambiguous success feedback (checkmark, count badge) for a change that cannot be saved — e.g. a visible "unsaved changes" state and an explicit Save control, matching the pattern used on the Config tab.

ACTUAL

The Skills tab shows full success feedback (checkmark, badge count) for an action that fires no network request and has no save mechanism of any kind on that tab. The change is silently discarded on reload with zero indication to the user that nothing was persisted.

EVIDENCE

- Before add: `bug-hunter/evidence/library-agents-id/BUG-20260828-092630-library-agents-id/01-before-add.png`
- After add (checkmark + badge "1", repro 1): `bug-hunter/evidence/library-agents-id/BUG-20260828-092630-library-agents-id/02-after-add-checkmark-badge.png`
- After reload (state lost, repro 1): `bug-hunter/evidence/library-agents-id/BUG-20260828-092630-library-agents-id/03-after-reload-silently-discarded.png`
- After add (repro 2, different skill): `bug-hunter/evidence/library-agents-id/BUG-20260828-092630-library-agents-id/04-repro2-after-add.png`
- After reload (repro 2, state lost again): `bug-hunter/evidence/library-agents-id/BUG-20260828-092630-library-agents-id/05-repro2-after-reload-discarded.png`

BROWSER SIGNALS

- Console: no errors observed.
- Network: no request fires for the Add-skill click in either repro; `browser_network_requests`

taken immediately after each click shows only the pre-existing page-load calls

(/api/agents/library, /api/skills/library, /api/hooks/library, etc.).

- State/URL: `location.href` unchanged throughout (/library/agents/material-analyzer); the checkmark/badge state lives only in React component state and does not survive a reload.

ID `BUG-20260828-092630-library-agents-id` Found by `bug-library-agents-id-r2`

Fingerprint `/library/agents/<id>|agent-detail-skills-tab-add-skill|click-add-skill-then-reload|checkmark-and-badge-shown-but-no-network-call-and-state-lost-on-reload`

Concurrent `PUT /api/settings/preferences` calls silently lose one write and return a false-success response with the WRONG persisted value

/settings/ai-model

SUMMARY

`PUT /api/settings/preferences` is not safe under concurrent writes for the same user. When two `PUT` requests with different `preferred_model` values are in flight at the same time (e.g. a double-click on Save, or a rapid model-then-Save, model-then-Save sequence before the first request's response returns — plausible given the page's own network latency and the documented app-wide pattern of only-the-last-action-wins races), the backend performs what looks like an unsynchronized read-modify-write: one of the two writes is completely lost (never reaches the database in any form), **and** the losing request's own HTTP response is `200 OK` with a JSON body whose `preferred_model` field reports the *other* request's value — not the value that request itself sent, and not an error. This is worse than a simple race: the client that requested, say, Opus 4.6 receives a "successful" 200 response body telling it the saved preference is Sonnet 4.5, which is silently false. Reproduced 3 times in a row with different model pairs, 100% reproduction rate (not intermittent-only — every trial showed the same shape: both responses converged on one value, and the final persisted state matched neither request's program order, i.e. it was not simply "last network request wins").

REPRODUCTION

1. Sign in as qa-admin, confirm baseline `preferred_model` is `eu.anthropic.claude-haiku-4-5-20251001-v1:0` (Claude Haiku 4.5) on `/settings/ai-model`.
2. Via `browser_evaluate`, fire two `PUT /api/settings/preferences` requests concurrently (`Promise.all`) with different `preferred_model` values, e.g. request A = `claude-opus-4-6-v1`, request B = `claude-sonnet-4-5-20250929-v1:0`.

1. Observe: request A returns `200 OK` with body `preferred_model: "...sonnet-4-5..."` — NOT `opus-4-6`, the value A itself requested. Request B correctly returns `sonnet-4-5`.

1. `GET /api/settings/preferences` immediately after confirms the DB state is `sonnet-4-5` —

request A's write (opus-4-6) is gone with no trace and no error was ever surfaced anywhere.

1. Repeated with a second model pair (sonnet-4-5 vs haiku-4-5) — same shape: the losing request's response body reported the winner's value instead of its own requested value.

1. Repeated a third time (opus-4-6 vs sonnet-4-5, order swapped) — same result again: 3/3 reproductions, 100% hit rate on this exact concurrent pattern.

1. Reloading /settings/ai-model after each trial confirms the UI dropdown reflects the DB's (non-deterministic, program-order-independent) winner — the losing selection vanishes with no error toast, no console error, and no network-level indication anything went wrong (the losing request's own network tab shows 200 success).

1. Restored preferred_model to the original eu.anthropic.claude-haiku-4-5-20251001-v1:0 via a final PUT, verified via reload.

EXPECTED

Either the second concurrent write should simply win deterministically (last-write-wins, which is an acceptable UX for a single-user preference) with each response accurately reflecting what that specific request persisted, or the API should serialize/reject overlapping writes (e.g. via optimistic concurrency / a 409). At minimum, a 200 OK response body must never report a preferred_model value different from the one the response's own request just sent and that request's write must not be silently discarded without any error surfaced to the caller.

ACTUAL

One of the two concurrent PUT requests is a silent no-op: its write never reaches the database, and its own 200 OK response body falsely reports the *other* request's value as the current preference — a false-positive success with fabricated response content, not merely a lost update.

EVIDENCE

- Before (baseline, Haiku 4.5 selected): bug-hunter/evidence/settings-ai-model/BUG-20260828-094120-settings-ai-model-r2/01-before-race-test.png
- Failure (after the race, UI shows Sonnet 4.5 though the concurrent test also targeted Opus 4.6): bug-hunter/evidence/settings-ai-model/BUG-20260828-094120-settings-ai-model-r2/02-

after-race-lost-update.png

- Restored (Haiku 4.5 reinstated as original baseline): bug-hunter/evidence/settings-ai-model/BUG-20260828-094120-settings-ai-model-r2/03-restored-haiku45.png

BROWSER SIGNALS

- Console: none observed — no client-side error at any point
- Network: both concurrent PUT /api/settings/preferences requests return 200 OK; the losing

request's JSON body contains the winning request's preferred_model value instead of its own

- State/URL: URL stays /settings/ai-model throughout; verified via direct GET /api/settings/preferences

before/after each trial that the DB-persisted value silently diverges from one of the two

requested values with no error path anywhere in the stack

ID BUG-20260828-094120-settings-ai-model-r2 Found by bug-settings-ai-model-r2

Fingerprint /settings/ai-model|pipeline-model-preferences-api|two-concurrent-PUT-preferences-requests|lost-update-plus-response-body-reports-wrong-preferred-model

66 **MEDIUM** **`/create/<type>` composer has no tier gating at all; a basic-tier account reaches a fully configured, submit-ready wizard for a deliverable its own Usage & Limits page says it cannot run**
`/create/prototype` (reproducible pattern likely applies to any tier-restricted `/create/<type>`)

SUMMARY

`/settings/usage` correctly tells a basic-tier account it only has "Deliverable access" to 2 of 8 types (Product Requirements, Presentation) — confirmed accurate against `TIER_PIPELINES["basic"]` in `backend/app/core/entitlements.py`. The dashboard catalog enforces this consistently too: the "Build an interactive prototype" card is rendered disabled with a "Requires Pro plan" lock badge for a basic-tier account, with no click path to its composer. However, that gating lives ONLY on the dashboard card — the `/create/prototype` route itself performs no entitlement check at all. Navigating a basic-tier session directly to `/create/prototype` renders the identical, fully interactive multi-step composer a Pro/Enterprise account gets: brief textarea, Attach file/Voice, the full Template gallery (60+ templates, category filters, "Upload custom"), Design System and Discovery tabs, and — after typing a brief and picking a template — an enabled "Continue" button that advances the wizard further. Nothing in this flow indicates the account is not entitled to run it; the only place that information exists is the dashboard card the user never has to visit if they have (or guess) the URL. Backend launch endpoints DO still enforce `can_run_pipeline` at actual run creation (`run_commands.py`'s `_require_tier_entitlement`, added as a P1 fix per its own comment), so this is a client-side-only gap — but a real one: a basic-tier user can invest genuine effort (write a brief, pick a template, click through the wizard) before any rejection, directly contradicting what `/settings/usage` told them on the very page whose stated purpose is to explain what their plan allows.

REPRODUCTION

1. Sign in as `qa-admin@flowinqa.com` (enterprise), then switch session to

qa-basic@flowinqa.com / flowin-e2e-pass (tier: basic per /api/auth/me).

1. Navigate to /settings/usage. Confirm "Basic plan" heading and "Deliverable access" listing only "Product Requirements" and "Presentation" — Interactive Prototype is absent.

1. Navigate to /dashboard. Confirm the "Build an interactive prototype" card is disabled, shows a lock icon and a "Requires Pro plan" badge, with no click target reaching its composer.

1. Navigate directly to http://localhost:3000/create/prototype (same basic-tier session, no re-login). Observe: the full "Configure your prototype" wizard renders — brief textarea, Attach file/Voice controls, Advanced (5 agents), and a Template tab with the entire template gallery, all fully interactive, disabled: false on every control.

1. Type a brief into the textarea, leave "No template" selected. Observe the footer now shows an enabled "Continue" button (disabled: false) alongside "Save workflow" / "Save as my version" — a basic-tier account reaches a submit-ready wizard state for a pipeline type its own plan page says it cannot access, with no lock, no warning, no upgrade prompt anywhere in this flow.

EXPECTED

A route the dashboard already gates by tier (disabled card + "Requires Pro plan") should not be independently reachable and fully usable via direct URL for an unentitled tier — at minimum the composer should show the same locked/upgrade state the dashboard card does, consistent with what /settings/usage tells the same account about its own plan.

ACTUAL

/create/prototype performs no tier check: it loads and is fully interactive for a basic-tier account, reaching an enabled "Continue" state, with zero indication anywhere in the flow that the account is not entitled to this deliverable — a UI-only gap, since the backend launch endpoint still correctly rejects the pipeline_type at actual run creation.

EVIDENCE

- Before (dashboard catalog card, basic tier, locked "Requires Pro plan"): bug-hunter/evidence/settings-usage/BUG-20260828-094937-settings-usage-r2/01-dashboard-catalog-locked-basic.png

- Failure (/create/prototype fully loaded and interactive, basic tier, direct URL): bug-hunter/evidence/settings-usage/BUG-20260828-094937-settings-usage-r2/02-create-prototype-fully-loaded-basic.png
- Failure (brief typed, "Continue" enabled, basic tier): bug-hunter/evidence/settings-usage/BUG-20260828-094937-settings-usage-r2/03-continue-button-enabled-basic.png

BROWSER SIGNALS

- Console: no errors on navigation to /create/prototype as basic tier.
- Network: page renders from client-side route/component state; no entitlement-check request is

ever made client-side for this route.

- State/URL: location.href correctly reads /create/prototype throughout; GET

http://localhost:8000/api/auth/me confirms tier: "basic" for the session used.

- Code: backend/app/api/run_commands.py::_require_tier_entitlement (calls can_run_pipeline) IS wired into the REST launch/revision/resume endpoints — the backend gap this resembles was already fixed as a P1; only the composer route's client-side gating is missing. Related in shape (catalog-level lock is cosmetic, the underlying route is unguarded) to BUG-20260828-050115-library (Coming-Soon library agents reachable by direct URL) but a distinct route, component and trigger (subscription-tier entitlement vs. unreleased-feature flag), not a duplicate of it.

ID `BUG-20260828-094937-settings-usage-r2` Found by `bug-settings-usage-r2`

Fingerprint `/create/prototype|composer-wizard|direct-url-navigation-basic-tier|full-multistep-wizard-loads-and-reaches-enabled-continue-despite-tier-not-entitled`

"Clear" on Constitution deletes the saved value immediately, with no confirmation and no relation to "Save"

/settings/constitution

SUMMARY

The "Clear" button next to "Save constitution" does not behave like a form-reset control — it fires `DELETE /api/settings/constitution` immediately on click, permanently erasing the user's saved constitution (the standing instruction set the page says "is prepended to every agent on every run") with zero confirmation dialog, zero undo, and no requirement to also click "Save". Its placement directly beside "Save constitution", and the fact that typing into the textarea never auto-saves, strongly implies "Clear" is a local/pending-edit reset — but it is in fact a destructive, irreversible, immediately-persisted account-wide delete. A user who clicks it to reset a draft edit (or misclicks it instead of "Save") loses their real saved constitution with one click and no way back except retyping it from memory.

REPRODUCTION

1. Sign in as qa-admin, navigate to `/settings/constitution`. Confirm existing saved content ("E2E S-09-12: answer in exactly one sentence.", 44/4000 chars).

1. Click "Clear" (no typing needed — reproduces on the saved value directly, and also reproduces after typing unsaved draft text first).

1. Observe: textarea instantly empties, counter resets to "0 / 4000 chars", a "Constitution cleared." status message appears, and the "Save constitution" button becomes disabled.

1. Inspect network: `DELETE http://localhost:8000/api/settings/constitution` fires immediately

on the click and returns `204 No Content` — no confirm dialog was shown, no separate "Save" click was needed.

1. Confirm persistence: `GET /api/settings/constitution` (direct, with bearer token) returns `{"content": null}`. Reloading the page shows the empty textarea — the deletion is real and

permanent, not merely a pending local edit.

1. Reproduced twice independently (once after first typing unsaved draft text over the saved value, once by clicking Clear directly on the untouched saved value) — both times the backend content is deleted instantly with no confirmation.

1. Restored the original content ("E2E S-09-12: answer in exactly one sentence.") via the textarea + "Save constitution" both times, and verified via GET and a full page reload that the original text is back before finishing.

EXPECTED

A destructive, irreversible action that erases a user's standing instructions for every future agent run should require an explicit confirmation step (dialog, or at minimum require also clicking "Save constitution" to commit the empty state) before it is persisted to the backend — consistent with how "Save constitution" itself requires a deliberate, separate click to persist any change.

ACTUAL

"Clear" deletes the saved constitution on the backend instantly on click, with no confirmation and independently of "Save constitution" — a single misclick permanently destroys the user's constitution.

EVIDENCE

- Before: bug-hunter/evidence/settings-constitution/BUG-20260828-095800-settings-constitution-r2/01-before.png
- Failure (cleared instantly after one click on "Clear"): bug-hunter/evidence/settings-constitution/BUG-20260828-095800-settings-constitution-r2/02-failure.png
- After restore + reload: bug-hunter/evidence/settings-constitution/BUG-20260828-095800-settings-constitution-r2/03-after-restore-reload.png

BROWSER SIGNALS

- Console: none observed
- Network: DELETE http://localhost:8000/api/settings/constitution → 204 No Content, fired

synchronously on the "Clear" click, no preceding confirmation request/dialog

- State/URL: URL stays /settings/constitution throughout; GET /api/settings/constitution

confirms content: null after Clear, and confirms restored content after re-save + reload

ID BUG-20260828-095800-settings-constitution-r2

Found by bug-settings-constitution-r2

Fingerprint /settings/constitution|clear-button|click-clear|immediate-unconfirmed-delete-of-saved-constitution

68 **MEDIUM** **Mid-session token expiry drops the original protected route; re-login always lands on /dashboard instead of using the working `?redirect=` mechanism**
`/login?expired=true` (entered via a protected route's 401)

SUMMARY

The app has a working return-path mechanism: a **signed-out** user deep-linking a protected route is sent to `/login?redirect=%2F<path>` and, after signing in, is correctly returned to that exact route (verified here on `/settings/ai-model`, and already established by a sibling worker across `/create`, `/settings`, `/workflow`, and `/runs/<id>/workspace`). But when a session expires **mid-session** on a protected route (the 401 ladder auto-redirecting an already-signed-in user to `/login?expired=true`), that same mechanism is not used: no `redirect` query param is ever attached, and after signing back in the user is unconditionally sent to `/dashboard`, losing the page they were actually on. This is distinct from the filed High (BUG-20260827-223300-login-expired-true, an already-authenticated user with a **still-valid** token seeing the login form) — this bug concerns a **genuinely** expired/invalid token and the destination after a **successful** re-login.

REPRODUCTION

1. Signed in as qa-admin, navigate to `http://localhost:3000/settings/ai-model` (confirms loads normally).
1. In devtools, corrupt the token: `localStorage.setItem('auth_token', 'corrupted-expired-token-xyz')`.
2. Reload/renavigate to the same URL. The 401 ladder fires and the app redirects to `http://localhost:3000/login?expired=true` — no `redirect` param.
1. **Contrast check:** clear `auth_token` entirely (fully signed out) and navigate to `http://localhost:3000/settings/ai-model` directly. The app correctly redirects to `http://localhost:3000/login?redirect=%2Fsettings%2Fai-model`.
1. From the `?redirect=` URL, sign in as qa-admin: correctly lands back on

/settings/ai-model (the mechanism works).

1. Repeat steps 1–3 from <http://localhost:3000/runs/b9feac1c-ec21-4531-8ba7-bb391786993e/workspace>

instead: same result, /login?expired=true with no redirect param.

1. From the ?expired=true state (both routes tested), sign in as qa-admin: lands on /dashboard in both cases, not /settings/ai-model or the run workspace.

EXPECTED

A mid-session expiry on a protected route should preserve that route the same way the signed-out deep-link guard does (?redirect=%2F<path>), so re-authenticating returns the user to what they were doing — consistent with the app's own established pattern for this exact scenario elsewhere.

ACTUAL

The 401-triggered redirect to /login?expired=true never attaches a redirect param. Signing back in always lands on /dashboard, discarding the original route, even though the identical mechanism works correctly when the user was signed out to begin with.

EVIDENCE

- Before (settings/ai-model loads normally): bug-hunter/evidence/login-expired-true/BUG-20260828-101500-login-expired-true-r2/01-before-settings-ai-model.png
- Failure (corrupted token → /login?expired=true, no redirect param, settings route): bug-hunter/evidence/login-expired-true/BUG-20260828-101500-login-expired-true-r2/02-expired-no-redirect-param-settings.png
- Failure (re-login lands on /dashboard, not /settings/ai-model): bug-hunter/evidence/login-expired-true/BUG-20260828-101500-login-expired-true-r2/03-landed-dashboard-not-settings.png
- Second route repro (run workspace, same no-redirect-param failure): bug-hunter/evidence/login-expired-true/BUG-20260828-101500-login-expired-true-r2/04-expired-run-workspace-no-redirect-param.png
- Second route repro (re-login lands on /dashboard, not the run workspace): bug-hunter/evidence/login-expired-true/BUG-20260828-101500-login-expired-true-r2/05-after-relogin-dashboard-not-run-workspace.png

BROWSER SIGNALS

- Console: no errors related to the redirect logic itself
- Network: 401 on the protected route's API call triggers the auth-guard redirect to

`/login?expired=true`; no redirect query param is constructed for this path, unlike the signed-out guard which does build `?redirect=%2F<path>`

- State/URL: confirmed on two distinct protected routes (`/settings/ai-model`, `/runs/<id>/workspace`); both produce `/login?expired=true` with no redirect param, and both land on `/dashboard` after re-login, contrasted against the working `?redirect=` flow verified from a fully signed-out state

ID `BUG-20260828-101500-login-expired-true-r2` Found by `bug-login-expired-true-r2`

Fingerprint `/login|auth-guard|mid-session-token-expiry-on-protected-route|redirect-param-dropped-lands-on-dashboard-not-original-route`

Create-User dialog silently swallows a duplicate-email 409, giving the admin zero feedback that submission failed

/admin

SUMMARY

The "Create New User" dialog on /admin does not check for an existing email before enabling submission, and when the backend correctly rejects a duplicate with 409 Conflict and a clear message ("A user with this email already exists."), the frontend discards the error entirely. The modal stays open with the same values still filled in, the "Create User" button re-enables, and nothing on the page — no toast, no inline field error, no alert region text — tells the admin the submission failed. The row count and stat tiles stay unchanged (the correct outcome), so the *data* is safe, but the admin has no way to tell "it failed because the email is taken" apart from "nothing happened, maybe it's still working" without opening devtools. This is a distinct component/trigger from the already-filed /runs/{id} resume-button 409 swallow (different route, different action, same underlying "response awaited but never surfaced" shape) and from D-32 (D-32 is specifically the admin's own-tier change being applied *optimistically* before the refusal; here nothing is applied and nothing is reported either way).

REPRODUCTION

1. Sign in as qa-admin, navigate to /admin.
2. Click "Add user", create a throwaway user zz-hunt-dup@flowinqa.com / password zzhuntpass123, Basic plan, no admin grant. Confirm it appears in the table (row count 4→5).
1. Click "Add user" again, enter the SAME email zz-hunt-dup@flowinqa.com with a different password, leave Plan as Basic, click "Create User".

1. Observe: console shows Failed to load resource: ... 409 (Conflict) @ .../api/admin/users,

network tab confirms POST /api/admin/users → 409 with body

{"detail":"A user with this email already exists."}. The modal remains open with the typed

values still present and the button re-enabled. No error text, toast, or alert appears

anywhere in the DOM (`document.body.innerText` contains neither "already" nor "exists").

1. Repeated step 3-4 a second time (clicked "Create User" again on the same open dialog) — identical result: another 409, still zero visible feedback.

1. Cleanup: closed the dialog, deleted the throwaway `zz-hunt-dup@flowinqa.com` user via its row's Delete action, confirmed via the table that the user count returned to 4 (original 4 seeded users only).

EXPECTED

On a 409 (or any) rejection from `POST /api/admin/users`, the dialog should surface the server's message (e.g. via the same toast pattern already used for a successful create — "User X created") — something like "A user with this email already exists." — so the admin knows the create failed and why, rather than being left to guess.

ACTUAL

The rejection is caught (the modal doesn't crash and the button re-enables) but never displayed.

The admin sees no difference between "it's still processing" and "it silently failed."

EVIDENCE

- Before (5 users, throwaway already created once): `bug-hunter/evidence/admin/BUG-20260828-103434-admin/01-before-user-created-once.png`
- Failure (duplicate-email submit, dialog still open, no error text anywhere): `bug-hunter/evidence/admin/BUG-20260828-103434-admin/02-failure-duplicate-submit-no-error.png`
- Second reproduction (resubmitted, identical silent failure): `bug-hunter/evidence/admin/BUG-20260828-103434-admin/03-repro2-still-no-error.png`
- Network: `bug-hunter/evidence/admin/BUG-20260828-103434-admin/network.log`

BROWSER SIGNALS

- Console: Failed to load resource: the server responded with a status of 409 (Conflict) @ `http://localhost:8000/api/admin/users:0` (both attempts).
- Network: `POST /api/admin/users → 409 Conflict`, body `{"detail":"A user with this email already exists."}`.
- State/URL: stays on `/admin` with the Create New User modal open; table/user count correctly unchanged (no phantom row created).

ID `BUG-20260828-103434-admin` Found by `bug-admin-r2`

Fingerprint `/admin|create-user-dialog|submit-duplicate-email|409-rejected-no-user-feedback`

File explorer/download parses a markdown subheading in the deliverable as a phantom "file" with no real name or extension

/preview-fullscreen (ready state, real `__app_preview__`` payload from a completed App Builder run's "Full Screen" button)

SUMMARY

Round 1 could not obtain a real payload for this page and hand-seeded `sessionStorage['__app_preview__']`, flagging that as weaker evidence. This round used a genuine payload: the existing completed App Builder run "A hello world app." (37aabc96-6e71-4d2b-ac30-90a827c8b862), reached the fullscreen preview via the real `AppBuilderPreview.handleFullscreen()` "Full Screen" button (not a forged payload). The run's single Documentation Agent produced a README containing a `### Via Node.js` subsection with a fenced javascript code sample. The App Builder preview's file-list builder incorrectly parses this markdown subheading as a standalone project file — it appears in the Explorer tree, in the file-count footer ("4 files"), and in the individual-file code viewer, with name/path literally set to "Via Node.js" (a phrase with a space, no file extension, and no relation to any real project path). Clicking its per-file "Download" button (`AppBuilderPreview.handleDownload`, `frontend/src/components/preview/AppBuilderPreview.tsx:248-254`) sets `a.download = file.name` verbatim, so the browser saves an extensionless file literally named "Via Node.js". The same phantom entry is also bundled into "Download ZIP" as a top-level, extensionless 203-byte entry. The deliverable's real generated code files (per the README's own "Project Structure" section: `app.js`, `package.json`, etc.) are not present in the preview at all — only documentation files and this one phantom "file" are shown — so the file tree materially misrepresents the actual project.

REPRODUCTION

1. Sign in as qa-admin, go to /runs, filter "App Builder", open the completed run "A hello world app." (37aabc96-6e71-4d2b-ac30-90a827c8b862).
2. On the Preview tab, in the App Builder file explorer, observe the file list: `API_DOCUMENTATION.md`, `ARCHITECTURE_DECISIONS.md`, `README.md`, `Via Node.js` — footer reads "4 files".
3. Click "Full Screen" — a new tab opens at /preview-fullscreen reading the real `sessionStorage['__app_preview__']` payload just written by the opener. Same 4-entry file list appears, including "Via Node.js".
4. In that fullscreen tab, select the "Via Node.js" entry (or it is selected by default) — the code viewer header shows filename "Via Node.js", language "js", "8 lines" — actually just the JS snippet copy-pasted out of the README's code fence.
5. Click the per-file "Download" button next to "Copy" in the code viewer header.
6. Observe (via `Playwright.download.suggestedFilename()`, reproduced twice): the browser download's suggested filename is "Via Node.js" — no extension, containing a literal space.
7. Click "Download ZIP" and inspect the archive (`unzip -l`): it contains a top-level `Via Node.js` entry (203 bytes, no extension) alongside the three real `.md` files.

EXPECTED

The file explorer/preview should only list files that genuinely exist in the generated project (matching the deliverable's own "Project Structure" section), and any per-file download should save with a real, valid filename/extension. A markdown subheading inside a documentation file's content should never be split out into its own top-level "file".

ACTUAL

A `### Via Node.js` subheading (with an embedded JS code sample) inside the generated README is parsed as an independent project file named "Via Node.js", shown in the Explorer, counted in the footer's file count, and downloadable (both individually and via "Download ZIP") as a malformed, extensionless file literally named "Via Node.js".

EVIDENCE

- Before (fullscreen tab, real payload, explorer showing the phantom "Via Node.js" entry among real files): `bug-hunter/evidence/preview-fullscreen/BUG-20260828-105300-preview-fullscreen-r2/01-before-explorer-with-phantom-file.png`
- Failure (phantom file selected in code viewer, showing filename/extension/Download control): `bug-hunter/evidence/preview-fullscreen/BUG-20260828-105300-preview-fullscreen-r2/02-phantom-file-selected-code-viewer.png`
- Additional: `bug-hunter/evidence/preview-fullscreen/BUG-20260828-105300-preview-fullscreen-r2/notes.md` (source deliverable excerpt confirming the markdown origin, relevant `AppBuilderPreview.tsx` lines, and the ZIP-pollution check)

BROWSER SIGNALS

- Console: none observed
- Network: none relevant — this is a client-side parsing/rendering defect, not a failed request
- State/URL: reproduced on `http://localhost:3000/preview-fullscreen` (genuine `__app_preview__` payload written by the real "Full Screen" button on run `37aabc96-6e71-4d2b-ac30-90a827c8b862`); `download.suggestedFilename() = "Via Node.js"` on two independent clicks

ID `BUG-20260828-105300-preview-fullscreen-r2` Found by `bug-preview-fullscreen-r2`

Fingerprint `/preview-fullscreen|app-builder-preview-file-list-builder|open-fullscreen-preview-for-a-real-app-builder-run-whose-deliverable-contains-a-markdown-###-subheading-with-a-fenced-code-sample|subheading-is-listed-as-its-own-file-with-a-malformed-name-and-downloads-as-an-extensionless-file-named-after-the-heading-text`

Audit tab's CSV and JSON export strip the Security/Governance category distinction and never populate the promised "severity" column

`/runs/b9feac1c-ec21-4531-8ba7-bb391786993e/audit`

SUMMARY

The Audit tab's rows visibly carry two different category badges on screen — "Gate" for governance/gate rows and "Security" for secret-scan rows (pill counts read "Governance 96, Security 4"). The Export menu explicitly advertises that both the CSV ("flattened rows timestamp · agent · category · event · outcome · severity") and JSON ("raw rows incl. category · step · outcome · severity · timestamp") exports include a category and a severity field. In the actual downloaded files, every one of the 100 exported rows has category hard-coded to "gate" — including the 4 rows the UI itself badges "Security" — and severity is an empty string on every single row, despite the page's own persistent footer copy promising "every entry carries severity + timestamp · exportable to CSV / JSON for compliance review."

REPRODUCTION

1. Sign in as qa-admin, navigate to `http://localhost:3000/runs/b9feac1c-ec21-4531-8ba7-bb391786993e/audit`.
2. Note the on-screen category badges differ per row: governance/gate rows show "Gate", the 4 secret-scan rows show "Security" (pill counts "Governance 96", "Security 4").
1. Click "Export" -> "CSV — flattened rows timestamp · agent · category · event · outcome · severity". A file downloads.
1. Open the downloaded CSV. Every row's category column reads gate, including the rows whose label is "Secret scan — scanned before write" (the ones badged "Security" on screen). Every row's severity column is empty.
1. Repeat with "Export" -> "JSON — raw rows incl. category · step · outcome · severity · timestamp". Same result: category is "gate" for all 100 objects, severity is "" for all 100 objects.

EXPECTED

The exported category field should reflect the same Security/Governance distinction visible in the UI (or at minimum not silently collapse it to a single value), and the severity field that both the export menu and the page's own footer copy explicitly promise should be populated per the app's stated severity model, not permanently empty.

ACTUAL

Both export formats always write category: "gate" regardless of the row's on-screen badge, erasing the Security/Governance split the UI itself displays, and severity is unconditionally empty on every row in every export, contradicting the feature's own advertised column list and its "compliance review" framing.

EVIDENCE

- Before (Audit tab loaded, badges show distinct "Gate"/"Security" labels per row):

`bug-hunter/evidence/runs-id-audit/BUG-20260828-audit-export-r2/01-before-export-menu-open.png`

- Downloaded CSV (all 100 rows category=gate, severity empty):

`bug-hunter/evidence/runs-id-audit/BUG-20260828-audit-export-r2/03-export.csv`

- Downloaded JSON (same defect, raw rows):

`bug-hunter/evidence/runs-id-audit/BUG-20260828-audit-export-r2/04-export.json`

- Repro notes and verification commands:

`bug-hunter/evidence/runs-id-audit/BUG-20260828-audit-export-r2/notes.md`

BROWSER SIGNALS

- Console: no errors on export click.
- Network: n/a — export is generated client-side from already-loaded row data (both CSV

and JSON downloads succeed with correct row COUNT — 100 — so this is a field-mapping defect, not a fetch failure).

- State/URL: stays on `/runs/{id}/audit` throughout; downloads complete normally, only the field content is wrong.

ID `BUG-20260828-audit-export-r2` Found by `bug-runs-id-audit-r2`

Fingerprint `/runs/{id}/audit|export-csv-json|click-export-then-csv-or-json|category-always-gate-severity-always-empty-contradicting-promised-columns`

SUMMARY

Clicking the "Inspect <card> details" button on any dashboard catalog card opens a dialog (role="dialog", e.g. "App Builder workflow", "Retry Loop") showing context providers, capabilities, and compaction info for that workflow. Pressing Escape while this dialog is focused does not close it — the dialog remains open and the page stays visually dimmed/blurred behind the backdrop. Only clicking the explicit "Close" (X) button dismisses it. This is a distinct component from the already-documented Advanced workflow modal (D-22 in DEFECTS-OBSERVED.md), which is reached via the "Advanced" button inside a launch panel/composer, not via a catalog card's Inspect affordance on the dashboard itself.

REPRODUCTION

1. Sign in as qa-admin and land on /dashboard.
 2. Click the "Inspect <card title> details" button on any catalog card (e.g. "Build an end-to-end application" or "Retry Until It Passes").
1. Observe the dialog opens (heading with workflow name, Context Providers / Capabilities / Compaction sections, an X close button top-right).
1. Press the Escape key.
 2. Observe the dialog is still present and the page is still dimmed behind it.
 3. Repeat steps 2-5 on a second, different card ("Retry Until It Passes" → "Retry Loop" dialog) — same result.

EXPECTED

Pressing Escape while a modal dialog is open should close it, matching standard modal dismissal behavior and the accessible-dialog convention already used elsewhere in the app.

ACTUAL

Escape has no effect; the dialog remains open indefinitely until the user explicitly clicks the "Close" (X) button.

EVIDENCE

- Before: bug-hunter/evidence/dashboard/BUG-20260827-221400-dashboard/01-before-inspect-open.png
- Failure (card 1, "App Builder"): bug-hunter/evidence/dashboard/BUG-20260827-221400-dashboard/02-escape-pressed-modal-stays-open.png
- Reproduction on card 2 ("Retry Loop"): bug-hunter/evidence/dashboard/BUG-20260827-221400-dashboard/03-reproduced-second-card.png

BROWSER SIGNALS

- Console: no relevant error observed.
- Network: no request involved; purely client-side dialog state.
- State/URL: URL stays /dashboard throughout; dialog is a client-rendered overlay, not a route change.

ID `BUG-20260827-221400-dashboard` Found by `bug-dashboard-r1`

Fingerprint `/dashboard|catalog-card-inspect-modal|press-escape|dialog-remains-open`

SUMMARY

Each saved-workflow card on /workflows has a button[aria-label='Workflow actions'] that opens

a small dropdown menu (Edit / Rename / Duplicate / Delete). The menu is correctly scoped per-card (opening it on "My presentation" only ever shows/affects "My presentation" — verified separately) and it DOES close correctly when the page's invisible outside-click overlay (`div.fixed.inset-0.z-10`, rendered behind the menu specifically to catch outside clicks) receives a genuine mouse click. However, pressing the **Escape** key while the menu is open has no effect at all: the trigger button's `aria-expanded` attribute stays `"true"` and the menu list stays visually present and interactive. This was verified with real Playwright keyboard events (`page.keyboard.press('Escape')`), not synthetic/no-op JS calls, and reproduced twice on two different cards ("My presentation" and "My prototype"). This is a distinct component and route from the already-filed [BUG-20260827-221400-dashboards](#) (Escape not closing the dashboard catalog card's "Inspect details" modal, a full `role="dialog"` overlay with Context Providers/Capabilities/Compaction sections on /dashboards) — here the affected control is a lightweight per-card action dropdown on the My Workflows list, reachable only via /workflows, with a completely different purpose (workflow management actions vs. a read-only info panel) and a different close mechanism (an outside-click overlay that itself works fine, unlike the dashboard modal's X-only close).

REPRODUCTION

1. Sign in as qa-admin, navigate to /workflows.
2. Locate the "My presentation" card and click its button[aria-label='Workflow actions'] (scoped via the card's unique title text, per the definition's `deleteWorkflow` recipe approach). Observe the menu opens: Edit, Rename, Duplicate, Delete, and the trigger

button's `aria-expanded` reads `"true"`.

1. Press the Escape key (`page.keyboard.press( 'Escape' )`, a genuine keyboard event).
2. Observe: the menu is still fully visible and interactive; `aria-expanded` on the trigger button

is still `"true"`.

1. For contrast, confirm the menu's own outside-click handling otherwise works: a genuine Playwright mouse click on the invisible `div.fixed.inset-0.z-10` overlay rendered behind the open menu correctly closes it (`aria-expanded` becomes `"false"`, no menu items visible).

1. Repeated steps 2-4 on a second, different card ("My prototype") — identical result: Escape has no effect, `aria-expanded` stays `"true"` after the key press.

EXPECTED

Pressing Escape while the "Workflow actions" dropdown is open should close it, consistent with standard menu/dropdown dismissal behavior and with the fact that this same menu already responds correctly to an outside click.

ACTUAL

Escape is not wired to the menu's close handler at all — the menu remains open indefinitely after Escape, and only an outside click (or presumably selecting a menu item) dismisses it.

EVIDENCE

- Before (menu open on "My presentation"): `bug-hunter/evidence/workflows/BUG-20260828-003300-workflows/01-before-menu-open.png`
- Failure (Escape pressed, menu still open): `bug-hunter/evidence/workflows/BUG-20260828-003300-workflows/02-failure-escape-pressed-menu-still-open.png`
- Contrast (a genuine outside click on the overlay does close it): `bug-hunter/evidence/workflows/BUG-20260828-003300-workflows/03-outside-click-does-correctly-close-menu.png`
- Reproduction 2, before (menu open on "My prototype"): `bug-hunter/evidence/workflows/BUG-20260828-003300-workflows/04-repro2-before-escape-my-prototype.png`
- Reproduction 2, after (Escape pressed, still open): `bug-hunter/evidence/workflows/BUG-20260828-003300-workflows/05-repro2-after-escape-still-open.png`

BROWSER SIGNALS

- Console: no relevant error observed.
- Network: no request involved; purely client-side dropdown/menu state.

- State/URL: URL stays /workflows throughout; aria-expanded on the button [aria-label='Workflow actions']

trigger read directly via DOM both before and after the Escape key press to confirm the state change (or lack thereof), not just a screenshot read.

ID `BUG-20260828-003300-workflows` Found by `bug-workflows-r1`

Fingerprint `/workflows|workflow-actions-dropdown-menu|press-escape-while-menu-open|menu-remains-open-aria-expanded-stays-true`

Saved-workflow detail view lists agents by raw internal slug instead of the friendly display name used everywhere else in the app

`/workflows/{id}`

SUMMARY

The read-only detail view for a saved workflow (`/workflows/{id}`) renders its agent roster by printing the raw `agent_ids` values verbatim — e.g. `documentation-agent`, `prototype-specify` — straight from `GET /api/user-workflows/{id}`, with no lookup against the agent catalog. Every other place in the app that displays these same two agents resolves them to a human-readable name and role/subtitle instead: the Library page shows "Documentation Agent" / "API & Technical Writing", and this exact saved workflow's own Composer canvas (`/workflows/{id}/edit`) shows "Documentation" / "API & Technical Writing" and "Spec Writer" / "Specification & Architecture" for the identical `documentation-agent` / `prototype-specify` ids. The one screen whose entire purpose is to summarize a saved workflow for a human reader is the one screen that fails to resolve the names, showing raw internal identifiers instead. Confirmed against the live API response (`agent_ids: ["documentation-agent", "prototype-specify"]`) so this is not a stale-data issue — it is the display component itself never mapping the id to a label, unlike its sibling composer view which clearly has that mapping available and uses it.

REPRODUCTION

1. Sign in as qa-admin, navigate directly to `http://localhost:3000/workflows/656ca387-e69c-474d-b7ff-5fd9eb017cc0` ("My prototype", a seeded saved workflow — deep-link only, per known D-28).
1. Observe the "2 AGENTS" list renders two rows reading exactly `documentation-agent` and `prototype-specify` — raw, hyphenated, lower-case internal ids, not sentence-case names.
1. Click "Edit" to open the same workflow's Composer (`/workflows/{id}/edit`). Observe the canvas nodes for the identical two agents show friendly labels: "Documentation" (subtitle "API & Technical Writing") and "Spec Writer" (subtitle "Specification & Architecture").

1. For a third data point, navigate to `/library` and locate the `documentation-agent` card — it displays "Documentation Agent" / "API & Technical Writing", confirming the friendly-name mapping is a standard, app-wide convention for this exact agent id, not something specific to the composer.

1. Cross-checked the live API directly: `GET /api/user-workflows/656ca387-e69c-474d-b7ff-5fd9eb017cc0`

returns `"agent_ids": ["documentation-agent", "prototype-specify"]` — confirming the detail

view is printing the raw field value with no name resolution, while its sibling views resolve the same ids to display names.

1. Reloaded `/workflows/656ca387-e69c-474d-b7ff-5fd9eb017cc0` a second time — identical raw-slug

rendering both times, deterministic.

EXPECTED

The saved-workflow detail view's agent roster should resolve each `agent_id` to the same human-readable display name (and ideally role subtitle) that the Library page and this workflow's own Composer/Edit view already show for the identical agents.

ACTUAL

The detail view prints the raw, internal `agent_id` strings verbatim (`documentation-agent`, `prototype-specify`) with no name resolution, while every other agent-listing surface in the app — including this exact workflow's own Edit page — shows friendly names for the same ids.

EVIDENCE

- Detail view showing raw agent slugs: `bug-hunter/evidence/workflows-id/BUG-20260828-004300-workflows-id/01-detail-raw-agent-slugs.png`
- Composer/Edit view for the same workflow showing friendly names for the same two agents: `bug-hunter/evidence/workflows-id/BUG-20260828-004300-workflows-id/02-composer-friendly-names-same-agents.png`

BROWSER SIGNALS

- Console: no relevant error observed.

- Network: GET `/api/user-workflows/656ca387-e69c-474d-b7ff-5fd9eb017cc0` returns

`agent_ids: ["documentation-agent", "prototype-specify"]` verbatim — matches what the detail

view renders unresolved, confirming the gap is in the detail view's rendering, not the API.

- State/URL: URL stays `/workflows/{id}` throughout; the same session's `/workflows/{id}/edit` navigation (no reload of underlying data, same auth/session) shows the resolved names, isolating the defect to this one component.

ID `BUG-20260828-004300-workflows-id` Found by `bug-workflows-id-r1`

Fingerprint `/workflows/{id}|agent-roster-list|render-saved-workflow-agent_ids|raw-agent-slug-shown-instead-of-friendly-name`

SUMMARY

On a fresh load of `/workflow` (the orphaned legacy builder, D-17's `/workflow`, unlinked from `routes.ts`), "Run Workflow" starts correctly disabled while "Workflow Agents (0)" and the brief textarea are both empty. Typing any non-empty text into "Describe your idea" — with the agent count still genuinely 0 (Workflow Agents (0)), matching D-17's finding that this page's Add Agent picker returns "No agents found" for every category, so 0 agents is not just the starting state but the only reachable state) — flips "Run Workflow" to enabled (`button.disabled === false`, confirmed via direct DOM read, not just a visual read). Clicking the now-enabled button does nothing observable: no navigation, no toast/alert text, no console error, and critically no network request at all — `browser_network_requests` shows no new request fired by the click (confirmed by diffing the request list immediately before and after, and by confirming `GET /api/runs?limit=50` count does not grow, i.e. no run was created). The gating logic only checks the brief text's presence, not the agent count it displays right next to it, so a workflow with literally zero configured agents presents as launchable and then quietly fails to launch with no feedback to the user at all.

REPRODUCTION

1. Sign in as qa-admin, navigate to `http://localhost:3000/workflow` (fresh load). Confirm "Workflow Agents (0)" and "Run Workflow" has the `disabled` attribute.
 1. Type any non-empty text into "Describe your idea" (e.g. `zz-hunt-test legacy builder idea text for probing run workflow button state`).
 1. Observe "Run Workflow" is no longer disabled (`button.disabled === false` via direct DOM read), while "Workflow Agents" still reads (0) — no agent was ever added.
 1. Click "Run Workflow". Observe: URL stays `/workflow`, no toast/alert appears, no console

message is logged, and `browser_network_requests` shows no new request fired by the click (the only requests present are the page's own initial load calls).

1. Reload to a fresh `/workflow`, repeat steps 2-4 with different brief text

(zz-hunt-repro2 second reproduction of run workflow enabled with zero agents) — identical

result: button enables on brief-text-only, click is a silent no-op, no request fires either time.

EXPECTED

"Run Workflow" should stay disabled (or clicking it should show a clear validation message, e.g. "Add at least one agent") while the workflow has zero configured agents, since a 0-agent workflow cannot meaningfully run. At minimum, if the button is going to be clickable, clicking it should do **something** observable (an error, a toast, a request) rather than nothing at all.

ACTUAL

The button's enabled/disabled state is gated only on the brief textarea having content, ignoring the agent count shown immediately beside it. With 0 agents it still enables, and clicking it is a complete no-op — no request, no error, no feedback of any kind.

EVIDENCE

- Before (fresh page, 0 agents, button disabled): `bug-hunter/evidence/workflow/BUG-20260828-011000-workflow/01-before-brief-typed-enabled.png`
- Repro 2 (fresh reload, brief typed, button enabled with 0 agents): `bug-hunter/evidence/workflow/BUG-20260828-011000-workflow/02-repro2-enabled-zero-agents.png`
- After click (no visible change, no request fired): `bug-hunter/evidence/workflow/BUG-20260828-011000-workflow/03-after-click-silent-noop.png`

BROWSER SIGNALS

- Console: no relevant error or message logged on click, in either reproduction.
- Network: no new request of any kind fires from the click; `GET /api/runs?limit=50` count

confirmed unchanged (no run created) — verified via `browser_network_requests` diff before/after.

- State/URL: URL stays `/workflow` throughout; `button.disabled` verified `false` via direct DOM

read before each click, not inferred from a screenshot.

ID `BUG-20260828-011000-workflow` Found by `bug-workflow-r1`

Fingerprint `/workflow|run-workflow-button|type-brief-with-zero-agents-attached|button-enables-but-click-fires-no-request-and-does-nothing`

76 **LOW** Skill with an empty `category` field renders a blank category label and is unreachable through any category pill

`/library?tab=skills`

SUMMARY

GET `/api/skills/library` returns 186 skills; `total_count` (186), the raw array length (186), and the rendered card count (`document.querySelectorAll('div.cursor-pointer.p-4').length = 186`)

all agree — no cap or pagination gap at this scale (unlike the Run History 50/273 cap). The nine category pills (Collaboration, Creative, Debugging, Planning, Research, Security, Specialist, Testing, Workflow) are correctly derived from the categories actually present in the skill data, and clicking each pill returns exactly the count of skills carrying that category value (verified for Research: 2/2, and Testing: 48/48) — so this is NOT the same mechanism as the known D-26 (agents whose pills are built from an entirely different, mismatched source list, orphaning 52 of 93 agents). Here, exactly one skill record — `html-deck-to-pptx` (id `html-deck-to-pptx`) — has `category: ""` (empty string) in the API response, and also `tags: []`. Its card renders in the grid with a visibly blank category-badge line (the `<p class="...uppercase tracking-[0.1em] font-semibold capitalize">` element that shows "TESTING"/"PLANNING"/etc. on every other card is present but empty) and no tag chips. Because no category pill's filter value is an empty string, this single skill can never be reached by clicking any specific category pill — only "All" (or search by name) surfaces it.

REPRODUCTION

1. Sign in as qa-admin, navigate to `http://localhost:3000/library?tab=skills` (All pill, 186 cards).

1. Confirm scale integrity: `document.querySelectorAll('div.cursor-pointer.p-4').length = 186;`

GET `/api/skills/library` response `total_count = 186` and `skills.length = 186` — all three

agree, no pagination/cap defect at this page.

1. In that same API response, find the skill with category: `""` — `html-deck-to-pptx` ("Turn a finished HTML slide deck into a real, editable PowerPoint file with PptxGenJS...").

1. On the "All" pill, use `browser_find` for `"html-deck-to-pptx"` — the card is present. Inspect its DOM: the category-badge `<p>` under the title is empty (`""`), unlike every neighboring card which shows its category in caps (e.g. `"PLANNING"`, `"TESTING"`).

1. Click the "Testing" category pill (URL becomes `?category=testing`). Card count is 48 (matches the 48 skills whose `category === "testing"` in the API data exactly). Search `document.body.innerText` for `"html-deck-to-pptx"` — not found; confirmed absent.

1. Repeat against the "Research" pill (2/2 cards match exactly) for a second confirmation that pill filtering itself works correctly and is not the cause — the gap is specific to this one skill's missing category value.

1. Return to the "All" pill — the card reappears, confirming this is a stable, reproducible, category-specific reachability gap tied to the skill's own data, not a rendering race.

EXPECTED

Every skill should have a non-empty category so its card shows a real category badge and the skill is reachable through the category pill that corresponds to it, consistent with every other skill in the library.

ACTUAL

The `html-deck-to-pptx` skill has an empty category value, so its card renders a blank category-badge line and the skill is permanently unreachable through any of the nine category pills — only visible via "All" or by searching its name directly.

EVIDENCE

- Before (card visible under "All", scrolled into view): `bug-hunter/evidence/library-skills/BUG-20260828-021300-library-skills/01-before-card-visible-in-all.png`
- Failure (card's category-badge line is blank): `bug-hunter/evidence/library-skills/BUG-20260828-021300-library-skills/02-failure-blank-category-label.png`

- Unreachable via a specific pill ("Testing" selected, 48/48 other cards shown, this one absent): bug-hunter/evidence/library-skills/BUG-20260828-021300-library-skills/03-unreachable-via-testing-pill.png

BROWSER SIGNALS

- Console: no relevant error observed.
- Network: GET `http://localhost:8000/api/skills/library` returns 200 OK; response body

confirms `total_count: 186`, `skills.length: 186`, and exactly one entry

(`id: "html-deck-to-pptx"`) with `category: ""` and `tags: []`.

- State/URL: `?category=<slug>` query param updates correctly per pill click; the affected skill's card is absent under every specific `category=` value and present only under "All"/unfiltered.

ID `BUG-20260828-021300-library-skills` Found by `bug-library-skills-r1`

Fingerprint `/library?tab=skills|skill-card-category-badge-and-category-pill-filter|render-card-for-skill-with-empty-category-field|blank-category-label-and-card-unreachable-by-any-specific-pill`

Hook detail page's "Copy" button gives zero feedback that a copy occurred

`/library/hooks/<hook-id>` (e.g. `/library/hooks/post-design-quality`, `/library/hooks/post-quality-gate`)

SUMMARY

Clicking a hook card in the Library's Hooks tab performs a genuine client-side route navigation to `/library/hooks/<hook-id>` (a real page, not a modal), which renders the full hook detail (event, trigger, "how to use" steps, compatible agents, tags) plus a "Copy" button in the header next to the close (X) button. Clicking "Copy" (presumably intended to copy the hook's markdown content field, which the GET `/api/hooks/library` response carries per-hook, to the clipboard) produces absolutely no observable feedback: the button's text stays exactly "Copy" (never "Copied!" or similar), its icon stays the same `lucide-copy` glyph (never swaps to a checkmark), no toast/alert appears anywhere on the page, and `document.body.innerText` shows no new text after the click. The only DOM change is a transient `:active` pseudo-state from the click itself. A user has no way to tell, from the UI alone, whether the copy succeeded, failed, or did anything at all. This reproduces identically on two different hooks (`post-design-quality` and `post-quality-gate`), so it is the button component's own behavior, not a per-hook data issue. This is a distinct control and mechanism from the ledger's other no-feedback findings: the composer's documented "Run once" quirk (a run-launch action), and `BUG-20260828-011000-workflow` (a "Run Workflow" button that is a complete no-op with zero network activity on the orphaned `/workflow` legacy builder). Here the button is a client-side clipboard-copy action on a completely different page (the new hook detail route), and the underlying action may well succeed (browser clipboard permissions blocked a direct read-back verification in this sandboxed session) — the defect is specifically the total absence of any success/failure UI feedback for an action whose entire purpose is a one-shot, silent side effect the user cannot otherwise verify.

REPRODUCTION

1. Sign in as qa-admin, navigate to `http://localhost:3000/library?tab=hooks`.

2. Click the "Design Quality Check X" hook card — the app navigates to

`/library/hooks/post-design-quality`, rendering the full hook detail panel with a "Copy"

button (lucide copy icon + "Copy" label) in the top-right of the detail header.

1. Click "Copy". Observe: the button's accessible name and rendered text remain "Copy", its

icon/outerHTML is unchanged aside from a transient `:active` state, and no toast, alert, or

any new text appears anywhere in `document.body.innerText`.

1. Repeat on a second, different hook: navigate to `/library/hooks/post-quality-gate`, click its

"Copy" button — identical result: button still reads "Copy", no confirmation of any kind.

EXPECTED

Clicking "Copy" should give the user some positive confirmation that the copy succeeded (e.g. the

label briefly changing to "Copied!", the icon swapping to a checkmark, or a toast), consistent

with standard copy-to-clipboard UX and with how a user is expected to trust that a silent,

one-shot action actually happened.

ACTUAL

The button gives no feedback whatsoever after being clicked — same label, same icon, no toast —

on every hook tested, leaving the user with no way to confirm the copy occurred.

EVIDENCE

- Before (Design Quality Check X detail, Copy button unclicked): `bug-hunter/evidence/library-hooks/BUG-20260828-025500-library-hooks/01-before-copy-click.png`
- After click, no visible change: `bug-hunter/evidence/library-hooks/BUG-20260828-025500-library-hooks/02-after-copy-click-no-change.png`
- Repro 2 (Quality Gate detail), before: `bug-hunter/evidence/library-hooks/BUG-20260828-025500-library-hooks/03-repro2-before-copy.png`
- Repro 2, after click, no visible change: `bug-hunter/evidence/library-hooks/BUG-20260828-025500-library-hooks/04-repro2-after-copy-no-change.png`

BROWSER SIGNALS

- Console: no relevant error observed.
- Network: no request involved; this is a pure client-side clipboard action with no accompanying

API call.

- State/URL: URL stays on `/library/hooks/<hook-id>` throughout; `button.outerHTML` read directly

via DOM confirmed identical text/icon before and after the click in both reproductions.

ID `BUG-20260828-025500-library-hooks`

Found by `bug-library-hooks-r2`

Fingerprint `/library/hooks/<id>|hook-detail-copy-button|click-copy-button|no-visual-state-change-no-toast-no-confirmation`

File explorer search leaves orphaned empty folders visible and the footer file count never reflects the active filter

`/preview-fullscreen (ready state, valid `__app_preview__` payload)`

SUMMARY

In `frontend/src/components/preview/AppBuilderPreview.tsx`, the file-tree search (`FileTreeNode`) filters only leaf **files** by `searchQuery` (`if (!matchesSearch) return null;` inside the file branch) — the folder branch has no equivalent check and is always rendered regardless of whether any of its descendants match. Consequence: typing a search query that matches a file nested two or more levels deep (e.g. `src/utils/helper.js`) leaves every sibling folder that contains *no* matching files (e.g. `src/styles/`) still visible in the tree and still clickable/expandable — expanding it renders a folder header with literally nothing underneath, a dead-end empty branch the search should have pruned. Separately, the sidebar footer stat line ("N files · X KB total") is computed from the full unfiltered `files` array and never reflects the active search — it continues to read the original total file count even while the visible list is filtered down to a single match, so the UI's own count contradicts what's on screen.

REPRODUCTION

1. Sign in as qa-admin. Seed a realistic `__app_preview__` payload (shape read directly from `frontend/src/app/preview-fullscreen/page.tsx`'s `PreviewPayload` interface) via `sessionStorage.setItem('__app_preview__', JSON.stringify({projectName: 'SearchTestProject', files: [{path: 'index.html', ...}, {path: 'src/app.js', ...}, {path: 'src/styles/main.css', ...}, {path: 'src/utils/helper.js', ...}]}))`, then navigate to `http://localhost:3000/preview-fullscreen`.

The ready-state IDE preview renders correctly (title "SearchTestProject — IDE Preview", 4-file tree, `index.html` content shown) — noted here as the legitimate route taken since this round found no app-builder run in `/runs` history with a live, non-expired preview

payload; the payload shape matches the interface the page itself defines.

1. Expand the nested `src/styles` and `src/utls` subfolders (they are not auto-expanded; only top-level `src` is).

1. Type `app` into the "Search files" box. Observe: `index.html` (non-matching) correctly disappears, `app.js` (matching) correctly remains — but the `styles` and `utls` folder rows also remain visible even though neither contains a file matching "app". Expanding `styles` confirms it renders with zero children underneath. The footer still reads "4 files · 0.0 KB total" despite only one file (`app.js`) being visible.

1. Reproduced with a second, independent query: cleared and typed `helper` instead. Observe `helper.js` (the only match, inside `utls`) correctly shows, but `styles` (no match) still renders as a folder row and expands to nothing; the footer is still stuck at "4 files · 0.0 KB total" regardless of the query.

EXPECTED

A folder with zero descendants matching the active search query should be hidden along with its non-matching files (mirroring how the leaf-file filter already behaves), so an expanded search result never shows an empty folder. The footer file/size summary should reflect the currently visible (filtered) file set, not the unfiltered project total, while a search is active.

ACTUAL

Folders are never filtered by search match state — any folder that contains at least one non-matching file (even if it contains zero matching files) stays fully visible and expandable, rendering an empty branch when opened. The footer count is hardcoded to the full unfiltered `files` array and never updates to reflect the active filter, contradicting the visibly reduced file list.

EVIDENCE

- Before (ready state, full unfiltered tree, no search active): [bug-hunter/evidence/preview-fullscreen/BUG-20260828-041300-preview-fullscreen/01-before-search-empty.png](#)

- Failure (app search — styles/utils folders remain despite no matches, footer still "4 files"): bug-hunter/evidence/preview-fullscreen/BUG-20260828-041300-preview-fullscreen/02-search-app-orphaned-folders.png
- Expanding styles under the app search shows nothing inside: bug-hunter/evidence/preview-fullscreen/BUG-20260828-041300-preview-fullscreen/03-styles-folder-expanded-shows-nothing.png
- Reproduced with a second, independent query (helper), same defect: bug-hunter/evidence/preview-fullscreen/BUG-20260828-041300-preview-fullscreen/04-repro2-helper-search-same-defect.png

BROWSER SIGNALS

- Console: no relevant error observed.
- Network: none — filtering is purely client-side state in AppBuilderPreview.
- State/URL: URL stays on /preview-fullscreen throughout; confirmed via browser_snapshot DOM

reads that folder rows persist and expand empty under an active non-matching search, and that the footer paragraph text never changes from the unfiltered total across two independent queries.

ID BUG-20260828-041300-preview-fullscreen Found by bug-preview-fullscreen-r1

Fingerprint /preview-fullscreen|app-builder-preview-file-explorer-search|type-query-matching-only-a-nested-file|parent-folders-with-zero-matching-children-remain-visible-and-expand-empty-while-footer-count-stays-unfiltered

Failed-login error message renders with no ARIA live region or alert role, so screen readers never announce it

`/login?expired=1` (identical DOM structure to plain `/login`)

SUMMARY

Submitting the sign-in form on `/login?expired=1` with invalid credentials correctly renders a visible "Invalid email or password." message, but the element carries no `role="alert"`, `role="status"`, or `aria-live` attribute (own or inherited), and focus is not moved to it. The page's only accessible-tree "alert" node is Next.js's built-in `next-route-announcer` (a shadow-DOM element used only for route-change titles), which stays empty and is unrelated to this error. A screen-reader user who submits wrong credentials gets no notification that anything happened — the form simply appears to sit still, since neither the visual state change nor keyboard focus alerts them to the new error text. This is reproducible on `/login` generally (same component), but is being filed against `/login?expired=1` since that is the assigned route and its DOM is otherwise identical.

REPRODUCTION

1. Ensure signed out (`localStorage.clear(); sessionStorage.clear()`), navigate to `http://localhost:3000/login?expired=1`.
1. Fill the Email field with an invalid address (e.g. `wrong@flowinqa.com`) and the Password field with any wrong password.
1. Click "Sign in" (or press Enter from the password field).
2. Observe the DOM: a new `<div>` containing "Invalid email or password." appears above the form, but `getAttribute('role')` and `getAttribute('aria-live')` are both null on that element and on every ancestor up to `<body>`. `document.activeElement` remains on `<body>` (no focus is moved to the message or back to a field).
1. Repeated with a second, different pair of invalid credentials — same result both times.

EXPECTED

A programmatically-determinable status message (via `role="alert"`, `role="status"`, or `aria-live="polite"/"assertive"`) should be present so assistive technology announces the authentication failure, per WCAG 4.1.3 (Status Messages). Alternatively, focus should move to the error text.

ACTUAL

The error text is inserted as a plain, non-live `<div>` with no role. Screen-reader users receive no notification of the failed login; the failure is only conveyed visually.

EVIDENCE

- Before (empty form, no error): `bug-hunter/evidence/login-expired-1/BUG-20260828-041815-login-expired-1/01-before-empty-form.png`
- Failure (error shown, confirmed via evaluate to have no role/aria-live): `bug-hunter/evidence/login-expired-1/BUG-20260828-041815-login-expired-1/02-failure-error-shown-no-live-region.png`

BROWSER SIGNALS

- Console: none relevant (one benign 401 network-error console entry from the failed login request itself, expected).
- Network: `POST /api/auth/login` returns 401 as expected; not itself a defect.
- State/URL: URL stays `/login?expired=1` throughout; `document.activeElement` stays on `<body>` after the error renders, confirming no focus management either.

ID `BUG-20260828-041815-login-expired-1` Found by `bug-login-expired-1-r2`

Fingerprint `/login?expired=1|sign-in-error-message|submit-with-invalid-credentials|error-text-not-announced-to-assistive-tech`

80 **LOW** **Bare `/settings` redirects correctly on a cold/full load but renders the honest 404 page when reached via client-side (SPA) navigation**
`/settings`

SUMMARY

A full/cold browser navigation (typing the URL, or a fetch/reload) to bare `/settings` receives a server-level redirect to `/settings/profile` and renders correctly (this is the already-known, intentional-per-quirk-notes behaviour, C-2). However, when the exact same path is reached via client-side SPA routing (a `pushState` + `popstate` transition, which is what any in-app `navigate('/settings')` call or React Router `<Link to="/settings">` would trigger) the app's client router does not replicate that redirect. It instead falls through to the honest, fully-rendered 404 "Page not found" screen, with the URL bar frozen on `/settings`. Confirmed via `fetch('/settings')` that the redirect is applied at the network/server layer (`response.redirected === true`, final `response.url` is `/settings/profile`) — the client router has no equivalent redirect rule of its own, so any client-side transition to this path dead-ends at 404 instead of landing the user on their profile settings the way a fresh page load does.

REPRODUCTION

1. Sign in as qa-admin (or any tier — reproduced identically on qa-basic and qa-admin), land on `/dashboard`.

1. Full navigation to `/settings/profile` (baseline, confirms cold load redirect works).
2. In the browser console context, run `window.history.pushState({}, '', '/settings');`
`window.dispatchEvent(new PopStateEvent('popstate'))` — this simulates the exact history transition any in-app client-side `navigate('/settings')` call would perform.

1. Observe the URL stays `/settings` and the page renders the full "Page not found" 404 screen instead of redirecting to `/settings/profile`.

1. Reload the same `/settings` URL directly (full navigation) — it correctly redirects to

/settings/profile, proving the redirect logic exists but is not reachable from client-side routing.

1. Repeated on both qa-basic and qa-admin accounts with identical results (2 independent reproductions plus a confirmation on both tiers).

EXPECTED

A client-side transition to /settings should apply the same redirect-to-/settings/profile behaviour as a cold load, since it is the same route being resolved by the same app — the destination should not depend on how the URL was reached.

ACTUAL

Client-side navigation to /settings renders the honest 404 "Page not found" page and leaves the URL frozen on /settings, while a full/cold navigation to the identical path redirects successfully to /settings/profile.

EVIDENCE

- Before (cold load, correct redirect to /settings/profile): bug-hunter/evidence/settings/BUG-20260828-042311-settings/01-before-cold-load-redirects-correctly.png
- Failure (client-side nav, 404 instead of redirect): bug-hunter/evidence/settings/BUG-20260828-042311-settings/02-failure-clientside-nav-shows-404.png
- Repro 2 (same failure on qa-admin account): bug-hunter/evidence/settings/BUG-20260828-042311-settings/03-repro2-admin-account-same-404.png

BROWSER SIGNALS

- Console: none relevant.
- Network: `fetch('http://localhost:3000/settings')` returns `redirected: true, final url`

`http://localhost:3000/settings/profile` — confirming the redirect is server/network-layer only, with no corresponding client-router rule. No network request fires at all during the client-side popstate transition, confirming the 404 is a pure client-side render decision.

- State/URL: URL stays /settings throughout the client-side-nav failure case (never becomes /settings/profile), distinct from the full-load case where the URL bar itself changes to /settings/profile.

ID `BUG-20260828-042311-settings` Found by `bug-settings-r2`

Fingerprint `/settings|bare-route-router|clientside-navigation-to-settings|shows-404-instead-of-redirect-that-fullload-applies`

SUMMARY

On the Password card of /settings/profile, both "Current password" and "New password" inputs render an eye-icon button that toggles the field between masked (`type="password"`) and plain text (`type="text"`), letting the user verify what they typed. The adjacent "Confirm new password" input — functionally identical, and whose entire purpose is to let the user verify they retyped the new password correctly — has no such button and can never be revealed. Source inspection confirms the cause: `AccountSettings.tsx` only declares `showCurrent/showNew` state (lines 96-97) and wires a toggle button to each of the first two inputs (lines 283-300); the `confirm-password` input (from line ~305) has no matching `showConfirm` state or button at all.

REPRODUCTION

1. Sign in as qa-admin, navigate to /settings/profile.
 2. Click into "Current password", type any value; click into "New password", type any value; click into "Confirm new password", type any value.
1. Observe: "Current password" and "New password" each show a small eye/eye-off icon button on the right edge of the input. "Confirm new password" shows no icon.
1. Click the eye icon on "Current password" and on "New password" — each toggles that single input's type between password and text independently (confirmed via `DOM input.type`). No equivalent control exists for "Confirm new password" — it remains masked with no way to reveal it.
1. Reload to a fresh /settings/profile and repeat steps 2-4 — same result: `confirmHasToggle` evaluates `false` via `document.querySelector('input[placeholder="Re-enter new password"] ~ button')` while `currentHasToggle / newHasToggle` evaluate `true`, on both passes.

EXPECTED

All three password inputs in the same form should offer the same show/hide affordance, or at minimum the confirm field (whose entire purpose is letting the user verify their retyped value) should be revealable like its siblings.

ACTUAL

"Current password" and "New password" have a working eye-icon visibility toggle; "Confirm new password" has none and can never be shown in plain text.

EVIDENCE

- Before: bug-hunter/evidence/settings-profile/BUG-20260828-050900-settings-profile/01-before-empty-form.png
- Failure: bug-hunter/evidence/settings-profile/BUG-20260828-050900-settings-profile/02-confirm-field-no-toggle.png

(Current and New password revealed as plain text via their toggles; Confirm new password still masked with no icon present.)

BROWSER SIGNALS

- Console: none observed related to this defect.
- Network: none — this is a client-render-only defect, no request involved.
- State/URL: URL stays /settings/profile throughout; confirmed via direct DOM query

(input[placeholder="Re-enter new password"] ~ button → null) that no toggle button is attached to the confirm-password input, on two independent page loads.

ID `BUG-20260828-050900-settings-profile` Found by `bug-settings-profile-r2`

Fingerprint `/settings/profile|change-password-form|inspect-confirm-new-password-field|missing-visibility-toggle-inconsistent-with-current-and-new-password-fields`

82 **LOW** **"Upload custom template → From URL" leaks a raw Python errno/socket message to the user on an unreachable domain**
/create/prototype

SUMMARY

On the Template tab's "Upload custom" → "From URL" flow, entering a syntactically valid but unreachable URL (a DNS name that cannot resolve) and clicking "Fetch" surfaces the backend's raw Python socket exception text verbatim as the on-page error message: `URL fetch failed: [Errno 8] nodename nor servname provided, or not known`. This is a distinct, better-behaved code path than

a malformed URL (e.g. `not-a-valid-url`), which correctly shows a friendly, app-authored message ("URL must start with `http://` or `https://`") — proving the modal has a friendly-message path and simply falls back to dumping the raw backend exception string when the failure originates from the network fetch itself rather than client-side validation. GET

`/api/prototype/fetch-url?url=...` returns 502 Bad Gateway with body

`{"detail": "URL fetch failed: [Errno 8] nodename nor servname provided, or not known"}`, and the

frontend renders that `detail` string unmodified. Reproduced deterministically with two different unreachable domains, both producing byte-identical error text. This is a distinct mechanism from the already-filed raw-backend-error bugs on this ledger (the `/handoff/settings` PAT length-422 echoing the submitted secret value, and `/create` "Save as my version" 422 manifest-validation text) — different route, different component (URL-fetch, not save/validation), and a different class of leaked internal detail (an OS-level socket errno, not a Pydantic validation payload).

REPRODUCTION

1. Sign in as qa-admin, navigate to `http://localhost:3000/create/prototype`, Template tab.
2. Click "Upload custom" → "From URL" tab in the "Upload custom template" modal.
3. Type `http://example.invalid.nonexistent-domain-xyz123/` into the Template URL field, click

"Fetch". Observe: GET `/api/prototype/fetch-url?...` returns 502 Bad Gateway; the modal

displays "URL fetch failed: [Errno 8] nodename nor servname provided, or not known" as the inline error, verbatim from the backend response body.

1. Cleared the field and repeated with a second, different unreachable domain

(`https://another-totally-fake-domain-abc987.invalid/`) — identical 502 and byte-identical raw errno text rendered on-page, confirming deterministic reproduction.

1. Contrast: typing a malformed non-URL string (`not-a-valid-url`) and clicking "Fetch" instead shows a friendly, app-authored validation message ("URL must start with `http://` or `https://`"), proving the friendly-message path exists and is simply bypassed for network-fetch failures.

1. Cleaned up: clicked "Cancel" to close the modal; no template was uploaded/saved; wizard left in a normal, clean state.

EXPECTED

A network-level fetch failure (unresolvable domain, connection refused, timeout, etc.) should be translated into a friendly, human-readable message (e.g. "Could not reach that URL — check the address and try again"), consistent with how the modal already handles client-side URL-format validation, never a raw OS/Python exception string surfaced to the end user.

ACTUAL

The modal renders the backend's raw internal exception text unmodified: "URL fetch failed: [Errno 8] nodename nor servname provided, or not known" — an implementation-detail string that leaks the backend's runtime/OS and gives the user no actionable guidance.

EVIDENCE

- Before (Upload custom template modal open, From URL tab): `bug-hunter/evidence/create-prototype/BUG-20260828-053300-create-prototype-r2/01-before-upload-modal-open.png`
- Contrast (malformed URL shows friendly app-authored error): `bug-hunter/evidence/create-prototype/BUG-20260828-053300-create-prototype-r2/02-malformed-url-friendly-error.png`
- Failure (unreachable domain #1, raw errno text rendered): `bug-hunter/evidence/create-prototype/BUG-20260828-053300-create-prototype-r2/03-failure-raw-errno-leaked.png`

- Second reproduction (unreachable domain #2, byte-identical raw errno text): bug-hunter/evidence/create-prototype/BUG-20260828-053300-create-prototype-r2/04-repro2-raw-errno-leaked.png
- Network log: bug-hunter/evidence/create-prototype/BUG-20260828-053300-create-prototype-r2/network.log

BROWSER SIGNALS

- Console: Failed to load resource: the server responded with a status of 502 (Bad Gateway) @

http://localhost:8000/api/prototype/fetch-url?url=... on each unreachable-domain attempt.

- Network: GET /api/prototype/fetch-url?url=... → 502 Bad Gateway, response body

{"detail":"URL fetch failed: [Errno 8] nodename nor servname provided, or not known"} on both

trials, byte-identical.

- State/URL: stays on /create/prototype throughout; modal remains open after the failed fetch;

cleanly cancelled with no residual state.

ID BUG-20260828-053300-create-prototype-r2 Found by bug-create-prototype-r2

Fingerprint /create/prototype|upload-custom-template-from-url|fetch-unreachable-domain|raw-backend-errno-string-rendered-as-error

SUMMARY

The PPT built-in's real deliverable (GET `/api/workflows/ppt` → `deliverable: {"strategy": "ppt", "name": "presentation.pptx"}`) is neither of the app's three generic deliverable strategies. The Canvas view's "Workflow" tab renders this by silently defaulting its combobox to "Streamed text — agent's raw output" (already documented as part of the manifest-loss bug `BUG-20260828-005700-workflows-ppt-canvas`, filed there in the context of "Save as copy"). The Simple view, looking at the exact same unsaved built-in state, does not use that combobox at all — it renders a read-only "Deliverable type: Custom" label instead. So switching between the two tabs on the identical, unmodified workflow shows the user two different, mutually contradictory answers to "what does this workflow deliver" — "Streamed text" on one tab, "Custom" on the other — and neither matches the true `ppt/presentation.pptx` strategy. This is a distinct defect from the filed one: it is a display inconsistency between the Simple and Canvas components themselves, independent of any save action, and was never previously investigated (round 1 only looked at the Canvas tab and the save-as-copy POST body).

REPRODUCTION

1. Sign in as qa-admin, navigate to `http://localhost:3000/workflows/ppt/canvas` (fresh load, lands on Canvas tab by default).
1. On the Canvas tab, open the right-rail "Workflow" sub-tab (default) and note the "Deliverable strategy" combobox: selected option is "Streamed text — agent's raw output".
1. Click the "Simple" tab (top toolbar) — no save action, same in-memory workflow. Note the "Deliverable type" field under the "Workflow" card: reads "Custom" (not "Streamed text").
1. Click back to "Canvas" — combobox reverts to showing "Streamed text — agent's raw output"

selected.

1. Reload the page cold (<http://localhost:3000/workflows/ppt/canvas>) and click "Simple" directly without visiting Canvas first: still reads "Custom" — reproduced from a clean load, not just after tab-hopping.

1. Cross-check ground truth via GET `/api/workflows/ppt:deliverable.strategy` is "ppt", `deliverable.name` is "presentation.pptx" — neither UI label ("Streamed text" nor "Custom") matches it.

EXPECTED

The two views of the same workflow's configuration should agree with each other (and ideally reflect the real ppt strategy, or at minimum both fall back to the same honest label, e.g. both say "Custom" or both are disabled/blank) when a strategy value doesn't match any of the generic options the UI understands.

ACTUAL

Canvas shows a specific, wrong, selectable option ("Streamed text — agent's raw output") while Simple shows a different, generic label ("Custom") for the exact same unsaved, unmodified built-in workflow state — the two tabs contradict each other with no save/reload between them.

EVIDENCE

- Canvas tab, Workflow sub-tab, Deliverable strategy combobox showing "Streamed text — agent's raw output" selected: <bug-hunter/evidence/workflows-ppt-canvas/BUG-20260828-073350-workflows-ppt-canvas/01-canvas-shows-streamed-text.png>
- Simple tab, same unsaved workflow, Deliverable type showing "Custom": <bug-hunter/evidence/workflows-ppt-canvas/BUG-20260828-073350-workflows-ppt-canvas/02-simple-shows-custom.png>

BROWSER SIGNALS

- Console: no relevant error observed.
- Network: no requests fired by switching tabs (GET `/api/workflows/ppt` only on initial page load); `deliverable.strategy` confirmed "ppt" via direct API call, matching neither UI label.
- State/URL: URL stays `/workflows/ppt/canvas` throughout; purely a client-side rendering disagreement between the Simple and Canvas components reading the same in-memory workflow state.

ID `BUG-20260828-073350-workflows-ppt-canvas` Found by `bug-workflows-ppt-canvas-r2`

Fingerprint `/workflows/ppt/canvas|simple-vs-canvas-deliverable-display|switch-between-simple-and-canvas-tabs|deliverable-type-label-disagrees-custom-vs-streamed-text`

SUMMARY

On `/workflow`, clicking "Attach file" and selecting a file injects a literal

`[Attached: <filename>]` marker string directly into the "Describe your idea" textarea's value

(not just a visual chip) and shows a removable attachment chip below the field. Clicking the

chip's own "x" remove button correctly removes the visual chip from the DOM, but does **not**

clear the `[Attached: ...]` marker text it had injected into the textarea — `textarea.value`

(confirmed via direct DOM read, not just a visual screenshot) still literally reads

`[Attached: zz-hunt-test.txt]` after the file is "removed", with no attachment chip anywhere in

the page and no way to see this stale content since the textarea's placeholder styling makes the

leftover marker easy to miss. Because "Run Workflow"'s only gate is non-empty brief text (already

established as the root cause of the separately-filed zero-agent no-op bug), the button stays

enabled based on this now-orphaned marker even though the user explicitly removed the attachment

and the workflow still has 0 agents. This is a distinct defect from the already-filed

`BUG-20260828-011000-workflow`: that bug is about the enable-gate ignoring agent count on

legitimately-entered text; this one is about the remove-attachment action failing to fully

reverse the state it created, leaving stale/misleading content behind after an explicit user

undo action.

REPRODUCTION

1. Sign in as qa-admin, navigate to `http://localhost:3000/workflow` (fresh load). Confirm the textarea is empty and "Run Workflow" is disabled.

1. Click "Attach file (PDF, DOCX, PPTX, TXT)" and select a small `.txt` file (e.g.

`zz-hunt-test.txt`). Observe: the textarea now contains `[Attached: zz-hunt-test.txt]`

(confirmed via `textarea.value`), an attachment chip `zz-hunt-test.txt (32B)` appears below the

field, and "Run Workflow" becomes enabled.

1. Click the "x" button on the attachment chip to remove it. Observe: the chip disappears from the page — `document.body.innerText` no longer shows the filename outside the marker text.

1. Read `textarea.value` directly via DOM: it still equals `[Attached: zz-hunt-test.txt]`, and `runButton.disabled` is still `false` — the button remains enabled for a workflow with 0 agents and no attachment, based purely on the orphaned marker text the remove action failed to clear.

1. Reloaded to a fresh `/workflow` and repeated steps 2-4 — identical result: stale `[Attached: zz-hunt-test.txt]` marker persists in the textarea and "Run Workflow" stays enabled after the chip is removed.

EXPECTED

Removing an attachment via its chip's "x" button should fully reverse the attach action: the `[Attached: ...]` marker text it injected into the textarea should also be cleared (or, if the textarea legitimately had other user-typed content, only the marker segment should be stripped), and "Run Workflow" should return to reflecting the workflow's real state (0 agents, no attachment, effectively empty brief).

ACTUAL

The chip's remove control only deletes the chip element from the DOM/attachment list; the `[Attached: <filename>]` string it wrote into the textarea's actual value is left behind untouched, so the brief field is left in a misleading state (claims an attachment that no longer exists) and "Run Workflow" stays enabled off that stale text.

EVIDENCE

- Before (fresh load, empty textarea, Run Workflow disabled): `bug-hunter/evidence/workflow/BUG-20260828-074900-workflow-r2/01-before-fresh-load.png`
- Repro 1 (attachment removed, chip gone, stale marker still in textarea, Run Workflow enabled): `bug-hunter/evidence/workflow/BUG-20260828-074900-workflow-r2/02-repro1-stale-marker-after-remove.png`

- Repro 2 (fresh reload, identical result): `bug-hunter/evidence/workflow/BUG-20260828-074900-workflow-r2/03-repro2-stale-marker-after-remove.png`

BROWSER SIGNALS

- Console: no error or warning logged on attach, remove, or the resulting stale state.
- Network: no request fired by the attach or remove actions (purely client-side textarea/state

manipulation); confirmed via `browser_network_requests` diffing before/after both actions.

- State/URL: `location.href` stays `/workflow` throughout; direct DOM reads of `textarea.value` and `runButton.disabled` (not just visual/screenshot inspection) confirm the stale marker text and enabled state, reproduced twice from independent fresh page loads.

ID `BUG-20260828-074900-workflow-r2` Found by `bug-workflow-r2`

Fingerprint `/workflow|attach-file-remove-attachment|click-remove-x-on-attachment-chip|stale-attached-marker-text-remains-in-textarea-run-button-stays-enabled`

Tool-call summary row renders "[object Object]" instead of a readable preview for array/object arguments

/runs/b9feac1c-ec21-4531-8ba7-bb391786993e/steps

SUMMARY

On the Steps tab, expanding an agent row's "Tool calls" panel shows a one-line collapsed summary for each call (`<tool_name> <args preview> ok`). When a tool's argument value is an array of objects (the `write_todos` tool's `todos` array, an array of `{content, status}` items), the collapsed summary literally renders `todos: [object Object], [object` — the raw result of JavaScript's default `Array.prototype.toString()`/string-coercion applied directly to the object array, instead of a readable preview (e.g. a truncated JSON string or item count). The fully expanded "Arguments" panel underneath renders the same data correctly as pretty-printed JSON, so the underlying data is present and correct — only the collapsed one-line preview is broken.

REPRODUCTION

1. Sign in as qa-admin, navigate to `http://localhost:3000/runs/b9feac1c-ec21-4531-8ba7-bb391786993e/steps`.
2. Via `browser_evaluate + .click()` (per the `stepsTabRowBlocked` quirk — normal clicks are intercepted by adjacent controls), expand the "Deck Engineer Agent" lane row.

1. Observe the "Tool calls 4" section auto-expanded, listing 4 calls. Three of the four are `write_todos` calls; each renders its collapsed row as:

```
write_todos todos: [object Object],[object ok
```

1. Expand the first `write_todos` call to reveal its "Arguments" panel — it renders correctly as pretty-printed JSON (`{ "todos": [ { "content": "...", "status": "in_progress" }, ... ] }`),

confirming the underlying data is intact and the defect is isolated to the collapsed summary line's own rendering logic.

1. Repeated on the second and third `write_todos` calls in the same panel — identical `[object Object], [object` text in all three collapsed rows.

1. For contrast, the sibling `write_file` call in the same list (whose argument is a plain

string, not an array of objects) renders its collapsed preview correctly:

```
write_file file_path: /tmp/kindred-pitch-deck., content: <!DOCTYPE html> <html
la ok.
```

EXPECTED

The collapsed tool-call summary should show a readable preview of any argument value — a truncated JSON string, item count, or similar — regardless of whether that argument is a string, array, or object.

ACTUAL

Array-of-object arguments (e.g. `write_todos`'s `todos` list) are coerced with JavaScript's default array/object stringification, producing the literal text `[object Object]`, `[object` in the collapsed summary row shown to the user, on every occurrence (3/3 in this run).

EVIDENCE

- Before (agent lane list, unexpanded): `bug-hunter/evidence/runs-id-steps/BUG-20260828-080047-runs-id-steps/01-before-agent-list.png`
- Failure (`write_todos` collapsed row showing `[object Object]`, `[object`, with the correctly-rendered "Arguments" JSON panel visible directly below for contrast): `bug-hunter/evidence/runs-id-steps/BUG-20260828-080047-runs-id-steps/02-failure-object-object-preview.png`

BROWSER SIGNALS

- Console: none observed (0 errors, pre-existing 1 warning unrelated to this interaction)
- Network: no request involved — purely a client-side rendering defect on already-fetched step data
- State/URL: stayed on `/runs/b9feac1c-ec21-4531-8ba7-bb391786993e/steps` throughout; reproduced on

all 3 `write_todos` calls within the "Deck Engineer Agent" row's Tool calls panel

ID `BUG-20260828-080047-runs-id-steps` Found by `bug-runs-id-steps-r2`

Fingerprint `/runs/[id]/steps|agent-detail-tool-calls-panel|expand-agent-row-with-array-object-tool-args|collapsed-summary-shows-literal-object-object-instead-of-readable-preview`

SUMMARY

Every file-size label on the Files tab (Run input, agent outputs, and — where content is text — the Final output row) is computed from the text content's `character/.length` count rather than its true byte size. Any file whose content contains multi-byte UTF-8 characters (this pipeline's LLM output uses em dashes — and right-arrows →) is displayed smaller than the file the Download button actually delivers.

REPRODUCTION

1. Sign in as qa-admin, go to `/runs/b9feac1c-ec21-4531-8ba7-bb391786993e/files`.
2. Note the declared sizes: Run input `prompt.md` "620 B"; agent outputs

"01-presentation-strategist-agent.md" "9.5 KB", "02-deck-engineer-agent.md" "55.1 KB",
"04-pptx-code-generator.md" "28.8 KB".

1. Click each row's Download button and measure the downloaded file with `wc -c` (true bytes):
`prompt.md` = 626 B; `01-...md` = 9845 B (9.62 KB); `02-...md` = 56564 B (55.24 KB);
`04-...md` = 29584 B (28.89 KB).

1. Compute the UTF-16/`.length`-style character count of the same content: 620, 9686, 56398, 29459 respectively — these match the UI's declared sizes exactly (after KB rounding), not the true byte counts.

1. Repeated on a second, independently downloaded file set from the same run — same pattern held for all four rows checked.

EXPECTED

The size label next to each file should reflect the actual byte size of the file that Download delivers (matching what `wc -c` / the OS reports for the downloaded artifact).

ACTUAL

The size label reflects the text's character count (UTF-16 code units), which is smaller than the true UTF-8 byte size whenever the content contains multi-byte characters — off by 6 bytes on the smallest file (prompt.md: 620 declared vs 626 actual) and by 100–200+ bytes on the larger agent-output files.

EVIDENCE

- Files tab with declared sizes: bug-hunter/evidence/runs-id-files/BUG-20260828-082706-runs-id-files/01-files-tab-declared-sizes.png
- Byte-vs-character-count comparison table and methodology: bug-hunter/evidence/runs-id-files/BUG-20260828-082706-runs-id-files/notes.md

BROWSER SIGNALS

- Console: none related; page loaded cleanly.
- Network: GET /api/runs/{id}/sandbox returns true byte sizes correctly (e.g.

conversation_history/...:ppt-code-generator.md = 319756, presentation.html = 54551) — the

backend has the correct byte sizes, so the mismatch is introduced client-side when the Files

tab computes/display the per-row size for text content instead of using the API-provided size.

- State/URL: reproduces identically on /runs/b9feac1c-ec21-4531-8ba7-bb391786993e/files via

both a hard reload and client-side SPA navigation into the tab.

ID BUG-20260828-082706-runs-id-files Found by bug-runs-id-files-r2

Fingerprint /runs/[id]/files|file-size-label|render-files-list-with-non-ascii-content|size-shown-is-utf16-char-count-not-byte-count

87 **LOW** **Diverted run's Workspace tab falsely claims "the deliverable is still on the Preview and Files tabs" when no deliverable was ever produced**
`/runs/{id}/workspace (diverted run)`

SUMMARY

A diverted run never produces a deliverable — confirmed via the backend API on all four diverted runs checked (940ca699..., 277bc03a..., 4dec89fb..., b15f23d0...): `output: null`, `deliverable_filename: null` in every case, the Preview tab shows the empty "Output will appear here" placeholder, the Files tab lists only agent-intermediate outputs and the run input (no deliverable file), and the toolbar's "Download the deliverable" button is correctly disabled. Despite this, the Workspace tab's "Workspace expired" empty state unconditionally states "The deliverable is still on the Preview and Files tabs — only the raw working files are gone," which is factually false for a diverted run: there never was a deliverable on either tab. The copy appears to be a generic completed-run message reused without a status check for the diverted case.

REPRODUCTION

1. Sign in as qa-admin, navigate to a diverted run, e.g.
`http://localhost:3000/runs/940ca699-b21b-4666-8e44-3370a08a4561`. Header badge reads "Diverted".
1. Confirm via GET `/api/runs/940ca699-b21b-4666-8e44-3370a08a4561`: `"output": null`, `"deliverable_filename": null`.
1. Click the Preview tab (default) — shows "Output will appear here" (no deliverable rendered).
2. Click the Files tab — lists 3 files: 2 agent-output intermediates + run input `prompt.md`; no deliverable file present. Toolbar "Download the deliverable" button is [disabled].
1. Click the Workspace tab — shows "Workspace expired" with body text: "Run workspaces are cleared after a retention period. The deliverable is still on the Preview and Files tabs — only the raw working files are gone."

1. Reproduced identically on a second, independent diverted run,

`http://localhost:3000/runs/277bc03a-d2ba-4405-876d-d0aa861bc9ed/workspace` — same message,

same confirmed-null deliverable state via API.

EXPECTED

The Workspace-expired message should not assert a deliverable exists on Preview/Files when the run's own data shows none was ever produced — either the message should be conditioned on whether a deliverable exists, or a diverted run should get copy explaining the handoff (e.g. pointing at the successor run) instead of the generic completed-run wording.

ACTUAL

The message unconditionally claims the deliverable is available elsewhere on the run, actively misleading a user who has not yet checked Preview/Files into believing output exists that was never generated.

EVIDENCE

- Workspace tab false claim (run 940ca699...): `bug-hunter/evidence/runs-diverted/BUG-20260828-090219-runs-diverted/01-workspace-tab-false-claim.png`
- Reproduced on second diverted run (277bc03a...): `bug-hunter/evidence/runs-diverted/BUG-20260828-090219-runs-diverted/02-repro-second-diverted-run.png`
- Files tab confirming no deliverable file exists: `bug-hunter/evidence/runs-diverted/BUG-20260828-090219-runs-diverted/03-files-tab-no-deliverable-present.png`

BROWSER SIGNALS

- Console: none observed related to this defect.
- Network: GET `/api/runs/940ca699-b21b-4666-8e44-3370a08a4561` and

GET `/api/runs/277bc03a-d2ba-4405-876d-d0aa861bc9ed` both return output: `null`,
`deliverable_filename: null`.

- State/URL: reproduced on `/runs/940ca699-b21b-4666-8e44-3370a08a4561/workspace` and `/runs/277bc03a-d2ba-4405-876d-d0aa861bc9ed/workspace`.

Fingerprint /runs/{id}/workspace|workspace-expired-empty-state|open-workspace-tab-on-a-diverted-run|message-falsely-claims-deliverable-exists-on-preview-and-files

Composer's per-agent Skills "View details" modal renders raw markdown syntax instead of formatted content

/workflows/new (agent config panel → Skills tab → "View details" on any skill)

SUMMARY

The skill's own detail page at `/library/skills/<id>` correctly renders the skill's markdown body as HTML (proper `<h2>/<h3>` headings, etc — confirmed live for `windows-desktop-e2e`, `html-deck-to-pptx`, `benchmark`, `accessibility`). But the same markdown body, shown in the composer's per-agent Skills tab via the "View details" (eye) icon on a skill card, is dumped as raw unrendered markdown text — literal `###` characters visible in the modal instead of headings. Confirmed on two different skills (`windows-desktop-e2e` and `.NET Backend Expert`), so this is a property of the preview-modal component, not one skill's content.

REPRODUCTION

1. Go to `/workflows/new`, click "Add agent", add any agent (e.g. Domain Discovery Agent) to the canvas, click the agent node to open its config panel.
 1. Click the "Skills" tab (`[role="tab"]` labelled Skills).
 2. Type a skill name into `input[data-testid='agent-skills-search']` (e.g. "Windows Desktop"), locate the matching skill card, click its "View details" icon-button (`[aria-label="View <skill name> details"]`).
1. Observe the modal that opens.
2. Repeat with a different skill (e.g. ".NET Backend Expert") — same result.

EXPECTED

The modal should render the skill's markdown body as formatted HTML — headings as `<h2>/<h3>`, lists as `<ul>/<li>`, etc — matching what `/library/skills/<id>` shows for the same skill.

ACTUAL

The modal shows the markdown source verbatim: lines literally start with `# / ##` characters, no heading elements exist in the DOM (`querySelectorAll('h1,h2,h3,h4').length === 0` inside the

dialog), while the identical content on the skill's own detail page renders proper heading tags.

EVIDENCE

- Before (skill's own detail page, windows-desktop-e2e, correctly rendered `<h2>/<h3>`): bug-hunter/evidence/library-skills-id/BUG-20260828-093400-library-skills-id-r2/01-before-library-detail-rendered-correctly.png
- Failure (same skill, composer "View details" modal, raw # markdown visible): bug-hunter/evidence/library-skills-id/BUG-20260828-093400-library-skills-id-r2/02-failure-composer-view-details-raw-markdown.png
- Reproduced independently (different skill, ".NET Backend Expert", same raw-markdown modal): bug-hunter/evidence/library-skills-id/BUG-20260828-093400-library-skills-id-r2/03-repro2-second-skill-raw-markdown.png

BROWSER SIGNALS

- Console: none observed
- Network: none relevant — content comes from the already-loaded `/api/skills/library` payload, no failed request
- State/URL: URL stays `/workflows/new`; modal is `[role="dialog"]`; `document.querySelector('[role=dialog]').querySelectorAll('h1,h2,h3,h4').length` is 0 in both reproductions, and `/^#{1,3} /m.test(dialog.innerText)` is true

ID `BUG-20260828-093400-library-skills-id-r2` Found by `bug-library-skills-id-r2`

Fingerprint `/workflows/new|agent-skills-view-details-modal|click-view-details-on-a-skill|markdown-hashes-shown-as-literal-text-not-rendered-headings`

`/library/hooks/<id> → close (X) → /library?tab=hooks`

SUMMARY

On the Library "Hooks" tab, typing a search query and/or selecting a category pill (e.g. "PostToolUse", "SessionStart") narrows the card list and updates the URL with `?category=<name>`. Opening a hook's detail view (a full-screen modal reached at `/library/hooks/<id>`) and then closing it with the X button does not return the user to the filtered state they left — it navigates back to the bare `/library?tab=hooks` with the search box cleared and the category pill reset to "All", showing all 8 hooks again. The active filter context is silently discarded by the round-trip into a hook's detail view.

REPRODUCTION

1. Go to `http://localhost:3000/library?tab=hooks`.
2. Type `session` into the "Search hooks" box, then click the "SessionStart" category pill.

URL becomes `/library?tab=hooks&category=SessionStart`; list narrows to one card ("Session Context Loader").

1. Click that card to open its detail view (`/library/hooks/session-start`).
2. Click the X (close) button in the detail view's header.
3. Observe the resulting state: URL is `/library?tab=hooks` (no category param), the search

box is empty, the "All" pill is active, and all 8 hook cards are visible again.

1. Reproduced a second time with a different search term ("quality") and category ("PostToolUse") filtering to 2 cards — same reset behaviour after opening "Quality Gate" and closing it.

EXPECTED

Closing the hook detail view should return the user to the Hooks tab exactly as they left it — same search text, same active category pill, same filtered card set (consistent with round-trip behaviour elsewhere in the app where returning from a detail view preserves list context).

ACTUAL

The search text and category filter are both discarded; the tab resets to its default

unfiltered "All" state every time a hook detail view is closed.

EVIDENCE

- Before (filtered to "Session Context Loader" via search=session + SessionStart pill):

bug-hunter/evidence/library-hooks-id/BUG-20260828-093757-library-hooks-id/01-before-filter-search.png

- After closing the detail view (search cleared, "All" pill active, all 8 cards shown):

bug-hunter/evidence/library-hooks-id/BUG-20260828-093757-library-hooks-id/02-after-close-reset.png

BROWSER SIGNALS

- Console: none observed
- Network: none relevant — purely client-side state loss
- State/URL: /library?tab=hooks&category=SessionStart → /library/hooks/session-start →

closes to /library?tab=hooks (category param and search box state both lost); reproduced

identically with category=PostToolUse + search=quality

ID `BUG-20260828-093757-library-hooks-id` Found by `bug-library-hooks-id-r2`

Fingerprint `/library/hooks/<id>|hooks-tab-search-and-category-filter|close-hook-detail-view|filters-and-search-text-reset-to-default`

Chat message timestamps in the Preview transcript silently reset to the current time on every page reload, never reflecting when the message was actually sent

/runs/b9feac1c-ec21-4531-8ba7-bb391786993e

SUMMARY

Sent a follow-up chat message on a completed run (Ppt V2, run b9feac1c-ec21-4531-8ba7-bb391786993e). The message and the assistant's reply render with a clock time next to each ("09:51 AM"), which looks like a normal sent/received timestamp. Reloading the page repeatedly shows this pair of timestamps changing on every single reload to match the browser's current wall-clock time at that reload — never staying fixed at the moment the message was actually sent. Cross-checked the backend: `GET /api/runs/{id}/events?after=N` returns the `chat_message/chat_reply` event payloads with no `created_at/timestamp` field at all, confirming the frontend has no real send time to render and is falling back to `Date.now()` on each mount. This is a different defect from the already-filed run-header "stuck on just now" bug (BUG-20260828-011700-runs-id): that one is a single frozen/wrong value that never updates; this one is not frozen at all — it changes to a new, always-wrong "now" value on every reload.

REPRODUCTION

1. Sign in as qa-admin, navigate to /runs/b9feac1c-ec21-4531-8ba7-bb391786993e (completed run, Preview tab).
2. Type a short message ("Can you make the title larger?") into the chat composer and click Send. A `POST /api/runs/{id}/messages` fires (200 OK); the message and an assistant reply appear in the transcript, each tagged "09:51 AM" (the real send time).
3. Reload the page (`http://localhost:3000/runs/b9feac1c-ec21-4531-8ba7-bb391786993e`). Observe: both timestamps on the same message/reply pair now read "09:52 AM".
4. Wait ~1 minute (browser time confirmed via new `Date()`: 09:56:32) and reload again. Observe: both timestamps now read "09:56 AM" — tracking the reload time, not the original 09:51 send.
5. Confirmed via `GET /api/runs/{id}/events?after=9737`: the `chat_message` and `chat_reply` event payloads carry no timestamp field whatsoever, so there is no persisted send time for the frontend to render.

EXPECTED

A chat message's displayed timestamp should reflect when it was actually sent and stay fixed across reloads (or the backend should persist a `created_at` on the event and the frontend should render that, not the current time).

ACTUAL

The timestamp recomputes to the browser's current time on every page load, so the same historical message shows a different, always-"just reloaded" time each time the page is revisited — actively misleading about when the exchange happened.

EVIDENCE

- Before (message sent, shows "09:51 AM" at actual send time): `bug-hunter/evidence/runs-id/BUG-20260828-095700-runs-id-chat-timestamp/01-sent-0951.png`
- Failure (same message, after reload ~5 minutes later, shows "09:56 AM"): `bug-hunter/evidence/runs-id/BUG-20260828-095700-runs-id-chat-timestamp/02-reload-shows-0956.png`
- Additional: `bug-hunter/evidence/runs-id/BUG-20260828-095700-runs-id-chat-timestamp/network.log` (events API excerpt showing no timestamp field in the event payload)

BROWSER SIGNALS

- Console: none observed (0 errors)
- Network: GET `/api/runs/{id}/events?after=N` payloads for `chat_message/chat_reply` events contain no `created_at/timestamp` field
- State/URL: reproduced across three separate full-page reloads (09:52 AM, 09:55 AM, 09:56 AM), each matching the reload's wall-clock time, never the original 09:51 AM send time

ID `BUG-20260828-095700-runs-id-chat-timestamp` Found by `bug-runs-id-r2`

Fingerprint `/runs/{id}|preview-chat-transcript-message-clock|reload-page-after-sending-chat-message|displayed-message-timestamp-jumps-to-current-load-time-instead-of-original-send-time`

SUMMARY

Every number displayed elsewhere on the Analytics page (KPI tiles, By Pipeline Type, By Model, Token Breakdown, Model Details) is formatted with K/M abbreviations (e.g. 34.2M, 554.0K, 93.2M). The Daily Activity bar-chart tooltip is the one exception: hovering a bar shows the raw, unformatted integer token count with no thousands separators and no K/M suffix (e.g. Aug 24: 2364478 instead of Aug 24: 2.36M, Aug 27: 24220106 instead of Aug 27: 24.2M). The underlying value is correct (verified against `GET /api/analytics/summary?range=all's daily[].total_tokens`), so this is purely a formatting/presentation defect, not a data-accuracy one — but it is jarring and inconsistent with the rest of the page's number formatting.

REPRODUCTION

1. Sign in as qa-admin, go to `/analytics`, range = "All".
2. Hover the bar labelled "Aug 24" in the Daily Activity chart.
3. Observe the tooltip text: Aug 24: 2364478.
4. Hover a different bar, e.g. "Aug 27": tooltip reads Aug 27: 24220106.
5. Compare against any other number on the same page (KPI tiles, By Pipeline Type list, By Model list) — all of those use K/M abbreviations and no bare 7-8 digit raw integers ever appear elsewhere on the page.

EXPECTED

The tooltip should format the token count the same way as every other number on the page, e.g. Aug 24: 2.36M (or at minimum 2,364,478 with thousands separators).

ACTUAL

The tooltip renders the raw JS number with no formatting: Aug 24: 2364478.

EVIDENCE

- Before (page with all totals correctly formatted, e.g. 34.2M, 93.2M): bug–
hunter/evidence/analytics/BUG-20260828-102900-analytics-r2/01-before-all-time-

formatted-totals.png

- Failure (tooltip on Aug 24 bar shows raw 2364478): bug-hunter/evidence/analytics/BUG-20260828-102900-analytics-r2/02-failure-tooltip-raw-number-aug24.png
- Failure (tooltip on Aug 27 bar shows raw 24220106, reproduced a second time): bug-hunter/evidence/analytics/BUG-20260828-102900-analytics-r2/03-failure-tooltip-raw-number-aug27.png

BROWSER SIGNALS

- Console: none observed
- Network: GET /api/analytics/summary?range=all returns `daily[].total_tokens: 2364478` for

Aug 24 and 24220106 for Aug 27 — the tooltip value is data-correct, only unformatted

- State/URL: /analytics?range=all

ID BUG-20260828-102900-analytics-r2 Found by bug-analytics-r2

Fingerprint /analytics|daily-activity-chart-tooltip|hover-a-bar|raw-unformatted-integer-instead-of-K-M-abbreviation

Whitespace-only API key name bypasses the "Default" fallback, creating a permanently blank/unidentifiable key entry

/handoff/settings

SUMMARY

The "API key name" field defaults to "Default" when submitted empty (`keyName || "Default"` in `IntegrationsCard.tsx`), but a name consisting only of whitespace (e.g. three spaces) is a truthy string, so it bypasses that fallback and is sent to the backend verbatim. The backend accepts it as-is and the resulting key renders in the "VelocityAI API keys" list with a completely blank name — no text at all where every other row shows a name like `Default` or `e2e-handoff-fixture`. Because names exist specifically so a user can identify which key is which for revocation, a blank-named key is permanently unidentifiable by name once created; the only way to distinguish it from other rows is a token prefix. This is a persistent, unfixable (from the UI) piece of malformed data in an already-crowded list (Lead 1: 45+ existing revoked entries with no way to rename or filter).

REPRODUCTION

1. Sign in as qa-admin, navigate to `/handoff/settings`.
2. In the "API key name" field, set the value to three spaces (" ") — e.g. via `input.value = " " + an input event`, since a real keyboard also produces this with the space bar.
 1. Click "Create key".
 2. Observe the new key appears at the top of the "VelocityAI API keys" list with an entirely blank name area (compare to any other row, which always shows a name string before the token-prefix line).
 1. Dismiss the "New key — copy now" banner and re-check the list: the blank-named row persists with no name, distinguishable from other rows only by its `flowin_...` token prefix.
 1. Revoke the key (cleanup) — the row still shows no name, now with a "revoked" badge instead, confirming the blank name is permanent, not a transient render glitch.

EXPECTED

Either the frontend trims the name before deciding whether to fall back to "Default" (so a whitespace-only submission also defaults to "Default"), or the backend rejects/trims whitespace-only names, so every key in the list always has a human-readable, non-blank label.

ACTUAL

A whitespace-only name is accepted verbatim by both the client-side fallback check and the backend, producing a key list entry with no visible name at all, forever.

EVIDENCE

- Before/failure (blank-named key at top of list, no name text where "bughunt-r2"/"Default" appear on other rows): `bug-hunter/evidence/handoff-settings/BUG-20260828-103900-handoff-settings-r2/01-before-blank-name-key-appears.png`
- After revoke (blank name persists, only the badge changes to "revoked"): `bug-hunter/evidence/handoff-settings/BUG-20260828-103900-handoff-settings-r2/02-after-revoke-still-blank-labeled.png`

BROWSER SIGNALS

- Console: none observed.
- Network: `POST /api/settings/api-keys` succeeds (201) with the whitespace name accepted as-is;

no validation error returned.

- State/URL: URL stays `/handoff/settings` throughout. DOM inspection of the name element confirmed `textContent` was the literal three-space string, not empty/undefined — i.e. this is genuinely stored and rendered whitespace, not a missing-data placeholder.

ID `BUG-20260828-103900-handoff-settings-r2` Found by `bug-handoff-settings-r2`

Fingerprint `/handoff/settings|velocityai-api-key-create|submit-whitespace-only-name|key-list-entry-renders-with-no-visible-name-label-forever`

Every bug was found by driving the real application in Google Chrome via Playwright, reproduced at least twice, and recorded with screenshots. Status of all entries is **Open** — this hunt observes and documents, it does not fix.